

Superscalar NPU Architecture

Eight Problems in Coordination, Representation, and Progress

Technical article for the EFCL Zurich workshop

Research and architecture snapshot: 7 October 2026
Review draft

Abstract

A matrix accelerator is useful only when its surrounding system can deliver ready operands, perform non-matrix work, retain the right values, and complete the resulting effects. This article develops eight architectural problems that arise in that surrounding system: matrix-vector coordination; latency hiding and storage lifetime; mixed precision, layout, and scale handling; irregular access and atomic updates; reductions along a tensor's tail axis; scalar and event orchestration; precise state and memory ordering; and forward progress under finite resources. For each problem, we examine the responsibilities assigned to algorithms, instruction sets, compilers, runtime software, and hardware in NVIDIA GPUs, Ascend, TPU, Tenstorrent dataflow machines, and a proposed Superscalar NPU. CPU SIMD and published Groq designs provide additional comparisons where their mechanisms illuminate a particular question. The central claim is conditional: bounded dynamic dependency and resource management can reduce coordination work or exposed waiting when useful independent work exists, but its benefit must cover the added state, arbitration, movement, recovery, and progress machinery. Exact numerical and ownership contracts constrain the admissible schedules. Public specifications, version-scoped implementation observations, and proposed mechanisms are kept distinct. This article supplies no new benchmark, silicon measurement, or physical power, performance, and area result.

Contents

1	The architectural question	2
2	Evidence and comparison method	4
3	U001 Hybrid matrix and vector coordination	6
4	U002 Latency hiding and physical storage lifetime	12
5	U003 Mixed precision layout and scales	18
6	U004 Irregular access sparse computation and atomic updates	23
7	U005 Tail axis reductions and the placement of partial state	30
8	U006 Scalar and event orchestration	39
9	U007 Precise faults and ordering across asynchronous engines	44
10	U008 Forward progress under finite resource constraints	50
11	Cross problem synthesis and an evaluation agenda	57
A	Reading and reproducing this article	59

1 The architectural question

The useful work of a neural processor is seldom one isolated matrix product. A matrix result may feed normalization, indexing, selection, format conversion, another matrix product, or communication with another device. These stages differ in execution width, latency, physical layout, numerical behavior, and ownership. Their interfaces can therefore determine an operator’s latency even when the individual arithmetic engines are fast. An architecture must decide when a value is ready, where it lives, who may overwrite it, and which agent can advance the next dependency. These decisions remain necessary whether they are expressed in a compiler schedule, a GPU kernel, a dataflow protocol, or a dynamic scheduler.

This article asks whether a bounded superscalar organization is a useful way to manage some of those decisions in an NPU. Here, “superscalar” denotes the possibility of identifying independent ready operations within a finite window and issuing them to available resources. The term does not imply an unbounded instruction window, a CPU-sized general-purpose core, unrestricted speculative memory execution, or a thread context for every tensor element. The proposed programming surface combines scalar control with shaped, typed Tiles. Hardware can track operation dependencies and the availability of finite execution and storage resources. A Tile is an architectural value with a descriptor and a logical valid region; it is not necessarily one indivisible physical transfer, execution step, or storage allocation.

The proposal must be judged against competent alternatives. GPU kernels can distribute control among threads and specialized warps, use asynchronous matrix and copy engines, and compile sophisticated software pipelines. Ascend exposes matrix/vector pipelines, local memories, queues, and events whose composition depends on the product. TPU combines compiler-directed placement with scalar, vector, matrix, and DMA facilities, including low-level programmable asynchronous pipelines. Tenstorrent makes data-movement and compute roles, buffers, and explicit communication central to the programming model. Static scheduling can eliminate substantial dynamic control when shapes, routes, and execution times are predictable. Dynamic scheduling can exploit readiness variation only when work is present and admissible. Neither fact establishes a universal winner.

1.1 A running example with explicit boundaries

For the dense example, let a tile of queries $Q \in \mathbb{R}^{M \times D}$ attend to keys $K \in \mathbb{R}^{N \times D}$ and values $V \in \mathbb{R}^{N \times D_v}$. The logical computation is

$$S = \alpha Q K^T + B, \tag{1}$$

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_{k=0}^{N-1} \exp(S_{ik} - m_i)}, \quad m_i = \max_{0 \leq k < N} S_{ik}, \tag{2}$$

$$O = PV. \tag{3}$$

Here B represents an explicitly defined bias or masking rule. The equations describe real-arithmetic intent. A complete implementation additionally chooses input, accumulator, and output types, rounding and exceptional-value behavior, a permitted reduction order, and handling of empty or fully masked rows. These choices affect both numerical results and the schedules that are legal.

The matrix stages produce and consume blocks of scores or probabilities. The middle stage performs reductions and elementwise functions, and it may require retaining scores, exponentials, or partial statistics. If M is small, the implementation has fewer independent rows with which to cover these dependencies. This is a stated low-concurrency case, not a claim that every decoding workload has low concurrency: batch size, heads, parallel sequences, splitting, and

device partitioning can all supply more work. Even with many rows, limited storage can prevent all of that work from being resident simultaneously.

Attention does not stand in for every workload. Indexed accumulation is used separately to study irregular access and collisions in Section 6. A producer–communication–consumer pipeline is used to study events and bounded resources in Sections 8–10. Their control and numerical requirements differ, so an optimization valid for a dense attention tile cannot automatically be transferred to a sparse update or a distributed protocol.

1.2 Eight questions rather than eight mechanisms

The questions organize the argument because each exposes a distinct obligation:

- U001** Which coordination granularity preserves useful overlap between matrix, vector, and movement stages?
- U002** During which part of a value’s lifetime must scarce payload storage actually be occupied?
- U003** Which format, layout, packing, and scale work remains after competent optimization?
- U004** Who handles data-dependent addresses, sparse structure, collisions, and the transition back to regular compute?
- U005** Where does a reduction exchange values and retain state, and what numerical order may it use?
- U006** Which agent advances scalar submission and completion events while compute is busy?
- U007** What state, attribution, visibility, and ordering contracts make asynchronous failures understandable?
- U008** Why can admitted work eventually finish when every queue, buffer, and context is finite?

Renaming, partial readiness, late binding, extra scalar contexts, or event offload are possible answers to parts of these questions. They are not premises from which a favorable comparison may be derived. In particular, a scheduling optimization that shortens one wait can lengthen a value’s storage lifetime, increase conversion traffic, or obstruct the agent needed to release capacity. The final evaluation must close those loops across the whole core.

1.3 The conditional thesis

The working hypothesis is that the gap between *semantic dependencies* and *conservative software coordination* can sometimes be usefully managed by bounded hardware state. If a consumer needs only a defined subset of a producer’s output, a correct partial-readiness contract may release it sooner. If a long-latency request need not immediately occupy its final compute storage, separate request identity and physical binding may reduce occupancy. If an independent operation is ready while another waits, resource-aware issue may preserve utilization without an explicit software role split.

Each proposition has a corresponding obligation. Partial readiness requires exact subvalue validity and alias rules. Delayed binding requires a feasible destination or return path. Dynamic issue requires dependency checks, finite queues, and fair selection. Completion requires safe publication and release. Precise recovery may need retained versions or delayed externally visible effects. The design is attractive only where these obligations can be met for less total cost than

the coordination or waiting they replace. This is an architectural argument and an evaluation agenda, not a measured claim of improvement.

2 Evidence and comparison method

2.1 Four kinds of statements

The article distinguishes four evidence levels. A *public contract* specifies software-visible legality, numerical behavior, ordering, or supported use. A *pinned implementation observation* establishes that a particular source revision contains a mechanism or lowering path, within its reviewed scope. A *reported measurement* belongs to the authors, configuration, and methodology of the cited study; it is not a measurement performed for this article. A *conditional design inference* explains what a mechanism could do if its stated assumptions hold. A future design alternative belongs to the last category until an implementation and validation establish more.

These levels are not interchangeable. An opcode in a specification does not prove a production compiler can emit it, that a model computes it accurately, that RTL implements it, or that silicon achieves a particular rate. A compiler recognizer’s limited coverage does not shrink an ISA’s architectural expressibility. A functional simulation establishes neither a cycle count nor a clock frequency. A timing model’s host execution speed is not the modeled machine’s performance. An area estimate for one selector is not a whole-core PPA result.

The Superscalar NPU discussion uses the public PTO contract where available and the bounded project-inspection records assembled with the workshop source packet. The timing-model and typed bring-up records concern separate implementation paths; they must not be combined into one demonstrated complete-core implementation. Their inspection supports functional statements about dependency, ownership, and recovery structures, rather than a physical block diagram with verified area or timing. Public documentation whose branch or `latest` path can change is access-dated; exact revisions are used for claims that depend on a particular contract.

The target architecture includes functionality in submitted integration work once that work is merged. A pending change is therefore not described as architecturally nonexistent merely because of its merge status. At the same time, target intent does not establish new-head validation. Date-stamped implementation notes are retained where they explain a real limitation; they do not define the thesis. No private implementation sources or source-location maps are reproduced.

2.2 Comparison across five layers

A comparison begins with the same logical workload and numerical contract, then records where each responsibility is placed.

Algorithm.

Define decomposition, independent work, sparse representation, reduction order, and required results. Algorithmic changes that reduce useful work or alter accuracy must be exposed rather than credited to scheduling alone.

Instruction set or programming contract.

Specify which operands, layouts, masks, types, atomics, synchronization scopes, and asynchronous lifetimes the interface permits. A high-level API can expand to many instructions; an ISA instruction can execute for many cycles.

Compiler and kernel.

Record tiling, vectorization, role assignment, layout propagation, prefetch distance, conversion

placement, and inserted synchronization. Compare optimized kernels, including transformations that eliminate or amortize apparent source-level overhead.

Runtime and libraries.

Include allocation, launch, communicator construction, buffer pools, event namespaces, library composition, device placement, and teardown. Setup can be amortized across repeated calls, but still matters for short or infrequent work.

Hardware.

Identify the execution, storage, dependency, arbitration, movement, and completion functions that must exist. An undocumented internal circuit remains unknown; it is not assumed absent.

For example, a direct Tile reduction and a warp shuffle sequence can express the same mathematical operation at different abstraction levels. Counting one mnemonic against several source operations measures interface compression, not necessarily execution cost. Conversely, placing an entire row in one thread can remove communication that a nominally wider primitive would otherwise need. Ownership and lowering matter as much as the operation name.

2.3 A whole-core cost ledger

Let a workload execution perform useful work W and take end-to-end time T . Useful throughput is W/T , with W defined consistently across the alternatives. Reported utilization must name its denominator: a matrix unit's peak, an instruction issue rate, a memory bandwidth, or another resource. A high matrix utilization can coexist with poor task latency or excess energy if additional work was introduced to keep the matrix unit busy.

A useful accounting record contains at least the following quantities:

$$\mathcal{C} = (T, W/T, S(t), D, C_{\text{ctrl}}, A, P, E, f, \mathcal{S}, \mathcal{L}). \quad (4)$$

Here $S(t)$ is storage occupancy over time, D is movement at each relevant interface, C_{ctrl} is control state and work, A is area, P is power, E is energy, f is attainable clock frequency, and $\mathcal{S}$ and $\mathcal{L}$ denote required safety and liveness properties. This tuple is a ledger, not a scalar objective with invented weights. The article can reason about functional obligations and derived byte counts without having measured every component. Unmeasured components must remain unmeasured.

For a conservative first-order performance bound, let W_r be the work offered to resource r and μ_r an upper bound on its service capacity in the stated analytical model and access pattern. A measured average rate alone need not bound short bursts. Ignoring setup and interference gives

$$T \geq \max_r \frac{W_r}{\mu_r}, \quad T \geq T_{\text{dependency}}. \quad (5)$$

These lower bounds do not imply that all resources can operate at their isolated peaks simultaneously. Shared ports, banking, issue bandwidth, and live storage can couple them. Overlap shortens exposed elapsed time only if those coupled constraints allow concurrent execution. It does not make the overlapped arithmetic, traffic, or energy disappear.

2.4 Control functions to account for

A dynamic core may require a front end, finite issue queues, dependency tags, rename mappings, generation counters, readiness bits, issue selection, storage admission, bank arbitration, and completion/retirement state. The inspected timing-model record includes examples of such functions but does not verify a particular Age Matrix selector or exact physical organization. A

CPU-style speculative memory subsystem is not assumed. The typed bring-up path has its own identity, binding, completion, and recovery contracts, which require separate validation.

The alternatives also contain control. GPUs maintain execution state, scoreboards, eligible-warp selection, barriers, and asynchronous pipeline ownership. Ascend APIs expose queues and events around hardware engines. TPU compilers plan lifetimes and DMA synchronization while hardware still advances transfers and execution. Tenstorrent controllers and circular-buffer protocols manage readiness and ownership. Published deterministic streaming designs still need instruction execution, routing, timing alignment, and synchronization. It would be misleading to compare named blocks on one side with an empty box labeled “static” on the other.

The important question is which control can be eliminated by static knowledge, which is amortized over larger work, which is hidden behind independent activity, and which remains on the critical path. These categories are used throughout the chapters. They allow a favorable case to be stated without claiming that its mechanism has no cost.

3 U001 Hybrid matrix and vector coordination

A matrix engine can finish useful work only as quickly as the surrounding computation supplies its inputs and consumes its outputs. Attention makes this dependence particularly visible: a matrix multiplication produces scores, vector and reduction operations turn the scores into probabilities, and another matrix multiplication consumes them. An accelerator therefore needs more than a fast matrix datapath. It needs an execution contract that connects different operation shapes, readiness conditions, storage owners, and completion events without imposing avoidable waiting. The question in U001 is which granularity exposes useful overlap, who manages that overlap, and what the resulting control costs are.

This is a mechanism comparison. The NVIDIA case is principally Hopper and a pinned FlashAttention-3 kernel; a separate paragraph identifies a relevant Blackwell change. Ascend A2 and the newer 3510 architecture are kept distinct. The TPU case uses the public Pallas programming model rather than an invented description of a proprietary instruction scheduler. Tenstorrent is considered through its documented compute and circular-buffer interfaces. Our superscalar NPU, abbreviated SSNPU, is discussed through the supplied source-inspection record of a particular timing model, not as measured silicon. Published Groq designs provide a further comparison point. None of the mechanisms described here establishes a performance ordering between platforms.

3.1 The algorithm creates several different dependencies

For one query block, let $Q \in \mathbb{R}^{B_q \times d}$ and let the j th key and value blocks be $K_j \in \mathbb{R}^{B_k \times d}$ and $V_j \in \mathbb{R}^{B_k \times d_v}$. The score block is

$$S_j = \alpha Q K_j^T + M_j, \quad (6)$$

where M_j expresses the selected mask. A convenient online formulation retains a row maximum m_j , a row denominator ℓ_j , and an unnormalized output accumulator O_j . With rowwise broadcasting understood, one iteration is

$$m_j = \max(m_{j-1}, \text{rowmax}(S_j)), \quad (7)$$

$$a_j = \exp(m_{j-1} - m_j), \quad P_j = \exp(S_j - m_j), \quad (8)$$

$$\ell_j = a_j \ell_{j-1} + \text{rowsum}(P_j), \quad (9)$$

$$O_j = a_j O_{j-1} + P_j V_j. \quad (10)$$

For rows with valid scores, the recurrence starts from $m_0 = -\infty$, $\ell_0 = 0$, and $O_0 = 0$, with the first valid block handled consistently with those initial values. The final output divides each

row of O_j by its corresponding denominator. This mathematical formulation does not prescribe a particular instruction schedule or numerical implementation. Input precision, accumulator precision, approximation of exponential, exceptional values, and the treatment of fully masked rows must be specified separately.

There are at least three kinds of dependence in this small program. Within a block, probabilities require the appropriate maximum, and the contribution to the output requires those probabilities. Across blocks, the running state connects successive iterations. Across query rows or independent heads, substantial work can be independent. These distinctions explain why both of the following statements can be true: a score block cannot skip its softmax dependencies, and a kernel can execute some matrix work concurrently with softmax work. FlashAttention-3 explicitly exploits asynchronous matrix execution and rearranged block scheduling to obtain such overlap on Hopper. Its reported performance is evidence for its evaluated configurations, not a transferable result for the SSNPU or for every attention shape. [114]

The decomposition also prevents a misleading account of “the vector stage.” Row maximum, exponential, sum, broadcast, output rescaling, and normalization have different input requirements. A row maximum can accumulate partial maxima as score fragments arrive, but its final value requires every active fragment in that reduction domain. An elementwise operation with independent inputs may become eligible much earlier. The relevant dependency is therefore a relation between producer regions and consumer operations, not merely an arrow between two large boxes labelled matrix and vector.

3.2 Five granularities must be distinguished

We use five separate meanings of granularity. *Semantic work granularity* is the operation the program expresses, such as a score tile or row reduction. *Dispatch granularity* is the unit admitted or selected by a scheduler. *Readiness granularity* is the smallest region for which the implementation can establish that required data are valid. *Synchronization granularity* is the participating group and event scope used to communicate this fact. *Transfer granularity* describes a memory request or movement unit. These quantities can differ within one operation and can change between generations of the same product.

For example, the supplied benchmark inspection identifies a group score tile with $B_q = B_k = 128$, but the selected four-PE kernel gives each PE a 32×128 FP32 score shard. The group FP32 payload is 64 KiB, the per-PE payload is 16 KiB, and a modeled vector readiness cell is 128 bytes. A cell contains 32 FP32 values by capacity; it is neither the whole vector operation nor evidence that those values execute in one cycle. Doubling B_k doubles the shard’s logical byte count without proving that dispatch or readiness granularity changes. These are source-derived geometry observations and arithmetic, not dynamic instruction counts. [98, 119]

A corresponding NVIDIA example must name its own layer. A full warp executing one FP32 operation per active thread spans 128 bytes per operand when all 32 threads participate. Hopper WGMMA instead uses a warpgroup of 128 cooperating threads. For a dense BF16 $m64n128k16$ operation, the logical inputs contain 2 KiB of A and 4 KiB of B, while an FP32 accumulator contains 32 KiB. These figures describe matrix shapes and representations, not memory transactions or a universal software tile. [69] Equal byte counts between an SSNPU cell and a warp operand establish no equivalence between their scheduling semantics.

The historical Ascend attention design also identifies an operator whose vector piece is 8×1024 FP32, or 32 KiB. Its significance is a buffer and decomposition choice in that operator, not a fixed hardware asynchronous scheduling quantum. [34] Comparing this piece directly with a GPU warp or a 128-byte readiness cell mixes semantic work, capacity, and execution participation. A meaningful comparison first identifies the producer region, the required consumer region, and the actual operation that moves or consumes it.

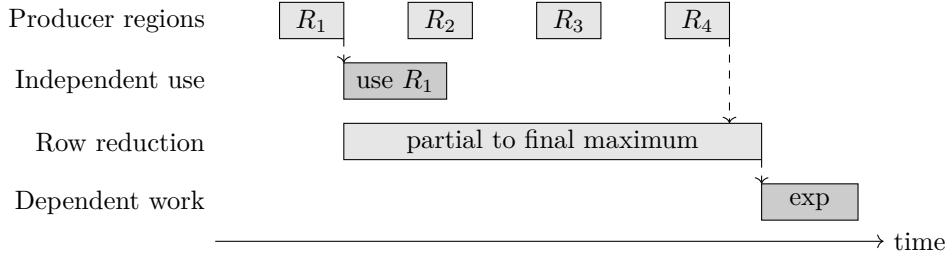


Figure 1: Conceptual readiness relationships. Region arrival can advance independent work and partial reduction, while the final maximum still gates dependent exponentials in this formulation. Horizontal distances are illustrative and do not represent measured timing or a particular architecture.

3.3 What finer readiness can and cannot change

Consider an illustrative producer that makes four independently usable regions available at times $t_1 < t_2 < t_3 < t_4$. A consumer requires only the first region. If the interface exposes one whole-tile completion event, the earliest dependency-permitted start is t_4 . If it exposes correct region readiness, that bound becomes t_1 . The opportunity is $t_4 - t_1$, but the actual improvement can be smaller or zero: the execution unit may be occupied, a destination may be unavailable, or bank conflicts may prevent admission. This is a dependency argument, not a prediction of observed cycles.

For a consumer v , write its earliest admissible start schematically as

$$s_v \geq \max\{r_v, d_v, e_v, b_v, o_v\}, \quad (11)$$

where r_v is the latest required input-ready time, d_v is destination availability, e_v is execution-resource availability, b_v is the relevant bank or port availability, and o_v expresses ordering constraints. Replacing whole-tile readiness with region readiness changes r_v only. A scheduling proposal that ignores the other terms can substantially overstate its benefit.

Now make each region one quarter of a softmax row. Partial maxima can begin early, but exponentiating every quarter relative to the final block maximum cannot begin merely because the first quarter exists. One may use an alternative online update scheme, but then its additional rescaling and retained state belong in the accounting. Alternatively, make each region a complete subset of rows. Those rows can complete their reductions without waiting for unrelated rows, provided the producer really finishes the corresponding reduction dimension and the consumer mapping supports that partition. Fine readiness is valuable when it matches the algorithm’s independence; an arbitrary byte partition does not create such independence.

This distinction also matters for low-concurrency workloads. Let N be the number of useful pipeline iterations, T_{fill} the initial preparation interval, I the achievable steady-state initiation interval, and T_{drain} the remaining work after the final iteration is admitted. An idealized latency decomposition is

$$T(N) = T_{\text{fill}} + (N - 1)I + T_{\text{drain}}, \quad N \geq 1. \quad (12)$$

Reducing I has little leverage when $N = 1$. Reducing a real dependency on the first or last tile can instead affect fill or drain. Conversely, splitting work more finely can increase setup, events, and intermediate storage. A claim about short-task latency must state batch size, heads, sequence lengths, dimensions, masking, and partitioning; it cannot be inferred from a steady-state utilization number or generalized to all decode.

3.4 NVIDIA assigns roles in software and eligibility in hardware

Hopper exposes asynchronous matrix and transfer facilities that a compiler or kernel author can compose into a fused pipeline. TMA moves appropriately described tensor data, while warps and warpgroups perform different producer and consumer roles. Hardware schedules eligible resident work and enforces exposed completion mechanisms; software determines tile shapes, layouts, stage count, role assignment, and synchronization placement. The Hopper tuning guide describes the hardware facilities, while a concrete kernel establishes how they are used. [68]

CUTLASS’s Hopper organization makes this division explicit. Producer warps wait for an empty shared-memory stage, initiate a transfer, and associate arrival with the stage’s completion state. Consumer warps wait for the stage to become full, issue matrix work, and release the stage when it can safely be overwritten. [73] This is already dynamic readiness-based coordination at the software-defined stage boundary. It should not be described as a machine that relies entirely on a fixed cycle schedule, nor should ready-warp scheduling be equated with unrestricted CPU-style speculative out-of-order execution.

The useful opportunity is to overlap independent stages and to use different engines concurrently. The cost includes producer instructions, barrier state, register and shared-memory capacity, layout preparation, and constraints on resident work. More software stages can improve latency tolerance while reducing other forms of concurrency. An application-level runtime can also select different decompositions: TensorRT-LLM documents multi-block attention choices intended to provide additional work when batch-times-head count is small. That option introduces its own partial-result and coordination costs; it is a relevant baseline for a low-concurrency comparison, not a guarantee of lower latency. [71]

The generation boundary is important. Blackwell SM100 introduces the `tcgen05` family with single-thread matrix instruction launch and Tensor Memory for accumulators. [70] FlashAttention-4 describes algorithm and pipeline changes for that organization. [152] Therefore the number of threads participating in Hopper WGMMMA cannot be presented as an immutable property of NVIDIA matrix execution. A fair comparison may choose Hopper for its pinned implementation evidence, but it must say so rather than treating an older interface as the only available GPU approach.

3.5 Ascend exposes pipeline and inter-core coordination

In the A2 separated organization, Cube and Vector cores have independent scalar control. The documented architecture distinguishes Cube operand and accumulator storage from the Unified Buffer used by Vector work; movement engines connect the relevant storage levels. Its instruction queues and synchronization interfaces permit overlap across pipelines while preserving dependencies. GM-addressed exchange is not synonymous with a mandatory HBM round trip because the architecture documents L2 caching of DMA traffic. [14]

At the algorithm and compiler level, host tiling and kernel code choose which Cube region becomes which Vector pieces, which layouts are used, and how much state remains resident. At the API level, queues and events identify ownership and inter-pipeline dependencies. Hardware executes those operations and supplies the movement and event mechanisms. A larger Cube tile can amortize setup and reuse operands, while smaller Vector pieces can begin useful work incrementally. Neither choice is free: exchanged data, local capacity, events, and the number of simultaneously live intermediates constrain the resulting schedule.

The `TQue` contract is especially relevant because it separates a queue of tensor references from the number of physical buffers allocated for that queue. Queue depth and multibuffering count answer different questions; increasing one does not automatically increase the other. [17] This is a concrete example of why an API-level queue size cannot serve as a universal hardware issue

granularity. Similarly, an operator’s 32 KiB vector allocation is not evidence that every useful handoff waits for 32 KiB.

For architecture 3510, the official specification identifies additional L0C-to-UB and UB-to-L1/L1-to-UB paths, as well as SSBuffer for AIC–AIV communication. It also documents SIMD register-based execution and SMT hardware, including warp schedulers, so an A2 description must not be treated as the execution model of every Ascend generation. [33] These changes alter the possible handoff and control costs. The present comparison does not project A2’s paths onto 3510, infer throughput from the existence of a path, or assume that every software release uses it optimally. The open measurement question is which decomposition sustains useful overlap for the selected shapes after exchange and control costs are included.

3.6 TPU gives the compiler a larger scheduling responsibility

Pallas describes a TPU TensorCore with scalar, vector, and matrix execution resources that can operate asynchronously under compiler management. The programmer sees a predominantly single-threaded program and can express DMA, memory spaces, and pipeline structure. [145] This organization separates an instruction stream’s presentation from the concurrency of the engines it controls. A sequential kernel body does not imply that every transfer, matrix operation, and vector operation is serialized to completion.

The compiler and kernel author choose blocking, layout, fusion, and the order of work; explicit buffer and semaphore operations establish important readiness conditions. The documented constraints on blocks and memory spaces, and the distinction between vector and matrix operations, matter when mapping a score tile to reductions. [144] A pipeline that overlaps HBM-to-VMEM DMA with computation does not by itself establish that the score matrix multiply overlaps every softmax stage. That requires an effective schedule and sufficiently independent work on the relevant engines.

A static description of that schedule should not be caricatured as an inability to handle runtime variation. Pallas supports scalar-prefetched information for runtime block indexing in documented sparse patterns. [146] Runtime completion and buffering can handle lateness by waiting rather than by corrupting data. The likely tradeoff, as a design inference, is between predictable scheduling with relatively explicit lifetimes and the ability to exploit unanticipated ready work. Which side matters more depends on shape regularity, memory behavior, the compilation, and available local capacity. It is not determined by the label “compiler directed.”

3.7 Tenstorrent separates placement from readiness

Tenstorrent’s Metalium programming model divides work among data-movement and compute roles. RISC-V control processors issue commands; the tensor arithmetic is performed by the corresponding compute engines. Circular-buffer operations coordinate publication, consumption, and available space. [120] Software placement of kernels and streams therefore coexists with runtime readiness and backpressure. A static placement is not necessarily a cycle-by-cycle static execution schedule.

Within Tensix, the matrix engine and vector engine meet through Dst storage. Unpack, math, and pack activities have explicit ownership synchronization; the documented `tile_regs_*` lifecycle separates acquiring compute access, committing results, waiting for packing access, and releasing the resource. [121] The implication for attention is that local ownership can be as important as nominal engine independence. Two operations assigned to different arithmetic facilities may still serialize because their command role, Dst region, or packing path is shared.

The official repository’s investigation of SDPA implementation paths distinguishes a non-streaming FP32-Dst path, where the math-thread exponential serializes with matrix work,

from a streaming path that issues exponential work from the pack thread. [143] This is useful implementation-specific evidence, not proof that all Tensix kernels either overlap or fail to overlap matrix and vector work. The algorithm must still provide independent blocks; the kernel must provide compatible Dst ownership and command sequencing; and L1 circular buffers must support the required run-ahead. Host asynchronous dispatch or replay cannot establish any of those internal overlap properties by itself.

The published Groq TSP designs show a further possible division of responsibility: compiler placement and timing arrange streams between functionally specialized slices, with network synchronization maintaining the timing contract. [1, 2] This can make predictable streaming attractive, but it does not remove the attention dependencies derived above. These published designs are a comparison of contracts, not a description of every current Groq product or a low-concurrency performance verdict.

3.8 The SSNPU opportunity is bounded dynamic coordination

The supplied SSNPU audit covers the handwritten TimingSim snapshot at commit `d5419b3e721d`. It observes logical operand tags, whole-tile readiness, conditional cell readiness, bounded issue queues, resource checks, and physical binding. CUBE selection is gated by dependence, chain order, speculation, and execution or storage resources. VEC selection is organized by execution class and is subject to initiation intervals, destination capacity, and backpressure. Cell-mode vector block entry retains an ordering restriction. [119] These facts support a bounded dynamic coordination design; they do not describe unrestricted global out-of-order execution.

The intended division is that software expresses operation geometry, layouts, logical dependencies, and reuse, while the hardware model enforces eligible readiness and resource admission. Conditional cell readiness can lower r_v in Equation 11 for supported producer-consumer combinations. It cannot lower the other admission bounds automatically, and an operation requiring whole-tile readiness remains constrained by that contract. The audit does not establish arbitrary nonlinear layouts, arbitrary independent source shapes, or an automatically optimal compiler decomposition.

The corresponding normative design requirement is precise: every early consumer must read the correct generation of every required region, no consumer may observe an unfinished or overwritten value, and issue must respect both program ordering and physical resource ownership. This requirement is independent of the particular mechanism used to implement it. Logical tags, scoreboards, readiness masks, bank reservations, and queue selection are implementation choices that carry state, wiring, arbitration, and verification obligations. An abbreviated source program cannot demonstrate that their total cost is lower than that of explicit software stages.

The target architecture should also be kept distinct from a narrow executable snapshot. Functionality delivered by a submitted change belongs in the after-merge target when the change is integrated; its absence from an earlier experiment does not establish architectural inexpressibility. Conversely, a proposed interface does not prove that the measured path used it. For U001, this distinction requires the exact build and configuration before attributing any overlap to cell readiness, ready bypass, or a particular binding path. TimingSim observations are not evidence of complete RTL integration, clock closure, physical area, or energy.

3.9 A useful evaluation isolates the source of overlap

A matched study should retain the same attention semantics and vary query rows, key length, head dimensions, masking, and available independent heads. It should record semantic tiles, per-owner shards, readiness regions, synchronization participants, and transfer units separately. End-to-end latency should include preparation, fill, steady state, and drain. A steady-state

engine-utilization trace should identify what useful work overlaps, rather than merely showing two queues nonempty.

For the SSNPU, a readiness-granularity ablation must preserve the rest of the build, resource budget, scheduling options, and numerical path. Otherwise a change attributed to smaller regions may instead come from different reduction ordering, queue policy, or storage capacity. Competent GPU, Ascend, TPU, and Tenstorrent baselines should use their own supported fusion and pipelining facilities. The resulting ledger must include event and issue work, retained payload, extra transfers, mapping and arbitration state, and correctness and progress constraints. Finer granularity is justified when it advances useful admissible work enough to repay those costs; neither finer readiness nor a larger matrix tile is intrinsically better.

4 U002 Latency hiding and physical storage lifetime

Latency hiding requires work to be outstanding before it is needed. It does not follow that every outstanding operation must occupy its eventual compute destination for the whole wait. The distinction is the subject of U002: when must a physical payload location be owned, what state must remain while it is not owned, and when is reuse safe? Compiler lifetime planning, software multibuffering, explicit dataflow queues, and hardware-managed logical-to-physical binding are different answers to this question. Their comparison requires separate accounting for requests, returned data, allocation pools, individual values, and genuinely live compute state.

This distinction is easy to lose in attention. A kernel can prefetch a future K block while retaining Q, running maxima and denominators, an output accumulator, and other K/V blocks being consumed. Some storage contains useful live data; some is reserved for a future arrival; some exists because the enclosing kernel allocated a fixed pool. Reclaiming one consumed K slot does not free the pool, and making part of a score tile readable does not establish that its physical storage can be released one cell at a time. The relevant objective is to reduce avoidable residency while retaining correct ownership and a completion path for every admitted operation.

4.1 Five intervals describe one asynchronous load

For a value i with logical payload size b_i and charged physical allocation size s_i , distinguish the following intervals. *Request tracking* begins when the system must remember an operation's identity and ends when the associated tracking state is no longer required. *Returned-data residence* covers a response that has arrived but has not yet been placed in or consumed from its next storage location. *Pool allocation* is the interval during which a larger capacity reservation exists. *Value ownership* is the interval during which a particular slot belongs to value i and cannot be overwritten by another value. *Compute-live residence* covers valid payload held in compute-local storage that is still required by remaining consumers. Returned data waiting elsewhere are also logically live; this last category isolates their residency after placement in compute storage. The intervals can overlap. The exact boundaries depend on the interface; these categories are an accounting framework rather than a universal implementation diagram.

Let a_i be physical destination acquisition and f_i safe release. For distinct, non-aliased allocations, let s_i include the physical envelope, padding, and capacity rounding charged to the value. The destination occupancy attributable to values is

$$D(t) = \sum_i s_i \mathbf{1}_{[a_i, f_i)}(t), \quad A_D = \int D(t) dt = \sum_i s_i (f_i - a_i), \quad (13)$$

where A_D is a byte-time measure. It exposes long avoidable holding intervals, but it is not an area estimate. Physical implementation also depends on peak simultaneous demand, banking, ports,

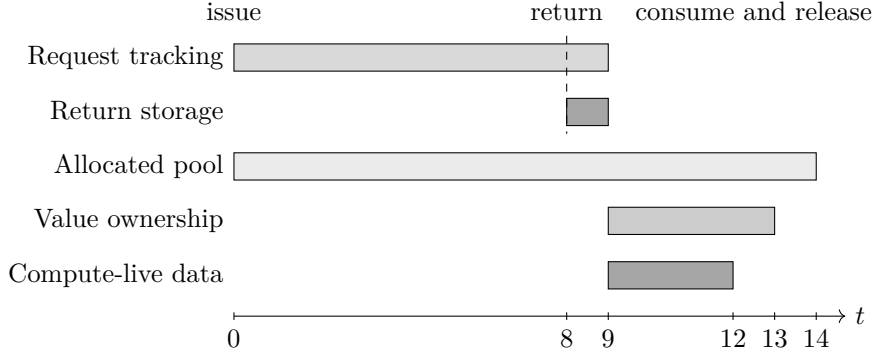


Figure 2: An illustrative late-binding lifecycle. The response arrives at $t = 8$, destination ownership begins at $t = 9$, the last modeled computation consumes the data by $t = 12$, and ownership is released at $t = 13$. The enclosing pool remains allocated until $t = 14$. These times are invented for explanation; tracking may last longer in other implementations.

fragmentation, and whether different pools can share capacity. If a fixed pool has capacity C , slot reuse can reduce value occupancy within it while leaving the allocated capacity C unchanged.

Similarly, request count $N_{\text{req}}(t)$ is not payload capacity. A request record might hold an address, length, tag, generation, and status rather than a complete data tile. Return buffers can have their own allocation and backpressure rules. If the same physical memory backs several roles, their occupancy must be counted as a union of physical regions rather than summed twice. If they are disjoint, each region’s capacity and lifetime belongs in the ledger. Neither “bytes in flight” nor “number of buffers” is sufficiently precise by itself.

For the example in Figure 2, suppose the logical payload and charged allocation are both 16 KiB. An early-bound destination held from $t = 0$ through $t = 13$ accounts for 208 KiB-time units; a destination acquired at $t = 9$ accounts for 64. The difference, 144 KiB-time units, is an illustrative reduction in that destination’s reservation interval. It is not a claim of saved SRAM area or energy. The late-bound design also requires return storage during $[8, 9)$, may temporarily hold two copies during movement, and needs mapping and arbitration state. If its enclosing pool is unchanged, physical capacity may be identical. If binding is delayed until $t = 11$, the consumer may start later; the smaller destination interval can then accompany worse latency. A residency result must therefore be reported together with completion time and occupancy elsewhere.

4.2 Outstanding work and live payload impose different bounds

For a steady stream of fixed-size requests with average request rate λ and average request latency L , the average outstanding count is λL under the usual stable-flow assumptions. At a delivered byte rate B , the associated bandwidth–latency product is BL . For variable request sizes, let ℓ be per-request latency, with $L = \mathbb{E}[\ell]$. Mean outstanding bytes are $\lambda \mathbb{E}[b\ell] = BL_b$, where $L_b = \mathbb{E}[b\ell]/\mathbb{E}[b]$ is byte-weighted mean latency. This is a statement about outstanding traffic, not proof that BL bytes occupy a particular SRAM. Bytes may still reside at the source, be traversing a network, wait in return storage, or have arrived at their destination. The architecture determines which of these states requires an explicit payload reservation.

An idealized pipeline producing one tile every I time units needs roughly $\lceil L/I \rceil$ requests in flight to cover a fixed latency L , before additional constraints are considered. Burstiness and variable service can require more slack or force stalls. A finite queue cannot guarantee a target throughput for arbitrary unbounded delay. Equally, adding destination buffers cannot overcome a sustained bandwidth deficit: if the consumer needs b/I bytes per unit time and the supply can deliver only $B < b/I$, eventual starvation remains.

There is also an independent lower bound from truly live values. For a chosen algorithm and schedule without spilling or recomputation, all values needed by future consumers must remain available somewhere. If $\mathcal{L}(t)$ is the set of such values, then storage across the locations retaining them must accommodate their live payload, with aliases counted once. Late binding can remove an unnecessary pre-arrival destination hold; it cannot remove the information content of $\mathcal{L}(t)$. A different algorithm may reduce this set through streaming, recomputation, or changed accumulation, but its extra compute and traffic must then be charged to that algorithm.

This is why the running state in Equation 10 matters. A K block can often be released after its last relevant QK use, while an output accumulator and row statistics remain necessary across subsequent KV blocks. Treating all of these as identically short-lived tiles exaggerates what an allocator can reclaim. The meaningful question is which values are held before they are present, after their final use, or in unnecessarily duplicated form, and whether shortening those intervals changes a binding capacity constraint.

4.3 NVIDIA stage reuse is a complete ownership protocol

In a Hopper TMA pipeline, shared-memory stages are software-selected destinations. The producer obtains a reusable stage, starts the copy, and allows transfer completion to establish readiness; the consumer waits, uses the data, and releases the stage for another iteration. [73] Copy completion and consumer completion are different events. Reusing a stage after the first but before the second can overwrite an operand still being read by asynchronous matrix work.

The pinned FlashAttention-3 source illustrates this distinction directly. At revision `dc8fd708575a`, the K producer acquires its pipeline stage before issuing the TMA copy. In the selected intra-warpgroup-overlap loop, QK for the current block is interleaved with PV for an earlier block. The loop waits for the appropriate matrix progress before releasing K, performs online softmax, then waits for the remaining matrix work before releasing V on the ordinary no-QV path. [11] The source establishes distinct K and V lifetimes. It does not establish that every template instantiation uses the same branch or stage geometry.

The compiler and kernel own the stage layout, number of stages, and consumer-release placement; hardware transfer and completion facilities implement the asynchronous operations. TMA’s direct movement avoids an intermediate general-purpose-register copy on supported paths. [72] That saves a particular movement and register role, but does not establish late physical allocation of the shared-memory destination. Register budgeting is another separate mechanism: PTX’s `setmaxnreg` permits supported redistribution within the CTA register pool. [74] It would be inaccurate to describe all register ownership as immutable, or to treat register redistribution as shared-memory late binding.

For a matched experiment, the costs include stage capacity, barriers and phase bookkeeping, producer and consumer instructions, resident register requirements, and waiting for reusable stages. More stages can absorb additional latency while reducing the room for other resident work. Private request and response-buffer policies cannot be reconstructed from this API description; they remain unspecified in this comparison. A software-managed stage is a real solution to safe latency hiding, with a different ownership boundary from logical-tag binding.

4.4 Ascend separates pool creation from tensor reuse

Ascend C’s `InitBuffer` establishes buffers associated with a `TPipe`-managed queue or buffer object. The queue form accepts both buffer count and per-buffer length. [16] A tensor allocated from this capacity acquires a slot; queue operations pass its ownership reference between stages. The `TQueue` description explicitly distinguishes queue depth from the number of buffers and describes dequeue as returning a tensor reference rather than copying the data. [17] Thus queue occupancy,

pool capacity, and payload movement are separate quantities.

FreeTensor releases a tensor previously obtained through **AllocTensor**. [15] Its name must not be read as proof that the entire physical pool disappears. It permits reuse within that pool; the encompassing management object has a separate lifecycle. At the algorithm level, the kernel decides when a tile’s remaining consumers have finished. At the API level, queue and event operations express the necessary handoffs. Hardware movement and execution pipelines perform the underlying work, subject to the chosen generation’s storage and transfer paths.

This division supports streaming and multibuffering, but it also places real obligations on the kernel: a queued address must continue to identify the intended data, a stage must not free an input still used by another asynchronous operation, and the next producer must not overwrite a live slot. Buffer count can increase tolerance of variable delays at a capacity cost. Whether a particular operator reduces its pool size or only reuses slots more frequently requires its actual tiling and allocation plan. The public APIs do not establish an issue-time or response-time policy for every hidden hardware buffer.

4.5 TPU compiler planning can shorten residency

Pallas exposes VMEM buffers and semaphore-backed asynchronous movement. Its TPU pipeline documentation allows per-argument multibuffering and currently documents two buffers as the default for inputs and outputs. [145] That default is a software-interface fact, not a universal TPU hardware capacity. The asynchronous-copy interface separates completion associated with reading the source from completion associated with writing the destination. [147] Neither event, alone, says that all downstream consumers have relinquished ownership.

Compiler management should not be equated with reserving every buffer for a worst-case delay. OpenXLA’s memory-space-assignment description, inspected at revision 82e6c3119a94, represents a value through allocation sequences and use segments, including prefetch, eviction, and sliced-copy allocations. It explicitly describes sliced copies as a way to defer destination allocation. [149] This is a compiler-side mechanism for changing residency intervals. Its presence does not prove that a particular Pallas kernel receives that transformation, or that a target compilation selects the best lifetime plan.

Runtime completion remains relevant even with a planned schedule. The Pallas distributed-computing documentation explains how additional slots permit bounded run-ahead and how receiver-to-sender synchronization prevents overwriting a buffer when devices progress at different rates. [148] The same distinction between correctness and performance is useful locally: completion and ownership rules prevent a race, while lookahead and buffer depth seek to prevent an idle engine. If a transfer is late, a correct program may wait. A performance plan based on expected latency need not allocate for every conceivable delay in order to remain correct.

The tradeoff is therefore more precise than “static versus dynamic memory.” A compiler can shorten a known live range and schedule just-in-time prefetch; a runtime completion mechanism can tolerate variation by delaying progress; hardware-managed binding can choose an acquisition point after observing an event. Each technique may reduce a different form of conservatism. Their costs include planning complexity, explicit synchronization, extra movement, delayed issue, or mapping state. A common workload and generated executable are needed to determine which of these costs dominates.

4.6 Tenstorrent makes streaming ownership explicit

Tenstorrent’s host circular-buffer interface allocates L1 capacity for the participating cores. [122] Within a kernel, reserving space, publishing produced tiles, waiting for available input, and popping consumed tiles describe a value lifecycle inside that capacity. The documented reserve

operation waits for sufficient free space without itself advancing the producer pointer. [124] Data arrival does not automatically publish a tile, and a read does not automatically reclaim it. The kernel must perform the matching operations at a safe point.

A representative reader sequence is to reserve an output slot, issue an asynchronous NoC read to that slot, wait for the relevant transfer completion, and publish the tile. The compute side waits for published input and eventually releases the consumed capacity. [120] This exposes both readiness and bounded producer run-ahead. It also exposes the cost of a slow consumer: once the circular buffer is full, the producer waits even if it could otherwise issue more requests. A deeper buffer trades L1 capacity for additional tolerance.

The Wormhole B0 NoC counter specification supplies a particularly useful distinction. A command-control slot can become reusable when the first-hop virtual channel is assigned, while read completion is recorded only after the response data have been written to the return location. [123] Command-slot lifetime and payload lifetime therefore differ. This is evidence against treating every outstanding request as a permanently occupied command register, but it is not evidence for a generic late allocator of L1 payload destinations. Likewise, cut-through network movement should not be modeled as a complete tile reservation at every router without a source establishing that organization.

4.7 SSNPU logical identity can precede physical binding

The supplied TimingSim inspection observes a distinction between a Tile’s virtual identity, including generation, and its physical binding. Ordinary issue-time binding coexists with a conditional real-L2 local-load path that requests destination binding after returned-data readiness; shared staging uses a different path. A failed bind waits for capacity. Bounded request and grant machinery, physical references, consumer checks, aliases, and recovery boundaries participate in safe allocation and release. [119] These observations concern the pinned model, not every load or an automatically enabled property of every benchmark.

The architectural opportunity is to keep the dependency identity alive while deferring ownership of a scarce compute-local destination. Software still describes the operation, geometry, reuse, and program dependencies. Hardware-model control can decide when the relevant physical capacity is available, connect the returned value to its logical name, and wake supported consumers. This moves some ownership coordination below the kernel API, but it does not eliminate the ownership protocol. The state now includes mapping and generation information, bind arbitration, reference or alias checks, and the handling of responses that outlive a cancelled or superseded use.

Safe release is more demanding than “the first consumer read the tile.” If the value has two consumers, releasing after the first can corrupt the second. If aliases refer to the same physical region, freeing one logical name can invalidate another. If recovery needs a version or an operation still references a source, speculative apparent last use may be insufficient. These examples explain the purpose of guarded release; they do not assert that a single reference-count rule is a complete correctness proof. Similarly, cell readiness in U001 permits eligible partial consumption but does not establish cell-by-cell physical allocation or reclamation.

The crucial future evaluation is whether the conditional late-binding path reduces a capacity that actually limits useful work. It may allow more logically outstanding requests without reserving as many destination tiles, but returns still need somewhere to wait, and admitted work still needs a guaranteed route to completion. If return buffers grow by the displaced amount, if binding contention delays consumers, or if a larger mapping structure changes timing, the net result may differ from the destination-only result. Neither a default configuration flag nor the existence of a binding function establishes the outcome of a run.

4.8 Programming examples must be compared at matching levels

The pinned SuperNPUBench attention kernel at revision 349c98964144 expresses loads, matrix multiplies, row maximum, exponentiation relative to a row maximum, and row sum through Tile operations. Q is loaded outside the KV loop; running statistics and output state persist across blocks. The operand relationships expose the algorithm’s dependences without an explicit empty/full barrier around every source-level operation. [98] This is evidence about the source interface. C++ lexical scope is not, by itself, the physical allocation or release contract.

There is also explicit synchronization at another level. The benchmark’s result-checking helper publishes inputs and waits for the participating PEs before exporting outputs. [99] These verification-wrapper waits should neither be hidden nor misrepresented as the production inner-loop protocol. A separately inspected Tile API revision provides a lower-level emission path, but it has not been established as the exact dependency used to build the selected executable. [118] The article therefore does not combine the benchmark source, a different API checkout, and a model snapshot into one proven end-to-end build.

The GPU counterpart has visible stage acquisition and release in its lower-level pipeline. Comparing those lines with only the highest-level Tile calls would count an abstraction boundary as a reduction in complexity. A fair comparison can instead use high-level kernels on both sides, or inspect matching lowered instructions and lifetime state on both sides. Useful measures include control operations per tile, simultaneously live stages, error-prone ownership conditions, recovery behavior, and the effort to change shape or numerical mode. Source-line count alone measures none of these reliably.

4.9 Admission and release must compose into progress

Deferring allocation introduces a liveness obligation. Suppose returned values occupy every return slot while waiting for destination capacity, and the work that would release that destination capacity cannot issue until another blocked return is processed. Backpressure prevents overwrite, yet the wait dependencies can still form a cycle. This is a hypothetical failure pattern, not an observed bug in a named architecture. Its purpose is to show why a storage optimization must identify which admitted operations hold which resources and which runnable operation can release them.

A design can address such risks with appropriate admission bounds, reserved drain capacity, compatible resource ordering, independent completion service, or other explicit invariants. These are alternatives to be proved for the actual queues and execution paths, not recommendations to add every mechanism indiscriminately. Fair arbitration also matters: absence of a circular wait does not guarantee that an old binding request will eventually win. Section 10 develops the distinction between safety, deadlock freedom, starvation freedom, and bounded latency.

The final accounting for U002 should therefore report allocated pool capacity, per-value destination occupancy, return occupancy, request and mapping state, peak live payload, and end-to-end completion together. Compare unchanged workloads and numerical semantics under predictable and variable latency, including the capacity-limited boundary where a proposed lifetime reduction should matter. Record whether a change eliminates a reservation, shortens it, transfers it to another structure, or merely tolerates more delay. Compiler planning, GPU stages, Ascend queues, TPU semaphores, Tenstorrent circular buffers, and SSNPU binding all solve parts of the same ownership problem. The design opportunity is to remove unnecessary holding intervals while preserving the complete correctness and progress contract.

5 U003 Mixed precision layout and scales

A matrix accelerator’s ability to multiply a low-bit-width operand does not by itself explain how that operand is produced, represented, or delivered. An attention kernel may form scores in FP32, keep a softmax intermediate in BF16, and feed a subsequent product with packed FP4 values and separately encoded scales. A weight-only kernel may store INT4 weights while presenting a floating-point operand to its matrix engine. These paths have different numerical meanings even when the stored payloads occupy the same number of bits. The relevant question is therefore: after competent compilation, which numerical conversions, scale computations, layout changes, and producer–consumer ownership changes remain, and who performs them?

This question connects storage lifetime in Section 4 to reduction execution in Section 7. Here the concern is the representation presented to a reduction or matrix operation; the scheduling and throughput of its internal arithmetic are separate. The comparison includes NVIDIA GPU, Ascend, TPU, Tenstorrent, x86 AVX, Arm NEON/SVE, and the proposed superscalar NPU (SSNPU). Product generations, public APIs, virtual instructions, and implementation observations are identified explicitly. None of the examples is a matched performance measurement.

5.1 Six contracts behind a low-precision tensor

An implementation must preserve six related contracts. The first is the logical tensor: indices, valid extents, and reduction axes. The second is physical storage: strides, blocking, padding, and swizzles. The third specifies the bit positions of subbyte values. The fourth assigns values to threads, registers, banks, or local-memory cells. The fifth defines numerical input, intermediate, accumulator, and output types, including rounding, saturation, and exceptional values. The sixth maps each value to its scale and, where applicable, zero point. A layout name only describes part of this information. A storage reinterpretation can change a view without performing the quantization expressed by an arithmetic conversion. MLIR’s quantization dialect makes the distinction between an expressed value and its storage representation explicit [61].

For an illustrative symmetric integer quantizer, let G_g be a group of source elements and choose a positive scale s_g . With a signed b -bit storage range $[q_{\min}, q_{\max}]$, the operation is

$$q_i = \text{clip}_{[q_{\min}, q_{\max}]}(\text{round}_{\rho}(x_i/s_g)), \quad \hat{x}_i = s_g q_i, \quad i \in G_g. \quad (14)$$

The rounding rule ρ , scale selection, and treatment of an all-zero group are part of the algorithm. An asymmetric quantizer adds a zero point, and a floating-point quantizer replaces the integer rounding operation with rounding to a specified floating-point format. Neither is obtained merely by changing the destination’s bit-width annotation. In particular, integer INT4, E2M1 FP4, and a block-floating representation with shared exponents are not interchangeable.

The scale’s position relative to a reduction also matters. Per-row scales for A and per-column scales for B permit

$$C_{ij} \simeq s_i^A s_j^B \sum_k q_{ik}^A q_{kj}^B. \quad (15)$$

If scales change along the reduction dimension, with blocks K_g , the corresponding expression becomes

$$C_{ij} \simeq \sum_g s_{ig}^A s_{gj}^B \sum_{k \in K_g} q_{ik}^A q_{kj}^B. \quad (16)$$

There is generally no single final multiplier that can replace these block-specific products. Native scaled matrix instructions can place this work inside the matrix execution path, subject to their legal formats and scale layouts. They do not make scale production, transport, or representation disappear.

5.2 A worked preparation path and its storage accounting

Consider an illustrative 32×64 output fragment with FP32 accumulated values and a packed four-bit destination. Ignoring alignment and padding, the accumulator payload is $32 \cdot 64 \cdot 4 = 8192$ bytes, while the packed payload is $32 \cdot 64/2 = 1024$ bytes. A one-byte scale for every row and every 64-column group contributes 32 more bytes; a one-byte scale per 32-column group contributes 64 bytes. These are payload counts, not a claim that either scale policy achieves a particular accuracy. A BF16 intermediate for the same fragment occupies another 4096 bytes if it is materialized. It may overlap other lifetimes or be eliminated by fusion, but the comparison must establish which occurs.

The pinned SuperNPUBench HiF4 level-one quantization example provides a concrete preparation sequence. It starts from FP32 matrix results, uses BF16 intermediates, derives and encodes E6M2 scales, applies broadcast scaling, assembles two 32×32 fragments into a 32×64 Tile, and converts to packed HiF4X2 output. The two logical values per byte affect output address calculations. The scale carrier also has its own physical descriptor geometry: one meaningful scale per row does not imply that its allocated carrier has exactly one physical column. This is a source observation for level-one quantization, rather than evidence for an entire multilevel HiF4 algorithm [100].

Three operations in this sequence should not be collapsed into the word “shuffle.” Assembly joins logical fragments and coordinates their lifetimes. Conversion changes numerical representation and packs the result. A transpose or layout-aware transfer changes where values reside. The last two may sometimes be fused, and assembly can sometimes be avoided by choosing a different producer decomposition, but removing one does not prove that the others are free. In the separate low-precision attention example, probabilities are converted into E2M1x2 fragment slots with E8M0 scales for 32-column groups before assembly for the *PV* product. That scale contract is different from the HiF4 example’s E6M2 scale per 64 columns [101].

This distinction also prevents a misleading bandwidth argument. Reducing the final payload from 8192 to 1024 bytes does not guarantee an eightfold reduction in total traffic. Scale traffic, index or descriptor traffic, intermediate reads and writes, padding, and format-specific movement may all contribute. Conversely, charging every conceptual intermediate as an external-memory round trip is equally unfair: a compiler can keep intermediates local, fuse producers and consumers, or remove their materialization altogether.

5.3 The baseline is an optimized representation pipeline

A meaningful baseline first propagates compatible layouts through producers and consumers. If two representations differ only in a view or in thread-local register naming, conversion need not move data. If a value must move between distinct owners, however, the implementation needs an exchange mechanism. On a GPU this can be a register shuffle or shared-memory exchange; in other machines it can be a vector permutation, local-memory movement, or a rearrangement in an engine’s input path. Triton’s layout model is useful precisely because it records ownership rather than assuming that a tensor’s logical dimensions determine its physical distribution [150].

Fusion and amortization are separate optimizations. A fused matrix epilogue can apply scaling, clamping, and conversion while accumulator data are available. CUTLASS documents the relationship between accumulator distribution, epilogue exchange, and efficient output stores [76]. Static weights may also be prepacked into an instruction-friendly order and reused for many invocations. Prepacking is advantageous only after its setup and additional stored representation are charged to the reuse interval. A transient activation with a new dynamic scale each invocation offers a different opportunity.

Vectorization and unrolling introduce another tradeoff. A compiler can combine conversions

with packing, select a consumer-compatible arrangement, and use unrolling to expose independent work or reuse a shared load. It must also account for live-register growth, spills, code size, and the profitability of mixed-type operations. LLVM’s vectorizers explicitly distinguish legal transformations from profitable ones [60]. A semantic Tile interface can reduce decomposition performed solely to match a fixed physical width; it does not remove reuse-driven unrolling, algorithmic blocking, or the need to choose numerical representations.

The accounting vocabulary should consequently be precise. A removed redundant conversion is *eliminated*. A static packing pass reused over many executions is *amortized*. A conversion overlapped with independent matrix work is *hidden*, but still occupies arithmetic, ports, and storage. A necessary scale reduction or ownership exchange is *remaining*. Combining these outcomes into one instruction-count reduction obscures where the actual resource savings occur.

5.4 NVIDIA: packed conversion, fragment ownership, and native scaling

PTX exposes packed floating-point conversions, but is a virtual ISA rather than final SASS. A supported FP32-to-E4M3x2 conversion produces two FP8 values in a 16-bit carrier; supported FP32-to-E2M1x2 forms similarly pack two FP4 values. The rounding and finite-saturation modifiers define numerical behavior, while the input must already carry the intended scaling. Packing two values does not establish a complete matrix-operand layout transformation. Hopper WGMMMA offers FP8 matrix paths with prescribed operand layouts and FP32 accumulation; its sign controls must not be mistaken for arbitrary quantization scales [77].

Blackwell adds native block-scaled paths with explicit restrictions. The SM100-oriented CUTLASS documentation distinguishes MX formats using UE8M0 scales for 32-element groups from NVFP4 using UE4M3 scales for 16-element groups. These scale values require the prescribed storage organization. SM120 paths are separately described, so the SM100 `tcgen05` interface is not a generic instruction description for all Blackwell products [75]. Libraries can implement scale policies on hardware without an identical native operand form; the proper comparison is the resulting optimized computation, not the presence or absence of a particular mnemonic.

Ownership remains the central cost question. Matrix instructions determine a fragment distribution; a following row reduction or output store may prefer another distribution. Software and the compiler choose tiling, warp roles, shared-memory layouts, and synchronization. Hardware supplies instruction execution, warp readiness, register and memory access, and asynchronous completion mechanisms. A compatible producer–consumer layout can avoid an exchange, while an incompatible one may require movement and additional live registers. The potential advantage of native scaling is largest when its group size, scale representation, and operand layout already match the algorithm. If they do not, the conversion into that native representation belongs in the cost of using it.

5.5 Ascend and TPU: legal conversion paths and compiler-selected precision

Ascend’s public interfaces expose distinct conversion, transfer, matrix, and postprocessing responsibilities. The CANN 9.1 `Cast` tables are generation-specific: A2/A3 include half-to-INT4 conversion, but the table is not evidence of a direct FP32-to-INT4 operation. Newer 950 tables add other combinations, including BF16-to-packed-FP4. A staged conversion must preserve the intended rounding behavior; FP32-to-BF16-to-FP4 can differ from a single rounding to the final format. Cast mask and repetition parameters depend on the larger element width, and subbyte operands impose paired-mask and count constraints [19].

On the 950 family, `MmadMx` supplies a different contract: supported FP4 pairs or supported FP8 pairs, E8M0 scale data, and FP32 result storage. Scales are grouped along K in sets of 32. The documented matrices and scale buffers have specific physical arrangements and alignment

constraints; independently supported FP4 and FP8 families do not imply every mixed-family pair [20]. The kernel or compiler still chooses its decomposition and stages data through the relevant UB/L1/L0 resources. Hardware executes the vector, transfer, Cube, and postprocessing operations. Format legality, local capacity, events, and scale loads bound the available overlap. API documentation is sufficient to establish these responsibilities without pretending to reveal an undocumented raw encoding. The public 3510 architecture preview additionally describes SIMD register-resident intermediates and SIMT hardware with warp schedulers. It also describes Fixpipe channel merging/splitting and supported layout conversion. These generation-specific mechanisms reinforce that staging and packing should be assessed on the selected path, rather than assuming every intermediate returns to UB [18].

TPU’s Pallas matrix-multiplication tutorial illustrates BF16 input data, an FP32 accumulator reference, and conversion to BF16 output [44]. These are three different choices. Requesting an FP32 result or accumulator does not alone establish multiplication of unrestricted FP32 operands at full FP32 precision. Pallas precision controls govern how supported operations are lowered, with generation-dependent arithmetic behavior [43]. Public compiler IR reveals matrix and conversion intent, not final machine instructions.

Current experimental interfaces broaden the set of explicitly controlled matrix paths: the recorded JAX sources include FP8 cases on newer TPU generations and INT8/INT4 FIFO-oriented cases on earlier supported generations. These are scoped compiler/API observations, not evidence that every installed kernel supports every combination [42]. The general ownership split is that the compiler chooses supported precision, layout, and scheduling; the runtime supplies buffers and launch context; and hardware executes matrix, vector, and transfer work. Scale materialization, VMEM intermediates, padded blocks, and conversion remain real costs. A compile-time array shape is compatible with runtime values and addresses, an important distinction revisited in Section 6.

5.6 Tenstorrent: storage formats do not directly name matrix arithmetic

Tenstorrent’s unpack–math–pack organization makes representation boundaries particularly visible. Data in L1 are unpacked into source registers, matrix or vector work writes destination registers, and packing returns results to L1. Input format configuration and output packing configuration are distinct controls. Destination accumulation width, including a supported FP32 destination mode, does not by itself specify full-FP32 multiplication. Circular-buffer ownership and engine synchronization surround these operations [125].

For the documented Wormhole path, BFP8 and BFP4 are block-floating storage representations with shared exponents over groups of 16 values. Unpacking interprets those values for the supported matrix input paths. BFP4 must therefore not be substituted for E2M1 FP4 or ordinary INT4 simply because all use a four-bit label somewhere in their representation [127]. Rounding can occur at both input and output boundaries, and fidelity phases are additional execution work. The documented matrix instruction’s address and source controls rely on accompanying format and engine state [126].

The algorithm chooses whether the format’s error and exponent-sharing behavior are acceptable. The compiler and kernel choose tiles, format transitions, buffer lifetimes, and the unpack/math/pack schedule. The hardware provides those engines, destination storage, and movement control. The benefit is strongest when data arrive in a reusable supported format and adjacent operations retain compatible engine state. Frequent transitions, poorly filled tiles, or long-lived destination values can instead consume control bandwidth and capacity. This account does not infer support or absence of unrelated low-precision formats on every Tenstorrent generation.

5.7 x86 and Arm: fixed width, scalable width, and narrowing placement

Fixed-width vector instructions make element count and destination occupancy visibly different. On an appropriate x86 AVX-512 target, `VCVTPS2PH` converts 16 FP32 values in a ZMM source into 16 FP16 values in a YMM result. The operation preserves 16 logical elements rather than filling a 512-bit narrow result. Other supported sequences can convert FP16 to integer words and then saturating-narrow to bytes, but that is integer conversion rather than FP8 quantization; scaling and range policy are still required [40].

Packing order can matter as much as width. For AVX2 `VPACKSSWB`, two 16-word inputs A and B produce bytes in 128-bit-lane groups:

$$[A_0 \dots A_7, B_0 \dots B_7, A_8 \dots A_{15}, B_8 \dots B_{15}]. \quad (17)$$

A consumer expecting all of A followed by all of B requires rearrangement, whereas a compatible consumer can retain this order. The existence of a reduction intrinsic likewise does not prove that one machine opcode performs it. The compiler's opportunity is to propagate useful lane arrangements across the entire computation rather than repeatedly normalize them [39].

Arm NEON supplies fixed 128-bit vectors. Its narrowing operations distinguish the lower and upper portions of a destination; paired narrowing instructions can combine two sources. The numerical conversion and scale formula remain separate [5]. SVE/SVE2 instead support vector-length-agnostic programming with predicates and implementation-dependent vector length. This changes loop portability and tail handling, but does not guarantee that each conversion fills every destination position.

SVE floating-point narrowing can preserve the source element positions in wider slots, while SVE2 bottom/top integer narrowing places results in alternating narrow positions. A consumer may use that arrangement directly or require a permutation. Narrowing stores provide another route: once numerical range and representation are correct, storing the low byte from each wider element can avoid materializing a fully packed vector register [3, 4]. These are concrete alternatives to assuming that every width change requires a fixed packing sequence. They also explain why vector-length agnosticism cannot be equated with free conversion or guaranteed useful-lane occupancy. Ordinary SVE's one-dimensional vectors and reductions are distinct from a two-dimensional Tile-axis contract; SME's ZA is a separate architectural interface.

5.8 SSNPU: typed geometry shifts decomposition into bounded microsteps

The SSNPU proposal exposes supported typed, shaped Tiles and lets the backend sequence physical work. A Tile's logical extent can change without changing the instruction encoding length. Physical capacity, valid rows and columns, and layout legality remain distinct: runtime valid extent does not create unlimited runtime storage. The inspected Linx API carries dtype, layout, geometry, and discrete capacity choices, while its conversion emitter encodes source type and destination representation/rounding fields [55, 56].

The pinned API's CUBE cell is 128 bytes. For a cell with R rows and b bits per element, its column capacity is

$$C_{\text{cell}} = \frac{128 \cdot 8}{Rb}. \quad (18)$$

Thus M32 cells hold one, two, four, or eight columns at 32, 16, 8, or 4 bits respectively; M16 cells hold twice those column counts. This is physical geometry arithmetic, not a throughput estimate. Logical columns must be padded to legal cell boundaries, with capacity rounding in addition. The four-bit M16 mapping also includes a local column-group arrangement, so a naive row-major nibble formula is not a universal address rule [56].

Typed row and column operations can communicate an axis without making software enumerate every physical lane. However, the inspected row/column sum interfaces do not expose an independent accumulator-width argument. An FP32 matrix accumulator elsewhere in the program cannot be used to infer widened accumulation for an FP16 reduction. Typed axis APIs also exist in other systems; the design opportunity is their integration with the chosen storage, dependency, and issue model rather than ownership of the concept itself [55].

At the recorded TimingSim revision, vector operations are split into cell-sized steps. Widening can map one source cell to multiple destination writes, while narrowing targets packed destination cells across source steps. The review also records a remaining whole-cell-overwrite issue in the narrowing value model and absence of numerical TCVT execution in that inspected compute path. Consequently, sequencing and footprint evidence cannot establish packed-value correctness. These are implementation observations at revision `d5419b3`, not restrictions inferred for the entire architecture and not RTL or silicon validation.

The proposed benefit is a reduction in explicit width decomposition and some producer-consumer coordination for supported operations. The algorithm still owns numerical policy and scale groups; the compiler still owns legal layouts, blocking, and profitable fusion; the runtime still provides the execution context and memory resources. Hardware or its model must manage cell sequencing, tags and generations, readiness, capacity admission, destination assembly, bank access, and release. Larger semantic Tiles reduce neither the number of values that must be converted nor the amount of live payload by definition. They can help when a compatible multi-stage pipeline would otherwise expose substantial width and ownership bookkeeping; they can be neutral or costly when extra control, padding, or layout transitions dominate.

5.9 What a convincing comparison must establish

A comparison should begin with one numerical task and its acceptable error, then state storage, matrix operand, accumulator, and final output types independently. It should specify scale groups and scale encoding, conversion behavior, exceptional values, physical layout, and valid tails. The optimized path should identify which intermediates disappear, which formats are reused, which operations overlap, and which remain on the critical path.

The resulting ledger includes bytes and lifetimes, scale and conversion work, movement between owners, register or local-buffer pressure, padding, and control state. A lower visible instruction count is useful only in relation to those quantities. No source considered here supplies a matched quantitative advantage for the proposed SSNPU, and no evidence supports a ranking of compiler effort, utilization, or PPA across these architectures. The defensible claim is narrower and more useful: typed geometry and native format support can relocate preparation work and coordination, while numerical meaning and physical representation continue to constrain every implementation.

6 U004 Irregular access sparse computation and atomic updates

Irregular work couples data-dependent addresses to control, storage ownership, and synchronization. An embedding lookup reads rows selected at runtime. A histogram updates bins whose destinations can collide. A routing or top- k stage may use an atomic counter's old value to allocate an output slot, then pass the collected values to a regular vector or matrix computation. These operations cannot be understood from peak matrix throughput, but neither do they require one universal execution model. The central question is who prepares addresses, groups traffic, resolves collisions, and establishes that the next consumer can safely proceed.

Three capabilities must remain separate. Structured sparse matrix computation consumes a

supported sparsity pattern and its metadata inside a matrix path. Unstructured indexed memory performs gathers or scatters at runtime-selected addresses. Collision-sensitive updates add an atomicity and ordering contract. A machine may offer all three through different engines or APIs; support for one is not evidence for another. In particular, ordinary scatter overwrites destinations, a returning atomic read–modify–write can provide an observed old value, and a no-return atomic reduction updates memory without producing such a result. Histogram construction is also different from scanning a histogram whose bins have already been built.

For SSNPU, the target discussed here includes the intended integration of scalar control, typed and shaped Tiles, indexed and masked memory, and supported atomic operations. Submitted compiler or model coverage is evidence about reachability and implementation, not the architectural ceiling. The argument has three levels: expressibility of a workload, convenience of programming it, and measured efficiency. The first two do not establish the third.

6.1 A worked example: counts, slot allocation, and output publication

Let eight candidate elements have bin indices

$$b = [3, 1, 3, 7, 1, 3, 2, 7], \quad m = [1, 1, 1, 0, 1, 1, 0, 1], \quad (19)$$

where m_i indicates participation. A count-only histogram computes

$$H_j^{\text{new}} = H_j^{\text{old}} + \sum_{i:m_i=1} \mathbf{1}[b_i = j]. \quad (20)$$

For initially zero bins, the nonzero results are $H_1 = 2$, $H_3 = 3$, and $H_7 = 1$. Six active requests target only three distinct destinations. Replacing atomic increments with independent load/add/store sequences is incorrect under collisions: several requesters can observe the same old value and overwrite one another’s increments. Ordinary scatter of six ones is also not a histogram, even if a particular implementation chooses a predictable duplicate winner.

A slot-allocation variant requires more information. Each active element executes an atomic fetch-add of one on its bin and uses the returned old value o_i as an index into that bin’s output segment:

$$o_i = \text{fetch_add}(H_{b_i}, 1), \quad P[\text{base}(b_i) + o_i] = x_i. \quad (21)$$

If requests to bin 3 serialize in some legal order, their returned values are 0, 1, and 2, assigned according to that order. Atomicity alone does not assign these values in increasing lane order. If the algorithm requires stable ordering, it needs an additional construction, such as an ordered local prefix within a group. Stability across groups additionally requires assigning their segment offsets in the required group order; unordered atomic reservation alone does not provide that guarantee. A count-only no-return reduction cannot directly substitute for fetch-add when the individual old values are required.

Nor does reservation imply publication. The counter can reflect a reserved slot before its payload store completes. A consumer using that counter as a ready count needs a correct publication protocol, with ordering and synchronization in the relevant memory scope. This is an example of the distinction between atomicity, visibility, completion, and reuse developed in Section 9. The example assumes sufficient output capacity and no counter overflow; neither property is created by the atomic instruction.

There are several legitimate algorithms. Direct atomics preserve each request. Local grouping can reduce count updates to one addition per distinct destination. Privatized histograms defer contention to a merge. Sorting and segmented reduction can improve locality at the cost of preprocessing and temporary storage. If old values are used for slot allocation, an aggregated reservation can still work, but it must distribute the reserved interval among contributors and preserve any required ordering. The comparison must charge this additional work rather than assume that a count-only optimization automatically preserves every downstream use.

6.2 Locality and duplicates define different bottlenecks

Locality concerns how many transfers are needed; duplication concerns whether updates compete for the same state. For a batch of n active addresses, a simple idealized transfer model counts L distinct cachelines and a duplicate model counts U distinct update destinations. Neither L nor U determines the other. Many distinct counters can share one line and contend in its memory path, while repeated reads of one location may be cheap if the system can reuse fetched data. A coalescer can combine traffic without changing the semantics of multiple atomic effects.

For independent uniformly distributed bin choices among B bins, the expected number of distinct bins is

$$\mathbb{E}[U] = B \left[1 - \left(1 - \frac{1}{B} \right)^n \right]. \quad (22)$$

This follows by summing the probability that each bin appears at least once. It is an illustrative distribution model, not a benchmark or a predictor for skewed real workloads. A local grouping algorithm might reduce n count updates to U , but first pays to identify and combine duplicates. Under a hot-bin distribution the potential reduction can be large; with mostly distinct destinations it may be small. The optimal batching window also depends on scratch capacity, latency requirements, and whether downstream consumers need results before the full batch arrives.

The minimal accounting therefore includes index and mask preparation, address arithmetic, useful data bytes, transferred bytes, request setup, local grouping, synchronization, return values, and final merging. Wide indices can dominate narrow payloads: a four-byte index accompanying a one-byte value costs more logical bytes than the value itself before any alignment overhead. Mask density changes useful work, but inactive lanes still influence whatever index, mask, and descriptor representation the chosen interface requires. Dynamic scheduling may overlap eligible requests; it cannot erase a hot-address dependency chain or manufacture locality.

6.3 GPU execution: flexible threads and optimized aggregation

CUDA threads naturally form runtime addresses, take data-dependent branches, and issue supported atomic operations. Their programming model is particularly convenient for independent long-running loops and divergent state machines. A warp can also cooperate to aggregate repeated destinations, or blocks can maintain private histograms and merge them later. The CUDA 13.0.3 contract distinguishes atomic operand types and scopes: traditional unsuffixed atomics have device scope and relaxed ordering, while floating-point addition and other operations have architecture-specific availability. Atomic access is not automatically a synchronization protocol for unrelated payload stores [64].

Hardware manages warp participation, dependency readiness, memory request execution, and the supported atomic memory path. The kernel and compiler choose how many elements a thread owns, whether to group duplicates, where to place private state, and how to hand results to regular compute. CUTLASS’s producer–consumer pipelines and warp specialization illustrate that GPU software can organize asynchronous handoff rather than simply wait at a global barrier after every stage [78]. The comparison with SSNPU must allow such optimized baselines.

Aggregation benefits depend on the grouping cost and distribution. Private bins occupy storage, introduce initialization and merge work, and can affect occupancy. Register-heavy grouping raises live-state demand. Scattered addresses increase memory transactions, while repeated atomic destinations serialize through the applicable service mechanism. GPU flexibility thus has a resource price, but it is not a claim that every element always requires independent software coordination. Structured sparse MMA is a separate facility with prescribed metadata and operand constraints; it does not provide a general irregular-memory or histogram instruction merely because its name includes sparsity [65].

6.4 Ascend: local gather/scatter and two distinct atomic routes

Ascend illustrates why API scope and memory space cannot be omitted. The CANN 9.1 **Gather** interface reads from a LocalTensor in Unified Buffer (UB), using a local offset tensor and byte-addressed offsets. A2/A3 support selected 16- and 32-bit operand types; the 950 family includes additional 8- and 64-bit forms. The cited **Scatter** interface is available on 950 and selected inference products, not A2/A3, and requires unique destination offsets. Its duplicate behavior cannot be used to implement a collision-sensitive reduction [22, 25].

These local operations do not themselves establish arbitrary indexed GM access. The kernel stages data, chooses partitions, and performs the necessary GM transfers. Separate DMA atomic modes, including **SetAtomicAdd**, configure supported subsequent writes to GM. The documented source-buffer paths and legal data types vary by generation; the kernel must manage destination selection, transfer completion, and the lifetime of the selected atomic mode [21]. Local accumulation can reduce the number of GM updates, but it introduces UB capacity, movement, and merge obligations.

CANN 9.1 also documents scalar GM **AtomicAdd** on 950, with int32, uint32, float, int64, and uint64 forms returning the old value. This is a separate route from the DMA mode and is not supported on A2/A3. Dependencies between these scalar operations and MTE2/MTE3 transfers require explicit synchronization in the documented cases; the scalar atomic bypasses DCache, so cache consistency also needs attention [24]. An instruction with the right arithmetic result is insufficient if a later transfer can observe the wrong version.

For histogram-like work, the 950 register-vector **Histograms** facility operates on uint8 input and uint16 bins, with separate 128-bin halves and overflow considerations. It can help form local counts but does not resolve concurrent GM collisions by itself [23]. Overall, algorithm and kernel software own partitioning, staging, and merge policy; APIs expose the available movement, arithmetic, and synchronization mechanisms; hardware performs their execution. The applicable opportunity is to exploit local regularity and reduce shared updates. The remaining costs include fragmented movement, local banks, temporary capacity, synchronization, and hot GM addresses. This is a division of work, not evidence that Ascend cannot express irregular algorithms. The public 3510 architecture preview further documents SIMT warp schedulers, a SIMT register file, and a GM data-cache path sharing UB capacity. Thus the local-vector and atomic API examples above are not an exhaustive model of newer Ascend execution. A selected SIMT route needs its own programming-contract comparison, including thread state and the capacity tradeoff with UB [18].

6.5 TPU: runtime block indexing and a separate SparseCore path

A compiler-directed tensor program need not have only compile-time-known addresses. Pallas scalar prefetch moves index information into scalar memory before the main pipeline, allowing block maps to select data-dependent blocks and skip irrelevant work. The documented block-sparse tutorial demonstrates this mechanism on a TPU v5 lite path. Block shapes, alignment, memory capacity, and pipeline preparation still constrain the implementation [45]. Static array shapes and dynamic block selection are compatible concepts.

SparseCore is a distinct route for irregular workloads. Pallas documents scalar/vector subcore execution, indexed HBM transfers from VMEM-held indices, and operations useful for sorting, uniqueness, histograms, and ragged processing. Generation and interface availability must be named. The documented indexed DMA transfers are natively 32-bit; 16-bit gathering uses packed 32-bit transfers followed by selection/unpacking in VMEM. Its scatter example writes indexed output locations and is not a general atomic-add contract [46].

For large embedding workloads, OpenXLA describes a broader division of labor. Host

preprocessing transforms and partitions IDs; the compiler plans buffers and schedules; SparseCores execute local gather/reduction and exchange work as required by the partition. Limits on IDs and unique IDs determine resource requirements and can motivate minibatching or a specified overflow policy. The cost includes preprocessing, HBM and VMEM movement, temporary capacity, routing, synchronization, and skewed partitions [94]. These are practical mechanisms for irregularity, not reasons to equate TPU with a purely static dense matrix machine.

A fair sparse-to-compute comparison should include the handoff to the regular tensor path and the lifetime of its intermediate representation. Eliminating duplicate IDs before gathering can avoid downstream work if reuse justifies the preprocessing. Conversely, a small batch or rapidly changing distribution may offer less amortization. The proposed SSNPU must be compared with the appropriate TPU path for that workload, rather than only with a dense operation forced to emulate all irregular processing.

6.6 Tenstorrent and CPU vector ownership

Tenstorrent data-movement kernels construct NoC addresses and issue asynchronous reads or writes between DRAM, remote L1, and locally owned buffers. The device API includes mechanisms for reusing request configuration in repeated reads. This supports software-defined indexed transfers, without establishing the presence of a SIMD gather opcode [128]. Reader, compute, and writer roles coordinate circular-buffer capacity and completion. An owner-local accumulation followed by explicit merge is a natural algorithmic strategy, provided partitioning gives that owner the intended updates.

Small scattered requests can pay address/setup and transaction costs, while larger grouped transfers improve useful bytes per request at the price of grouping and latency. Buffers cannot be consumed before the relevant completion condition or reused while a transfer still references them. The documented `noc_semaphore_inc` performs a remote-L1 uint32 synchronization operation with its own alignment and wrap semantics; it is not evidence of a general floating-point or vector atomic reduction [129]. Hardware provides NoC execution and engine control, while kernels choose address grouping, local accumulation, and the merge protocol. Undocumented internals should remain unknown rather than being filled in by analogy to another machine.

CPU vector ISAs expose a related split. AVX2 provides gather, and AVX-512 adds masked scatter and, with AVX-512CD, conflict detection. Detecting that two lanes in one vector target the same index permits software to batch conflict-free groups; it does not protect those accesses from another thread. Shared updates still need an applicable atomic mechanism or private accumulation and merge [41]. Gather/scatter support cannot be relabeled as vector atomic reduction.

Arm SVE supports predicated gather/scatter with defined addressing forms. Predication and vector-length-agnostic loops help represent active elements and tails; they do not by themselves provide collision-sensitive RMW semantics. Scalar atomic facilities, local partitioning, or another explicitly supported mechanism must supply those semantics [6]. Across CPUs, the compiler selects vectorization, conflict handling, and decomposition, while hardware performs lane address generation, cache access, and applicable scalar atomics. Useful-lane occupancy, register pressure, cacheline ownership, retries, and contention remain workload-dependent costs.

6.7 SSNPU address generation: byte offsets and explicit TLEA

The SSNPU target separates logical indexing from indexed memory addressing. The recorded specification baseline already defines signed or unsigned byte displacements and execution-mask state. The proposed TLEA integration does not introduce either capability. Its role is to convert a supported logical integer index into a 64-bit byte offset for an accessed element width of 8, 16,

32, or 64 bits [103, 108, 110].

For index i , element size e bytes, and a selected signed/unsigned extension rule, the conceptual operation is

$$o = (\text{Extend}(i) e) \bmod 2^{64}, \quad a = \text{base} + o. \quad (23)$$

The first expression is TLEA’s offset calculation. The second belongs to the memory address path and its architectural checking rules. TLEA does not add the GM base or consume a row stride. For a two-dimensional array whose row stride is 96 elements, accessing $(r, c) = (2, 7)$ first forms $i = 2 \cdot 96 + 7 = 199$. A four-byte element then produces $o = 796$. Supplying 199 directly to byte-offset indexed memory would address the wrong location; multiplying 796 a second time would also be wrong.

Signed extension matters for negative offsets. A signed index of -1 for a four-byte element has a modulo- 2^{64} byte-offset representation corresponding to -4 . This arithmetic alone does not establish that the eventual address is legal, aligned, or within an intended object. Nor does an element width of four bits fit this byte-element TLEA contract: subbyte transfers require a separate packing and selector contract rather than an assumed fractional-byte multiplier. The source review’s earlier logical-element versus byte-address mismatch arose from combining older snapshots and is not a reason to add implicit scaling to the target AGU.

Execution masks belong to the operation contract rather than to address scaling. They determine participation in the indexed operation; TLEA is not an atomic operation, a collision detector, or a mask generator. The old inspected frontend’s rejection of masked scatter describes that implementation snapshot, not absence of the target architectural mask state. A correct implementation must preserve the selected active set through address checking, request generation, and result handling.

The distinction carries a concrete cost. Indices must be generated or loaded, extended, and scaled if they are not already byte offsets. Masks and data are also operands with readiness and storage requirements. The compiler may fold reusable address arithmetic or retain a suitable offset representation, but a semantic instruction does not synthesize arbitrary index arrays for free. Hardware still must admit requests, group supported accesses, track returned fragments, and assemble their logical results.

6.8 The atomic contract is finite and stronger than ordinary scatter

At target specification revision `cab1978f`, every valid active atomic request is an intrinsic atomic event. Duplicate addresses serialize in an implementation-defined order, and every valid active request takes effect. Atom forms return the value observed before their individual update; reduction forms produce no return Tile. Alignment, page, and read/write-address checks are preflighted before atomic effects and local result publication [104, 105]. This provides a defined basis for the count and allocation examples, but does not impose row-major duplicate order.

The supported type matrix is intentionally finite:

Operation	Operand types in the reviewed target contract
ADD	FP16, BF16, FP32, FP64, S32, U32, U64
MIN / MAX	S32, S64, U32, U64
CAS	U16, U32, U64

Thus S64 addition, U8 atomics, and floating-point MIN/MAX must not be inferred from another entry. A source benchmark using FP32 CAS is not compatible evidence for this unsigned CAS signature unless its distinct dependency and contract are established. The type and operation

must be traced through the intended API, compiler, and implementation rather than inferred from a similarly named template.

Ordering is also bounded. Acquire and release selectors choose relaxed, acquire, release, or acquire-release behavior; the reviewed interface has no sequentially consistent selector. Tile/IndexedGenerated requests are IntraCore-shareable in the cited memory model. Portable coherence covers exact address-and-size matches, while partial mixed-size overlap fails closed. These conditions do not establish unrestricted cross-core Tile synchronization [107, 109].

Per-request atomicity must be separated from the optional whole-block `B.CATR atom=1` contract. Normatively, that flag requests a non-interleavable, all-or-nothing block including memory and register outputs. The reviewed executable ASL records the flag, but the generated reader reports no operation consuming `CurrentBundleAtomic`. Enforcement of the whole-block property is therefore not demonstrated by that executable path [106]. Preflight prevents the specified fault cases from occurring after some effects have begun; by itself it does not prove transactional visibility of an entire block to observers. General fairness and forward-progress guarantees also remain separate evidence obligations.

These qualifications preserve the actual design argument. There are defined masks, duplicate semantics, returned values, ordering choices, and fault rules; they should not be described as absent. At the same time, a normative contract does not establish that every compiler form, timing path, or hardware realization implements it correctly.

6.9 Implementation evidence and sparse-to-compute handoff

The submitted integration spans specification, API, compiler, benchmarks, and model. The candidate model at `aa91094d` adds descriptor-based 64-bit index assembly, explicit TLEA execution, and a bounded active-lane U32 RMW/coherence path. Its reviewed timing evidence is limited to a single-thread development profile and an exact U32 M32 32×1 `MGATHER.ADD` form. That does not establish generic atomic MIN/MAX, arbitrary shapes, or all no-return reductions, and it does not narrow the target ISA to that profile [59].

Compiler recognition is similarly versioned. The previously inspected LLVM revision `05a1e626` recognized particular fixed-trip loops and a tightly constrained conditional fetch-add pattern. Those observations are historical, not a current-compiler verdict. Integration metadata recorded on 7 October identifies LLVM #117 at `74c93843` and API #249 at `a70915a5`; the broader U32 SSA path at the newer compiler head has not received complete validation in this article.¹ The benchmark integration supplies source-level examples and previously reported evidence, not tests rerun for this comparison [57, 58, 102].

The proposed programming advantage appears at the transition from irregular work to regular compute. A scalar phase describes data-dependent loops or partitions; indexed and masked operations fill supported typed Tiles; dependency tracking can make a ready Tile available to a vector or matrix consumer. Logical tags and physical binding, discussed in Section 4, can let storage ownership follow actual progress. Typed geometry, discussed in Section 5, tells the consumer how to interpret the result. Where supported, this can reduce explicit producer/consumer role assignment, barriers, and buffer-lifecycle code.

However, the software still selects the algorithm, partitions work, supplies dependencies, and requests any required synchronization. Irregular loops with independently evolving long-lived state may be more naturally expressed as GPU threads. Translating them into scalar/Tile phases or work queues can add compaction, scheduling, and temporary-storage work. A shaped operation does not create a program counter for every element, nor is such a program counter

¹The recorded API head is `a70915a5e18e109d680d82068622a42e3a24138e`, observed at 05:28 UTC on 7 October 2026. These revision notes identify evidence boundaries, not an assertion that the changes are deployed or that their PR status defines the intended architecture.

necessary for every irregular algorithm. The expressibility claim must be stated for the actual operation, memory-scope, and progress contracts rather than as unrestricted equivalence between execution models.

Hardware takes on the displaced coordination: tag and generation tracking, readiness selection, physical-capacity admission, address mapping, return buffers, atomic arbitration, and completion. The historical ordinary gather path groups requests within bounded index-read windows and reconstructs logical output order from returned byte ranges. It does not imply whole-Tile deduplication or unlimited outstanding requests. Scattered addresses still multiply requests, and incomplete destinations still occupy return or assembly state. Bounded queues introduce backpressure; recovery and runnable drain paths must preserve liveness, as examined in Section 10.

6.10 Conditions for a useful architectural improvement

A defensible evaluation starts with a specified index width and unit, data type, mask density, locality distribution, duplicate distribution, contention scope, output requirement, and fault-/ordering policy. The same workload should be implemented with direct updates and with competent alternatives such as local grouping, privatization, and segmented reduction. The sparse-to-compute consumer and its required publication event belong in the measurement interval.

The SSNPU opportunity is strongest when irregular phases produce regular typed blocks, dynamic arrival makes static ownership conservative, and dependency/resource management replaces substantial explicit coordination without adding more critical-path control than it removes. It is weaker when one hot address dominates, little regular work follows, or independent divergent loops dominate the algorithm. Similar distribution-sensitive conditions govern GPU aggregation, Ascend local staging, TPU preprocessing, and Tenstorrent ownership partitioning.

Measurements must include end-to-end latency, useful throughput, transferred bytes, temporary storage and lifetimes, contention, control work, and hardware implementation costs. Fewer source-level barriers or roles can demonstrate a convenience improvement; they cannot alone demonstrate speed, utilization, energy, or PPA. The architectural conclusion is therefore conditional: dynamic dependency and resource management can make supported irregular-to-regular pipelines easier to coordinate, provided its atomic, ordering, capacity, recovery, and progress contracts are both precise and enforced.

7 U005 Tail axis reductions and the placement of partial state

7.1 The logical axis does not determine the physical reduction

For a logical array $X \in \mathbb{R}^{G \times K}$, a row sum produces G outputs,

$$y_g = \sum_{j=0}^{K-1} X_{g,j}, \quad 0 \leq g < G. \quad (24)$$

This notation omits the order and precision of additions. It also says nothing about whether a row belongs to one thread, several SIMD lanes, a vector register, multiple tiles, or several cores. Those ownership choices determine the exchanges and partial storage needed to form y_g . The tail axis is often physically contiguous, which can simplify reading values, but contiguity does not imply that all of them reach one accumulator without communication.

There are two distinct sources of parallelism. Different g values are independent unless the surrounding algorithm introduces another dependency. Within one row, reassociation may allow a parallel tree, but only if the numerical contract permits it. A binary tree combining K values

has $K - 1$ combines and ideal depth $\lceil \log_2 K \rceil$. A prescribed fold can have a dependence through every input. Neither count determines cycles: an instruction may process several rows, the hardware may be pipelined, and communication or operand delivery can dominate arithmetic. Consequently, many short rows and a few very long rows are different architectural cases even when GK is the same.

For a higher-rank array such as $X[B, H, Q, K]$, reducing the last axis leaves independent output coordinates (b, h, q) . If their layout permits, they can be mapped to one group index g . Rank four does not itself require four reduction levels. Reducing a middle axis is different because the retained and reduced coordinates can be interleaved. Stride-aware iteration, a transpose, or staged accumulation are alternatives; the correct choice depends on traffic, retained state, and available output-level parallelism. A row-major column reduction may simply vectorize across output columns while iterating over rows, so a transpose is not inherently required.

7.2 A four element group example

Consider 32 contiguous FP32 input elements partitioned into eight groups of four. The logical operation is

$$X[8, 4] \longrightarrow y[8, 1], \quad y_g = \text{reduce}(X_{g,0}, X_{g,1}, X_{g,2}, X_{g,3}). \quad (25)$$

The input contains 128 logical bytes and the output contains 32 logical bytes. These are element counts, not a claim about how many physical allocation bytes either side consumes. In a contiguous RowMajor representation, a 128-byte payload can hold all eight input groups. A blocked matrix layout has a different physical envelope, even when its valid region has the same 32 elements.

The pinned PTO CUBE contract defines each CELL as 128 bytes. For 32-bit elements, a CUBE_M16 CELL has shape 16×2 , whereas a CUBE_M32 CELL has shape 32×1 . Thus the four columns of one logical group span two column CELLS in M16 and four in M32. Eight valid rows require the physical envelopes

$$S_{M16} = 16 \cdot 4 \cdot 4 = 256 \text{ bytes}, \quad (26)$$

$$S_{M32} = 32 \cdot 4 \cdot 4 = 512 \text{ bytes}. \quad (27)$$

The extra rows are part of the layout envelope; they are not extra logical groups. It is incorrect to identify every 128-byte CELL with one four-element group or to infer the number of groups executed per cycle from the number that fit in storage. For 16-bit elements the CELL geometries become 16×4 and 32×2 ; for 8-bit elements they become 16×8 and 32×4 . These are descriptor and storage facts, not arithmetic-width or service-rate guarantees. [111]

The result also needs an allocation contract. The pinned row-reduction specification creates a new Local destination and preserves its source. The destination has one *valid* column. For CUBE M16/M32 it retains the source's physical column envelope and uses the minimum legal physical row envelope covering the valid rows. In this example, the logical result therefore remains eight FP32 values, but a CUBE destination with four physical columns still carries the 256- or 512-byte envelope above. RowMajor derives its physical rows from the selected destination capacity and one column; its allocation is also not proved to be 32 bytes merely by counting results. A later compaction or layout conversion may reduce storage if the ISA and allocator support it, but that operation and its lifetime must be accounted for. A prefix view alone does not establish that the unused part of an allocation has been returned to the allocator. [112, 113]

7.3 The ordered fold is an architectural constraint

At PTO revision e182c9b, TROWSUM is selected through the TEPL encoding carrier and dispatched as an SFU block operation. It is not a separate standalone opcode. Its semantic contract is a

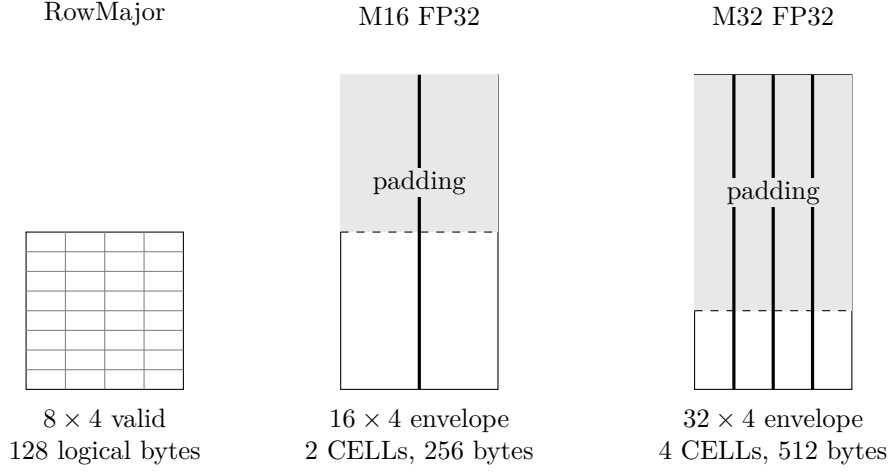


Figure 3: The same eight groups of four FP32 values in three representations. Each diagram has four logical columns; M32 rows are drawn at half the vertical scale. Shading denotes physical rows outside the valid region. CELL divisions describe storage, not execution cycles.

typed left fold beginning with typed zero, in increasing column order:

$$a_{g,0} = 0_\tau, \quad a_{g,j+1} = \text{TADD}_\tau(a_{g,j}, X_{g,j}), \quad y_g = a_{g,K}. \quad (28)$$

Here τ is the operation type, including its rounding and status behavior. The contract does not authorize replacing this recurrence by a balanced floating-point tree. Independent groups may run in parallel while each group’s observable result and numeric status follow the recurrence. Even then, the actual number of groups processed simultaneously depends on datapaths, ports, issue capacity, and implementation scheduling; it is not given by the semantic definition. [113]

A concrete FP32 round-to-nearest illustration explains why the distinction matters. For the four inputs

$$[2^{25}, 1, -2^{25}, 1], \quad (29)$$

the prescribed fold loses the first added 1 at the large magnitude, cancels the large values, and ends at 1. A pairing that first forms $(2^{25} + 1)$ and $(-2^{25} + 1)$ rounds both pairs to the large signed powers of two and then obtains 0. The real-arithmetic sum is 2. This example is a derived numerical illustration, not a benchmark or a claim about another architecture’s default reduction mapping. It shows that “four inputs, two tree levels” is not a legal optimization merely because it looks like the same mathematical sum.

The initial addition from typed zero is also part of the defined operation. An implementation may simplify it only when it preserves the type’s full observable semantics, including exceptional inputs and status. Integer addition in the pinned contract retains element-width bits at each step; no separate wider accumulator is implied. The input carrier can be an equal-width non-packed view where the contract permits, but a view is an interpretation, not an arithmetic conversion. An FP32 matrix accumulator elsewhere in the architecture does not grant FP32 accumulation to a typed FP16 reduction.

There is a further implementation boundary. The pinned legality list admits types whose floating fold helper does not yet define an element result: the public reader guidance identifies TF32, HF32, E4M3, and E5M2 as reaching an incomplete `ScalarFPBinaryProfile` path. The article therefore does not present FP8 sum as an established complete executable capability. A type name in the legal-type list and a complete numeric implementation are separate evidence. Resolving this gap requires a specified arithmetic profile and validation, not merely adding an instruction name to a table. [113]

7.4 Masks, padding, and incomplete groups

The current reduction contract includes every coordinate in the valid source rectangle. It rejects the generic Local CUBE ExecutionMask, including mask state supplied directly to the model. This is different from the shared PE mask: an all-zero PE mask is a strict no-op at a coarser participation boundary. The PE mask cannot be substituted for arbitrary per-element reduction participation. These rules prevent an implementation from silently treating a general element mask as a supported ragged reduction. [112]

For a flat length N grouped by four, let $G = \lfloor N/4 \rfloor$ and $r = N \bmod 4$. The G complete groups have shape $[G, 4]$. If $r \neq 0$, the final group requires separately defined handling: a smaller legal shape, a distinct scalar/vector path, or explicitly materialized identity values under a numerical contract that permits them. Padding *outside* a valid rectangle does not make those elements participants in the reduction. Conversely, extending the valid rectangle and filling it with zeros changes which typed operations occur. Such a change must preserve the required rounding, signed-zero, NaN, and status behavior rather than relying only on a real-arithmetic identity.

The same issue appears in masked softmax. An inactive logit is often represented by a suitable negative infinity before the maximum and exponential stages, but the behavior of an all-masked row must still be chosen. A sum's identity, a maximum's identity, and the output policy for an empty normalization are different decisions. The current nonzero-dimension requirement also means an empty Tile cannot be assumed to encode an empty reduction with a convenient default result. Compiler-generated tails have to respect these legality and result rules.

7.5 Four PE local reductions and a shared Tile Register merge

Four processing elements (PEs) can cooperate in two fundamentally different ways. First, each PE can own complete, independent reduction groups. For the eight groups of four in Equation 25, assign groups 0, 1 to PE 0, groups 2, 3 to PE 1, and so on. Each PE produces two final group results. No cross-PE combine is needed to compute those eight answers, although distributing inputs and handing results to their eventual consumers can still require movement and synchronization. This mapping exposes parallelism *between* groups without changing the prescribed arithmetic *within* a group.

Second, the four PEs can own disjoint subsets of the *same* global group. If one row has K inputs and PE p owns the subset I_p , then a local reduction produces a partial s_p . Under a numerical contract permitting this decomposition, the logical construction is

$$s_p = \text{reduce}_{j \in I_p}(X_j), \quad y = \text{merge}(s_0, s_1, s_2, s_3), \quad \biguplus_{p=0}^3 I_p = \{0, \dots, K-1\}. \quad (30)$$

Four local answers are not four final answers to that one global reduction. A consumer needing y must obtain the contribution from every required participant. Different ownership of the same logical axis changes the communication requirement, even when the local reduction instruction is unchanged.

The proposed target architecture makes a shared Tile Register available as a handoff location for PE-produced values. Its intended role here is to let a larger reduction be partitioned across PEs, locally compress each partition, publish those smaller partials, and read them for a final merge. This is a *target-design discussion*; the present source inspection does not demonstrate a particular physical shared-register topology, port count, merge datapath, throughput, or synchronization implementation. “Shared Tile Register” names the intended visible handoff resource, not a measured zero-latency or infinitely multiported structure.

For a long row, local compression can substantially reduce the logical data handed to the merge relative to forwarding every input. For example, four partitions of 256 FP32 values contain

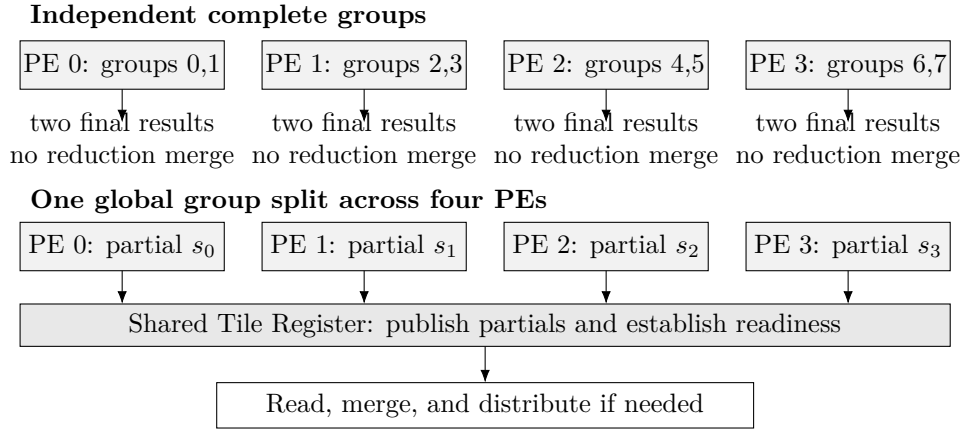


Figure 4: Two logical ownership choices for four PEs. The upper path assigns complete groups locally. The lower path needs a cross-PE merge and a numerical contract permitting its partial-reduction order. Input/output movement still exists in the upper case. This target-design diagram specifies neither physical topology nor timing.

4096 input bytes in total; four FP32 partials contain 16 logical bytes. That arithmetic is only a payload comparison. If all four partials are written to shared storage and read once by one merger, the handoff accounts for 16 logical write bytes and 16 logical read bytes, before physical transfer granularity, padding, metadata, contention, result storage, or broadcast. Keeping the merger’s own partial local could avoid one of those shared write/read pairs. The actual traffic and allocation follow the supported Tile and register organization. The earlier CUBE-envelope example is a reminder that 16 logical bytes do not establish a 16-byte physical allocation or shared transfer. Conversely, splitting a four-element group across four PEs leaves one value per PE, so there is no local reduction compression before the exchange; distributing whole short groups may be the more useful mapping.

A shared handoff needs an explicit producer–consumer protocol. Each partial requires a unique group and generation identity, storage ownership, and a rule establishing when its payload is visible. The merge must depend on all required partials, or incrementally combine arrivals while still waiting for the last required contribution before publishing the final result. Reusing a partial’s slot must wait until every relevant reader is finished. If multiple PEs subsequently normalize their own input partitions, the final sum, maximum, or reciprocal must also be made available to each consumer. A single merger’s completion does not by itself broadcast the result or release every participant’s state.

Hardware readiness tracking could enforce these dependencies without software executing a full barrier after every local step. Such an implementation still performs synchronization in the architectural sense: it prevents a dependent read from preceding publication and prevents reuse from preceding consumption. A shared register does not remove those causal edges. It moves their enforcement into a particular combination of descriptors, ready bits, dependency joins, arbitration, and possibly software events. The scope and granularity of publication must also be defined; a reader cannot assume that observing one ready CELL makes all other partials ready.

The cost ledger therefore includes shared-register read/write bandwidth, bank and port conflicts, arbitration, temporary partial storage, readiness joins, the merge engine, and result distribution. If r_p is the time at which PE p ’s required partial becomes visible, then publication of a final result using every contribution cannot precede $\max_p r_p$. A merge can overlap earlier work and consume partials as they arrive, but it cannot manufacture the missing slow participant’s value. Load imbalance, a delayed input, or contention on one PE can therefore dominate the final dependency. If all four PEs need the statistic, broadcast or repeated reads add another

resource demand whose cost depends on the actual fabric.

Exact fold versus distributed reassociation. Equation 30 is not automatically a legal implementation of the current floating-point TROWSUM. The pinned contract in Equation 28 begins from zero and visits columns in order. Independently folding each partition from zero and then merging its sum generally changes rounding and status compared with that global recurrence, even if the partitions are contiguous and the final merge visits them in column order. Passing the running accumulator to the next contiguous partition and continuing the ordered additions can preserve the arithmetic recurrence, using a separately supported sequence or implementation mechanism. Current TROWSUM exposes no seed-accumulator operand. Full instruction equivalence must also preserve complete-source preflight, accumulated numeric status, and atomic publication of the result and descriptor. An ordered handoff therefore does not by itself establish an implementation of the full instruction contract, and it does not provide four independent arithmetic folds of one row. A different summary representation or exact accumulator would require its own proof of equivalence, not an assumption that four rounded partial sums suffice.

Independent complete groups remain parallelizable under the current contract. A reassociative distributed sum needs separate, explicit numerical permission, including accumulation type, rounding, exceptional-value/status behavior, and the required reproducibility or error tolerance. Maximum also needs specified NaN, signed-zero, and tie behavior before exchanging arbitrary partial maxima is treated as interchangeable with an ordered reference. Hardware visibility and arithmetic equivalence are separate requirements: a perfectly synchronized merge can still compute the wrong specified floating-point result.

When to prefer each mapping. Keeping complete short groups local is a strong default candidate when there are enough independent groups, balanced work, and a suitable input layout. It avoids a cross-PE reduction merge and can reduce shared-port and readiness pressure. It is not a universal winner: there may be too few groups, an excessively long row, uneven ownership, or a producer/consumer layout that makes a shared handoff advantageous. A cross-PE partition can enlarge the usable parallel work and spread storage or local arithmetic over several PEs when its numerical contract permits it. Its benefit must exceed partial publication, read/merge, waiting, and distribution costs. The comparison should report both the saved local work per PE and the newly required shared work, rather than presenting “four local reductions” as either automatically complete or automatically faster.

7.6 NVIDIA GPU ownership can remove or introduce exchanges

In a common SIMT mapping, a thread combines its register-local values, then participating lanes exchange partial results with warp shuffles. Multiple warps usually need another coordinated stage, often using shared memory, and a reduction split over multiple thread blocks needs a further merge protocol. This hierarchy is a mapping choice rather than an intrinsic number of stages for every tensor. XLA’s GPU emitter documentation explicitly describes shuffle and shared-memory lowering for reductions. [95]

API and ISA generation matter. The CUDA 13.0.3 warp `__reduce*_sync` signatures cover integer arithmetic and bitwise functions; floating-point cooperative reduction examples can use shuffle-based lowering. That observation must not be generalized into the absence of all floating-point warp reduction instructions. PTX 8.7 documents FP32 minimum and maximum forms of `redux.sync` for `sm_100a`, including defined NaN behavior, but does not provide an FP32 addition form in that instruction family. Thus a softmax maximum and sum need not use the same mechanism. [86, 87]

A contrasting example is FlashAttention-4’s data-center Blackwell mapping: a thread can receive a complete 128-element score-tile row from tensor memory, making that row’s reduction register-local. This removes a cross-thread exchange for the cited mapping while increasing or retaining other obligations: register capacity, tensor-memory movement, buffering, and pipeline coordination. Its treatment of exponential work also changes the balance between special-function service and ordinary arithmetic/register use. It is evidence of algorithm–layout co-design, not proof that Blackwell reductions are universally local to one thread. [12]

Many-short-row cases can pack several independent groups within a warp; long-row cases can distribute work more broadly. The compiler must choose participation masks and merge structure consistent with the numerical requirement. Triton’s fused-softmax tutorial illustrates a row-resident strategy with power-of-two padding and explicit attention to register/shared-memory occupancy. Keeping a row on chip can avoid rereading it while reducing the number of resident programs. The appropriate comparison to a direct Tile operation therefore includes the best row-ownership mapping that satisfies the same contract, rather than a deliberately communication-heavy baseline. [151]

Pinned CUDA examples expose ownership transitions. The `csrc/flash_attn/src` path at FlashAttention repository commit `94e22c90` illustrates a local-then-exchange implementation. Its `thread_reduce_` loops over each thread’s fragment; `reduce_max` then calls `Allreduce<4>`. The helper in `utils.h` performs XOR shuffle exchanges with offsets 2 and 1, with a combine at each step. These exchanges form logical four-thread groups inside a warp; the helper uses a full-warp participation mask. This is local register arithmetic plus scoped lane exchange, not a shared-memory write/read or a CTA-wide barrier. The source and participation requirements must be respected rather than treating every occurrence of `sync` as the same operation. [9, 10]

The same softmax path keeps row-sum partials local while updating the running statistics and performs `quad_allreduce_` in `normalize_softmax_lse` when the combined denominator is needed. Delaying that cross-thread sum avoids repeating an unnecessary exchange at every online update. This is a specific code path and revision; it does not describe every FlashAttention version, the separately discussed Hopper path, or every Blackwell/CuTe kernel. [9]

OneFlow commit `25c8978c` makes a second boundary explicit. Its `WarpAllReduce` uses an XOR-shuffle loop. Its `BlockAllReduce` instead calls CUB with shared temporary storage, has thread zero write a shared result, executes `__syncthreads()`, and returns the shared broadcast value. The caller’s broadcast/barrier is visible independently of CUB’s internal algorithm. Counting local accumulation, shuffles, shared accesses, and barriers separately is more informative than labeling both variants “many synchronizations.” [93]

Separately, CCCL commit `dc7fcbdc`’s deterministic warp-reduction specialization writes one aggregate per warp into shared storage, executes a block barrier, and lets thread zero combine the warp aggregates. That specialization also contains an alternative non-deterministic addition path; it is not a universal description of all CUB reductions. Nor does citing this current CCCL file prove that the pinned OneFlow build uses that same dependency revision or specialization. These examples establish scoped mechanisms, not matched latency measurements. [79]

7.7 Compare shared handoff resources at matching scopes

GPU shared memory and the proposed shared Tile Register can both serve as on-chip producer–consumer handoff storage. That functional similarity is useful, but their names do not establish the same physical memory tier, topology, access width, port organization, or visibility contract. In particular, “register” does not by itself prove lower latency or higher effective bandwidth than GPU shared memory. Conversely, a shared-memory-based reduction does not inherently imply an off-chip round trip.

There are several GPU comparison scopes. Register-local arithmetic has no inter-thread exchange for that combine. A warp shuffle exchanges values among participating lanes without staging the value through shared memory. A CTA shared-memory merge crosses warp ownership on the CTA's SM and requires the relevant visibility/coordination protocol. A cross-SM merge is a wider problem: ordinary CTA shared memory is not arbitrary chip-wide shared storage. Supported compute-capability-9.0 clusters provide distributed shared-memory access within a bounded co-scheduled cluster, with existence, synchronization, and lifetime requirements; other inter-block decompositions need an appropriate global-memory or multi-kernel protocol. The four-PE target must be compared with the scope those PEs actually occupy, rather than charging every GPU alternative for a cross-SM merge while treating the target's four PEs as adjacent local participants. [80]

The potential advantage of the shared Tile Register is conditional. If its actual implementation provides high effective bandwidth, short producer-to-consumer latency, and dependency-aware consumption of ready partial Tiles, it may avoid a more distant storage path or conservative whole-workgroup waiting in a particular baseline. The baseline might already use nearby shared memory, shuffles, or asynchronous barriers, so the avoided work must be identified rather than assumed. The comparison needs equal logical grouping, numerical semantics, data placement, and result-consumer scope.

Measure or model effective read/write bandwidth under the real access pattern, latency from production to dependent use, bank conflicts, simultaneous read/write ports, arbitration, and contention with matrix/vector traffic. Include readiness publication and joins, slow-participant delay, output broadcast or repeated reads, physical allocation granularity, and the state required for safe reuse. An apparent reduction in explicit barrier instructions is a programming/control-placement result until those costs and the resulting latency are established. Shared storage and dependency hardware can make a handoff cheaper without making synchronization disappear.

7.8 Ascend composes reduction scopes on a vector path

Ascend C exposes pairwise, block, repeat, and larger reduction interfaces. Local buffer placement, masks, source strides, destination strides, and pipeline dependencies determine how those operations compose. A documented CANN 8.0 example for 256 FP32 inputs compares repeated whole reductions, repeated block reductions, and a block reduction followed by a whole reduction. The relevant architectural lesson is that a larger reduction primitive is not necessarily the best first stage. Intermediate storage and barriers remain part of the sequence. Its example timing is not transferred to the Superscalar NPU comparison. [26]

For the documented Atlas A2/A3 `WholeReduceSum` path in CANN 9.0, operands are half or float with matching source and destination types. A contiguous repeat can cover up to 128 16-bit or 64 32-bit elements. The page specifies a tree-style combination and describes intermediate saturation in its half example. These numerical details differ from an exact typed increasing-column fold. A fair performance study must either require the same numerical semantics on both sides or explicitly quantify the changed accuracy/reproducibility requirement. Product-specific integer or newer-generation extensions must be checked separately. [28]

Ascend's higher-level `SoftMax` interface exposes the broader sequence: maximum, broadcasting, subtraction, exponentiation, sum, broadcasting, and division, with explicit temporary-space and tiling considerations. ND and blocked NZ representations require different traversal choices. A library can hide this orchestration from application code, but hiding it in a function call does not remove vector work, local-memory demand, or a dependency between the statistics and normalization. [27]

For a reduction distributed across Ascend cores, local `BlockReduceSum/WholeReduceSum` composition is only the first stage. Partial publication, cross-core ordering, and merging must

be added. The live developer-preview **SyncAll** documentation shows partial computation written to GM before synchronization and subsequent accumulation. It distinguishes a software signal/polling path from supported hardware synchronization; the pipeline-configurable form is listed for 950 products, not A2/A3. Its residency and event-ownership conditions still apply. This demonstrates a GM/workspace plus synchronization design pattern, not a claim that every Ascend reduction uses software polling or that a barrier itself transfers the partial values. [37]

7.9 TPU lowering depends on vector topology and axis

TPU documentation distinguishes matrix, vector, and scalar units; softmax is vector work in that architectural description. The scaling guide’s v5p discussion describes a vector organization with eight sublanes and 128 lanes, with different mechanisms for sublane and cross-lane communication. Consequently, reducing one physical axis is not automatically equivalent to reducing the other. This is a scoped explanatory model, not a universal cycle model for all TPU generations. [13, 54]

Pallas TPU exposes block-shape restrictions and local-memory layouts commonly organized around (8, 128) units. Its compiler can retain an output window while successive input windows are processed. A reduction can therefore use local vector combines, rolling partial accumulation, layout changes, or a staged algorithm, depending on the shape and retained state. General advice to use 32-bit elementwise computation on narrower inputs is a software choice; it should not be confused with a guarantee about every hardware operation. [51]

The cost ledger must distinguish vector ALUs, vector loads/stores, matrix work, transpose, and reduction/permutation resources. XProf exposes such distinctions in its utilization view. Low matrix utilization by itself cannot determine whether the bottleneck is exponentiation, cross-lane reduction, movement, or a preceding scalar dependency. A Superscalar NPU comparison should use the same decomposition rather than treating all non-matrix work as a single interchangeable “vector” capacity. [96]

When a TPU reduction crosses physical cores or devices, the two-dimensional VPU axis distinction is no longer the whole problem. Pallas’s core-specific interface provides an intra-device remote-copy example with semaphores; the distributed interface exposes asynchronous push DMA with separate send and receive completion. A distributed reduction must compose local partial computation with such movement/merging or an appropriate compiler/runtime collective. These are scoped communication interfaces, not evidence of a shared Tile Register across TPU cores. Buffer lifetimes, topology, semaphore matching, and numerical merge order remain in the comparison. [52, 53]

7.10 Tenstorrent reduces within an owned tile pipeline

TT-Metalium’s `reduce_tile` belongs to the FPU/matrix-engine compute API group. It supports row, column, or scalar results and selected reduction operations. The API consumes source/scalar tiles through circular buffers and produces a result in acquired destination state; result placement and packer-mask setup are explicit. This interface does not imply one generic horizontal SIMD instruction or the same implementation as the separate SFPU/vector path. [138]

The unpack-math-pack pipeline and destination ownership determine how reduction work overlaps with its neighbors. In the documented standard-tile configuration, 32-bit destination storage reduces tile capacity relative to 16-bit storage, and double buffering partitions available state further in exchange for overlap. Destination width alone does not guarantee that every selected compute operation uses full FP32 arithmetic. [137]

For a reduction spanning several tiles, the algorithm must combine tile-local partials. If ownership spans cores, that merge additionally involves NoC movement and synchronization. Assigning whole rows to cores can avoid cross-core merging but may expose load imbalance;

splitting long rows increases parallel work while adding communication and state. These are general decomposition consequences. The tile API alone establishes neither a distributed reduction guarantee nor complete softmax fusion.

7.11 Softmax is a chain with retained state

For one row, stable softmax requires a maximum, subtraction and exponential, sum, and normalization. The intermediate exponentials must be retained or recomputed/read again after the final normalizer is available. Online normalization can merge partial states (m_a, s_a) and (m_b, s_b) using

$$m = \max(m_a, m_b), \quad (31)$$

$$s = s_a \exp(m_a - m) + s_b \exp(m_b - m). \quad (32)$$

This identity can reduce passes for the normalizer while adding rescaling arithmetic and partial state. It does not make every output available before the final normalization is known, nor does it guarantee floating-point equality across merge orders. [62]

FlashAttention-3 demonstrates a Hopper implementation that overlaps asynchronous matrix multiplication, movement, and softmax work across tiles and warpgroups. The overlap consumes barriers, pipeline stages, registers, and explicit role allocation. It is useful evidence that vector work can sometimes be hidden by matrix work, while the remaining non-matmul service demand still matters. A faster low-precision matrix engine alone does not establish a proportional speedup for maximum, addition, exponential, reciprocal, or movement. [115]

For the Superscalar NPU, a direct row/column operation can carry semantic intent and permit hardware to choose an implementation satisfying it. The current exact-order sum contract restricts that choice more than an associative reduction API. Hardware could interleave independent row folds, pipeline typed additions, or overlap unrelated operations; those are candidate implementations, not proven throughput. A future relaxed-order form would need an explicit numerical permission, separate legality, and testable error/reproducibility guarantees. It cannot be retroactively assumed for TROWSUM.

The architectural benefit condition is therefore precise. A direct Tile reduction helps when it removes software reconstruction or unnecessary communication for a supported shape while preserving required numerics and fitting the live-state budget. Its cost includes source/result port demand, row accumulators, padding/allocation, status aggregation, dependency tracking, and coexistence with exponentials and matrix work. It must be assessed together with U002's lifetimes and U003's layout transformations. The logical reduction result becoming small is only the beginning of that analysis.

8 U006 Scalar and event orchestration

8.1 Fusion changes the rate at which control must be performed

A fused pipeline can remove intermediate launches and unnecessary memory boundaries while creating more frequent interactions between its internal stages. For every tile, an implementation may calculate addresses, prepare descriptors, reserve storage, submit work, publish readiness, handle a completion, and return credits. These operations are useful work in the control path. If they cannot keep up, independent compute and communication engines can wait even though their payload bandwidth is not saturated.

Consider a steady stream of tiles. Let c denote the required scalar/control instructions per tile, μ_s the sustainable control service rate in instructions per unit time, t_m matrix service time per

tile, t_v vector service time, and t_d movement/communication service time. Under ideal overlap with independent resources, a necessary lower bound on the steady-state initiation interval is

$$I \geq \max \left(t_m, t_v, t_d, \frac{c}{\mu_s} \right). \quad (33)$$

If two stages use the same non-overlappable resource, their demand must be combined rather than placed in separate terms of the maximum. A dependency chain or insufficient buffering can raise I further. The equation is an analytical accounting device, not a model calibrated to a particular processor.

Smaller tiles can reduce the delay before a consumer sees useful output, but they generally increase the number of control episodes per useful byte. If a tile carries b bytes and requires e events, the event density is e/b . Reducing b while retaining the same protocol can increase this density even when the arithmetic work scales proportionally. The crossover depends on scalar issue capacity, event hardware, message aggregation, buffer size, and the independence available to cover latency. “Fuse more” is therefore incomplete guidance unless the control path is included.

8.2 Four scopes and five milestones

Within a compute core, scalar control can submit matrix, vector, and copy operations and coordinate local dependencies. Across cores on one chip, communication can use a local NoC, shared global memory, hardware flags, or remote local-memory operations. Across devices, it may use peer loads/stores, remote DMA, NIC queues, or a collective protocol. Host/runtime setup allocates resources, registers memory, constructs communicators, launches work, and tears it down. These scopes have different agents, ordering rules, and failure modes.

For one transfer, at least five milestones should be distinguished:

1. The command was accepted or issued
2. The source is no longer read by the local transfer machinery
3. The destination payload is visible within the required recipient scope
4. The consumer has observed the readiness event and consumed the value
5. The storage and protocol identity can be safely reused

A particular API may combine some milestones, but no combination should be inferred from the word “complete.” A flush that permits local source reuse is not necessarily a remote-consumption acknowledgement. An event announcing a byte count is not necessarily an acquire for unrelated memory. Safe tag reuse may additionally require that stale events from an earlier generation cannot match the new owner. These distinctions connect directly to U007’s ordering and U008’s reclamation obligations.

An event that travels through device memory is not automatically a host round trip. PCIe is an interconnect, not an execution agent: a supported peer or NIC path can use PCIe without host software submitting every message. Similarly, an accelerator-side service processor is not the host CPU. A claimed weak-scalar or host-event bottleneck must identify the actual processor, path, and trace rather than generalizing to every NPU.

8.3 NVIDIA distributes control through roles and device communication APIs

For on-chip pipelines, Hopper TMA allows a thread to initiate bulk asynchronous movement while other work proceeds. Warp-specialized kernels distribute producer, matrix-consumer, and

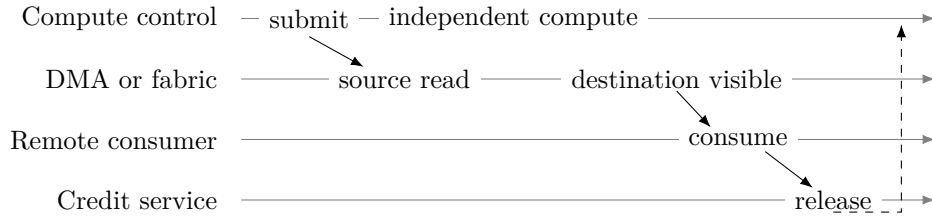


Figure 5: A conceptual producer-transfer-consumer protocol. Vertical arrows are required causal edges; horizontal spacing is illustrative and is not a latency measurement. Source reuse, destination readiness, and credit return can require different events and progressing agents.

epilogue work across roles, each with barrier and buffer ownership. CUTLASS contains concrete producer/consumer pipeline implementations. Dedicated roles avoid placing every instruction in one thread’s immediate sequence, but consume resident state, registers, barrier phases, and issue opportunities. Thread-block clusters add a bounded cross-block shared-memory scope; they do not make every block in an arbitrary grid simultaneously resident. [88, 89]

Across devices, the NCCL device API distinguishes local-access mechanisms using peer connectivity from network-oriented GIN operations. Its archived 2.28.7 documentation covers symmetric memory windows and device communication contexts, including puts, signals, waits, and flushes. These mechanisms require setup and transport support. In particular, local input reuse after a flush must not be conflated with the remote readiness/signaling obligation. The exact facility must be selected from the topology and library version rather than inferred from the GPU’s marketing family. [90]

NVSHMEM provides another division of responsibility. Traditional IBRC uses a CPU proxy; IBGDA permits GPU construction of NIC work-queue entries, doorbell updates, and completion handling in GPU memory. CPU-assisted variants place parts of the work differently. Device initiation therefore names a control capability, not one universal data path. The vendor’s performance guidance explicitly discusses the cost of GPU WQE construction, contention when issuers share queue pairs, and the resource/fence cost of more queues. Moving submission to the GPU can expose parallel issuing but does not establish a lower single-message latency in every configuration. [63, 91]

For a fused kernel, the relevant accounting includes communication-role occupancy, scalar instructions in the prologue and epilogue, polling or atomic traffic, staging memory, and NIC queue pressure. The GPU scheduler can select other eligible warps while one waits, but only resident work with available resources can benefit. Synchronizing persistent or collective kernels must follow the library’s launch and progress requirements; NVSHMEM documents collective-launch constraints for applicable synchronization/collective use. Ignoring those requirements would compare a correct bounded protocol against an inadmissible one. [92]

8.4 Ascend separates pipeline events from communication service

Ascend C describes scalar control as responsible for scheduling and instruction transmission and advises reducing unnecessary scalar computation and branching. This supports the possibility of scalar pressure in a specified kernel; it does not prove a universal single-thread bottleneck across every Ascend generation. Intra-core `SetFlag/WaitFlag` operations use pipeline event state, and the receiving pipeline waits for the corresponding event. They are not inherently host or PCIe events. Event identifiers are finite and pairing is part of the programming contract. [29, 31]

For supported separate-mode products, cross-core flags provide AIC/AIV synchronization with product- and mode-specific rules. CANN 8.5 documents modes spanning AIC/AIV groups and warns that high-level Matmul internally owns some synchronization resources. A hand-written

fused kernel must therefore avoid conflicting with library-owned event IDs. Correctness is a composition problem involving the compiler, library, runtime, and explicit user synchronization, even though the underlying event operation is hardware controlled. [30]

For cross-device work, the CANN 9.0 HCCL kernel interface uses an AI Core client and an AI CPU or CCU server, depending on supported hardware and configuration. A Prepare operation supplies the collective description; Commit authorizes execution when dependencies allow it; completion and finalization have separate roles. Descriptor preparation can be moved before compute so that some setup overlaps, while Commit must still respect data readiness. The AI CPU is accelerator-side, so its participation alone does not imply a host-CPU round trip. [32]

This organization pays for client scalar instructions, server work, handles, queues, communication workspace, and synchronization. Repeated use must maintain the documented Commit/Wait accounting, and finalization cannot race slower participating cores. The useful question is whether preparation, server execution, and completion service stay ahead of the tile stream, not whether the API name contains “asynchronous.” A server queue that fills can return pressure to the client, while a client that waits too early can fail to prepare later independent work. Product-specific traces are needed to determine which case limits an actual kernel.

8.5 TPU uses asynchronous DMA with explicit completion ownership

Pallas TPU has scalar control and scalar memory for control operands, distinct from vector and matrix computation. Its documented grid execution and memory movement model differs from GPU warp oversubscription: the compiler arranges local-memory transfers and supported multicore partitioning is explicit. Asynchronous engines can advance while scalar control performs other work, but that fact does not create a second independently executing scalar instruction stream. [51]

The distributed Pallas interface exposes device-issued asynchronous push DMA within a TPU pod. A descriptor includes distinct send and receive semaphores. Starting the copy can be followed by independent work; the send completion protects local source reuse, while the receive completion protects destination use. The distributed guide demonstrates buffering and pipelining and warns that mismatched waits or counts can cause hangs, crashes, or corruption. These are explicit device-side communication capabilities, so a model that routes every TPU tile exchange through host software would be an unfair baseline. [53]

The newer core-specific interface exposes physical core meshes, per-core storage/semaphores, and remote-copy examples between cores. It is a distinct interface from older Megacore guidance; their assumptions should not be silently merged. Multiple cores can split control responsibility, but the partition adds ownership, balance, and communication obligations. [52]

A custom fused Pallas kernel may also obscure opportunities that XLA’s higher-level scheduling could exploit. Its local benefit must be compared with the compiler-managed baseline, including asynchronous collective scheduling and neighboring fusion. The retained-state ledger includes VMEM/SMEM, DMA descriptors, semaphore values, in-flight buffers, and ICI contention. An independently progressing DMA protects progress only for work already submitted with adequate destinations and matching completion behavior. It cannot prepare a future descriptor that the stalled scalar program has not issued.

8.6 Tenstorrent gives data movement dedicated control roles

TT-Metalium’s data-movement programming example assigns separate kernels to the two data-movement RISC-V processors in a Tensix core. Reader, compute, and writer work can therefore be expressed as different roles. Circular buffers provide explicit producer/consumer capacity and readiness operations. This division can reduce competition for the compute role’s instruction

sequence while preserving dependencies through buffer ownership. The cost includes controller instruction/code state, circular-buffer memory, and the protocol connecting those roles. [140, 142]

Within a chip, NoC commands, barriers, and remote semaphore updates move data and coordinate consumers. Ordering is scoped by the chosen NoC, command buffers, and virtual channels. Separate NoCs or VCs do not gain a single total order merely because one controller issued commands in source order. A producer must establish the payload-before-signal edge using the appropriate ordered path or completion operation. [141]

Across chips, the Wormhole/Blackhole Ethernet guide describes staging through Ethernet-core L1 and asynchronous sends. The low-level send primitive does not itself supply the full receiver-completion and source-reuse protocol. Queue-busy handling, acknowledgements or credits, and teardown must be implemented or supplied by a higher-level fabric/collective facility. The guide recommends fabric infrastructure for managing Ethernet cores; comparing only link bandwidth would omit movement between Tensix and Ethernet storage and the resources consumed by fabric workers. [139]

This approach makes control agents visible and programmable, which can be advantageous for known streaming graphs and explicit placement. More stages, however, mean more protocol state, fill/drain work, and potential recovery boundaries. A dedicated reader can still block on a full circular buffer; a writer can still need credits returned by another participant. Dedicated controllers make independent progress possible but do not prove it under all backpressure conditions.

8.7 Superscalar NPU alternatives and their limits

The reviewed local timing-model record establishes readiness and bounded resource management within its scope. It does not establish the number of independently progressing scalar contexts, a complete network DMA protocol, or a cross-card event fabric. Those capabilities must therefore be treated as alternatives to design and evaluate, not as inherited properties of the word “superscalar.” Four approaches address different parts of the problem.

First, a stronger scalar front end can process independent address, descriptor, and loop-control operations faster. Its benefit depends on available instruction-level parallelism and shared issue resources. It cannot remove a true wait dependency, a saturated server queue, or a missing remote credit. A wider front end also requires evidence about code footprint, operand dependencies, execution units, and attainable clock frequency.

Second, multiple contexts or dedicated controllers can separate compute orchestration, communication submission, and completion service. Each requires a program counter or equivalent state, registers, queues, and a scheduling policy. A blocked context must not hold every resource needed by its unblocking agent. Fair arbitration of shared memory and descriptor ports remains necessary. Context count is therefore a resource and progress decision, not a free multiplier on scalar throughput.

Third, event and DMA offload can let hardware or firmware advance already-described transfers and dependency updates. This can lower instruction overhead per tile and keep progress independent of a compute wait. It requires explicit ordering scopes, source-lifetime and destination-visibility semantics, finite event allocation, generation-safe reuse, and error/cancellation behavior. Offloading an incorrectly specified dependency can make a race faster rather than making it correct.

Fourth, the compiler can generate asynchronous roles and place waits late, using resource-aware buffer planning. Static knowledge remains valuable even in a dynamically scheduled machine: constant descriptors, repetitive routes, and known shape constraints can be hoisted or amortized. The ISA/runtime contract must define which layer owns event IDs, queues, and buffer handles,

especially when a library and a custom kernel compose. Dynamic hardware can select among ready operations while the compiler still limits the offered working set.

A useful conditional combination is a compiler-planned pipeline with hardware-managed local dependencies and an independent bounded completion path. It could reduce explicit polling and barrier bookkeeping while avoiding unnecessary stalls on variable local readiness. Its proof obligations extend beyond average throughput: completion must still progress when compute is blocked, queues fill, or a peer arrives late. The next two chapters address those obligations.

8.8 What a matched evaluation must observe

A credible experiment would record the control instructions and events per tile, time spent preparing and issuing work, outstanding descriptors, queue-full stalls, live buffers, and the agent responsible for each wait. It would separately measure fill, steady state, and drain across tile sizes and rank counts. Register or local-memory growth from added roles must be included, as must host/runtime setup when it is not amortized. No fixed orchestration penalty or cross-platform speedup is inferred here.

The decision rule is whether the chosen control organization lets every necessary event arrive soon enough without excessive state or interference. Fusion removes some boundaries and brings others inside a kernel. The architecture succeeds when it makes those internal boundaries inexpensive, precise, and live under the actual resource limits, rather than merely invisible in source code.

9 U007 Precise faults and ordering across asynchronous engines

The useful correctness contract for a fused accelerator must say which effects have occurred, which state can be inspected, and which operations can safely be repeated. A matrix operation can finish after a younger scalar instruction, a vector consumer can start before an unrelated transfer finishes, and a remote receiver can consume an output before the sending host observes completion. These are ordinary consequences of overlap. The difficulty arises when a programmer, debugger, or recovery mechanism treats all these events as one undifferentiated notion of completion. The design objective is therefore to preserve necessary causal edges while giving faults a well-defined attribution and recovery boundary. Serializing every engine makes some executions easier to inspect, but discards much of the overlap that U001 and U006 seek to expose.

Consider a distributed tile pipeline. A local matrix stage produces an intermediate, a vector stage transforms it, a communication engine transfers the result, and a remote stage consumes it. The algorithm determines the producer-consumer relation. The ISA or device API determines how that relation becomes an ordering or completion guarantee. The compiler translates the relation into commands and synchronization; the runtime determines submission, participant identity, and failure handling; hardware tracks the resulting state. A missing edge can originate in any of these layers. Conversely, moving an edge from explicit software to dependency-tracking hardware changes its implementation and ownership, without eliminating the edge or the resources needed to enforce it.

9.1 Four contracts that must not be conflated

A *precisely attributed fault* identifies the operation that caused a failure, possibly including an instruction address, tile identifier, engine, memory address, and source location. A *precise recoverable state* is stronger: the saved architectural state corresponds to a specified instruction boundary, and execution can resume according to a stated restart rule. The classical precise-

Contract	Question answered	What does not follow automatically
Fault attribution	Which operation caused this error?	Registers and memory form a restartable snapshot
Precise local state	At which specified boundary can this domain resume?	Remote consumers can undo escaped effects
Memory ordering	Which observations must precede which others, for which observers?	All transfers are complete or all participants have arrived
Completion	Which operation milestone has been reached?	Every consumer has stopped using the payload
Ownership release	Who may overwrite or reclaim this storage now?	The containing pool can be deallocated
Trace or replay	Which events were recorded or can be reproduced?	Every race or unrecorded schedule has been covered

Table 1: Correctness and observability contracts answer different questions. Each guarantee needs an explicit scope.

interrupt definition is based on a sequential architectural boundary; in-order completion is one implementation, while retained versions and recovery structures permit execution and completion to be less restricted [116]. For a modern accelerator, the definition must identify its recovery domain and the status of pending effects outside that domain.

A *memory-order contract* specifies permissible observations and synchronization relationships, including address spaces, atomicity, participants, and scopes. It does not necessarily impose one total order over the machine. A *completion and ownership contract* specifies when an operation has reached a named milestone and when a resource may change owners. An operation can have finished reading its source while its destination remains unavailable to a consumer. A receiver can have received a tile while still retaining it for computation. These cases require different events even when one implementation happens to report several of them together.

An asynchronous error adds a further distinction between the *cause* and the *reporting point*. For example, the CUDA driver documents that synchronization-related calls can report errors from earlier asynchronous launches [83]. The call returning an error is thus a useful observation boundary, but need not be the offending operation. A debugger likewise needs to distinguish the faulting program counter from the later counter at which an engine stopped. Even a correct fault address cannot reconstruct a tensor already overwritten by younger work.

9.2 A publication and reuse example

Let device A produce tile X_e for epoch e in source slot S , and let device B receive it into slot D . Define events W_e for the final producer write, I_e for transfer issue, S_e for source-read completion, D_e for destination arrival, P_e for publication of readiness, C_e for the end of the consumer’s reads, and A_e for acknowledgment that the destination slot can be reused. The following is an abstract protocol requirement, not a vendor instruction sequence:

$$W_e \prec I_e, \quad S_e \prec \text{overwrite}(S), \quad (34)$$

$$D_e \prec P_e \prec \text{read}_B(D), \quad C_e \prec A_e \prec \text{overwrite}_{e+1}(D). \quad (35)$$

The symbol $\prec$ means an established causal relation under the selected API and memory model. It is not a claim that an arbitrary local fence implements that relation. Publication may need a release/acquire pair, a DMA completion event, an ordered data-and-signal primitive, or more than one of these. The choice depends on the path by which payload and signal reach the consumer.

Suppose the payload travels on one channel and the ready flag on another. Source-code issue order alone is insufficient unless the selected interface orders those channels. B can observe the flag while reading old or partially updated data. Adding a local fence with a scope that excludes B also fails to establish the required edge. In another failure, A reuses S after merely enqueueing the transfer, so the transfer observes a mixture of two epochs. In a third, A correctly waits for delivery but overwrites D before B finishes consuming it. All three are synchronization errors, but they require different repairs: publication ordering, source completion, and destination ownership respectively.

The protocol need not place a global barrier after every tile. A finite set of destination slots can carry distinct epochs, and the sender can advance whenever it owns a free slot and the required completion state. This retains overlap while making the ownership proof explicit. It also reveals why an epoch or generation identifier is useful: a delayed readiness signal for e must not satisfy a wait for $e + 1$. Whether a finite identifier can wrap safely depends on the maximum lifetime of outstanding work and the reuse protocol; a wide counter by itself is not a proof.

A communication API may permit source release before the receiver can consume the destination. Pallas remote DMA exposes separate send and receive waits [48]. Tenstorrent’s documented on-chip NoC path similarly distinguishes source-flush completion from destination-arrival completion, while its low-level Ethernet path requires a higher-level acknowledgment protocol for comparable guarantees [130]. These mechanisms instantiate different subsets of the abstract events. They should not be collapsed into one generic barrier in a performance diagram.

9.3 What a local precise-state design must establish

For a proposed superscalar NPU, take a local architectural instruction sequence $i_0, i_1, \dots$ and a retirement frontier k . A precise exception at i_k requires a state consistent with the specified boundary before that instruction, subject to its documented fault or partial-completion semantics. All older architectural effects must be represented as completed effects or irrevocably committed pending effects according to the contract. Younger effects must be invisible to observers covered by the rollback guarantee. This is a local sequential boundary, not a promise of globally sequentially consistent memory.

A sufficient construction for *faults detected before commit* can be stated with four invariants. First, committed register and tile mappings identify the versions produced by the retired prefix. Second, speculative versions cannot overwrite the only recoverable version of any live architectural value. Third, externally observable side effects are either withheld until their issuing operation is irrevocable or covered by an explicitly specified transactional recovery protocol. Fourth, every outstanding operation retains a generation-valid identity and reports either successful completion or a fault before it becomes eligible to commit. An implementation may enforce these invariants with renaming, checkpoints, result buffers, ordered publication, or a mixture; a ROB is only one part of the construction.

The corresponding argument is inductive. Initially the committed mappings and effects represent the empty prefix. A successful retirement updates them only after the instruction’s required results and fault checks are complete, preserving the next prefix. If the oldest faulting instruction reaches the frontier, younger speculative versions are discarded and the previous committed mappings restored. The side-effect invariant prevents discarded instructions from leaving an observable effect inside the promised recovery domain. The identity invariant prevents a delayed response from an old execution from updating a recycled destination. Under these assumptions the recovered state represents the last retired prefix. This argument is conditional on the invariants; their names in a design document are not evidence that every engine enforces them.

Several difficult cases remain visible rather than disappearing into the proof. An in-place tensor

update needs an undo method or a preflight rule strong enough to prevent partial modification on the covered fault classes. A long matrix instruction that permits partial architectural completion needs a restart cursor or another precise description of which suboperations are committed. An error discovered after retirement, such as a later asynchronous integrity failure, falls outside the before-commit proof unless the architecture retains the necessary recovery information. Cancellation must establish whether a command has actually stopped, whether its responses can still arrive, and which resources it continues to own. Silently treating “cancel requested” as “all effects canceled” breaks both state recovery and allocation safety.

Remote publication is the hardest boundary. Suppose a younger instruction on A sends a value that B uses to update its own state, and an older instruction on A subsequently faults. Restoring A’s rename map cannot undo B’s computation. A design must therefore delay the send’s visibility until the relevant local boundary is irrevocable, define the send as an explicit non-rollback boundary, or use a distributed protocol that can coordinate recovery. Delaying publication can retain a large output and add retirement-head blocking. A distributed protocol adds epochs, acknowledgment, duplicate suppression or idempotency, and agreement about what has committed. None follows from an instruction carrying a ROB identifier.

9.4 NVIDIA GPU ownership and tooling

PTX provides an explicit scoped memory model for the documented `sm_70`-and-later scope, with specified exclusions, rather than treating asynchronous execution as unrestricted memory behavior. Its proxy rules also matter when asynchronous engines and ordinary accesses interact; ordinary ordering and asynchronous completion are separate obligations [85]. At the runtime layer, streams and events let software establish dependencies without synchronizing the entire host and device. A stream wait can defer downstream work while unrelated work remains eligible [82].

NVIDIA’s debugging facilities expose useful but different contracts. Compute Sanitizer distinguishes precisely attributed instrumented access errors from less precisely attributed hardware exceptions. Its race and synchronization checking have documented coverage; they do not prove arbitrary global or distributed code race-free, and error termination does not roll back earlier outputs [81]. CUDA-GDB documents fault-origin reporting separately from the later stopping PC, and offers autostep to improve localization in selected regions [84].

The algorithm or library chooses synchronization scope and buffer ownership; the compiler preserves the relevant source semantics; runtime events connect launches; device mechanisms implement the specified ordering and completion. The resulting control inventory includes per-thread or warp execution state, readiness and dependency tracking, barrier state, asynchronous command identities, and debug instrumentation where enabled. The exact circuit topology need not be known to recognize these functions. Conversely, profiler or debugger features do not establish a CPU-style ROB implementation or universal instruction restart.

The benefit condition is concrete: explicit scopes and event dependencies are valuable when independent work exists outside the required dependency cone. Broad waits may be simpler to audit but retain less overlap; finer dependencies require more precise ownership reasoning and state. Instrumentation can improve attribution while perturbing the schedule that exposed the original error. A fair comparison therefore records production semantics, diagnostic mode, and recovery scope separately rather than comparing an instrumented GPU with an uninstrumented proposal.

9.5 Ascend compiler and framework synchronization

Ascend C describes scalar issue to asynchronous vector, matrix, and movement pipelines. In the cited CANN 8.5 interface, cross-pipeline flags connect producer completion to consumer execution, while pipeline barriers constrain a selected pipeline. The compiler and framework insert substantial synchronization, including documented Vector and Cube responsibilities; manual cases remain when the automatic analysis or movement-address assumptions are insufficient. The documentation distinguishes portable TQueSync interfaces from hardware-sensitive ISASI forms [35]. Consequently, describing Ascend kernels as universally hand-scheduled barriers would overstate the author burden, while treating framework synchronization as an unrestricted proof would hide its assumptions.

msSanitizer documents memory-hazard checking across supported memory and pipeline combinations, with source-context reporting and release-dependent coverage [8]. msDebug supports hardware debugging and core analysis, but explicitly warns that execution can continue for several instructions before an exception is reported: saved registers and memory can be inaccurate even when the PC is corrected [7]. This is direct evidence that accurate attribution and coherent recoverable state must be assessed separately, without dismissing the diagnostic value of the tools.

Responsibility is distributed across algorithmic staging, tensor-lifetime analysis, framework-owned flags, runtime completion, and hardware pipelines. The design costs include event state and ownership, local buffer lifetimes, waits inserted conservatively when aliases are uncertain, and the resources used by diagnostic instrumentation. Automation is especially valuable when a kernel stays inside the framework’s analyzable ownership model. Custom aliasing, unusual pipeline interactions, or library composition can shift obligations back to the author. The cited term “committed” in a pipeline-barrier description should be interpreted under that API, not promoted to an unprovided architectural retirement guarantee.

9.6 TPU compiler dependencies and low-level Pallas control

OpenXLA uses tokens to express side-effect dependencies, including joins through **AfterAll** [97]. This is a compiler-IR contract. It does not reveal a complete physical TPU memory model or imply that independent physical engines execute sequentially. Pallas makes a lower-level boundary visible: software pipelining coordinates asynchronous units and buffer lifetimes, trading additional storage for earlier movement and later waits [49]. The compiler can absorb regular scheduling work when the kernel supplies the necessary dependence and lifetime information.

For distributed kernels, the programmer still needs correctly matched transfers, synchronization counts, slot reuse, and participant identity. Pallas documents separate completion milestones and failure modes for mismatched synchronization [48]. Its debugging guide describes interpreted race detection and compiler-pass dumps [47]. A simulation can explore memory and synchronization behavior and a compiler dump can expose a missing edge; neither is evidence of the exact silicon execution that failed, nor proof that all permitted schedules were checked.

The costs arise in both generated and explicit control: semaphore state, buffer versions, scheduling metadata, and the retained payload needed while communication overlaps computation. A compiler can simplify the normal programmer’s contract without removing these functions. The proposed superscalar alternative should therefore be compared against the compiler’s optimized schedule and actual debug facilities, not against an imagined TPU with no synchronization or visibility. Public evidence reviewed here leaves general instruction-level precise restart unspecified; that is an evidence boundary, not a claim that a particular hidden microarchitecture cannot support recovery.

9.7 Tenstorrent explicit ownership and scoped communication

TT-Metalium documents that writes on the same NoC and virtual channel follow program order; using different NoCs or virtual channels does not automatically preserve that order [132]. Reader, compute, and writer roles must connect movement completion to circular-buffer publication and consumption. A queue-ready condition says that the producer has published data under the queue contract, so the producer must establish the required payload completion first. The receiver’s eventual slot release is another event.

Watcher provides kernel identities, per-controller waypoints, and selected address or bounds checks. Its current hardware-exception PC/cause reporting is explicitly Quasar-specific [133]; that capability cannot be silently attributed to Wormhole or Blackhole. The experimental NoC Debug Dump reports selected missing barriers and outstanding writes, while warning that timing can hide faults and tracing adds overhead [131]. Waypoints and traces are valuable localization aids, but do not constitute a coherent global checkpoint.

The ownership split is particularly visible: kernels and libraries arrange roles, channels, buffer handoff, and completion; controllers and engines advance the specified operations; runtime or fabric software manages broader program lifetime. The benefit is that communication and compute control can be separated and overlapped with explicit storage boundaries. The price includes controller instructions, buffer and semaphore state, trace capacity, and the need to compose several local protocols correctly. This is a different distribution of control from a superscalar instruction window, not an absence of control logic.

9.8 The proposed superscalar NPU and the full cost of precision

The supplied project inspection records ROB control, renamed identities, completion and non-flush gates in a timing model, with a separate typed bring-up path containing its own identity and recovery contracts. These observations motivate a local precise-state design; they do not establish one integrated, verified implementation. In particular, neither a model component nor a source-level retirement routine proves that every tensor update, local transfer, atomic, and remote effect obeys the invariants above.

The proposal’s strongest plausible advantage is a simpler local programming and debugging boundary: source-level operations could retain stable provenance while dependency tracking permits safe internal overlap. That advantage applies where the architecture can identify dependencies and control publication, and where the cost of retaining recoverable state is acceptable. It becomes weaker for irrevocable remote effects, uncertain memory aliases, very large in-place updates, or long-latency operations that block the retirement frontier. These cases need explicit semantics, rather than a blanket claim that out-of-order hardware makes asynchronous code easy to debug.

A useful provenance chain maps source operation to compiler IR, emitted instruction or command, dynamic epoch, engine issue, completion, retirement, and fault. Many-to-one fusion and one-to-many lowering must remain representable. Trace records should identify lost records and distinguish per-engine order from cross-engine timestamp order. Otherwise a visually tidy timeline can suggest causal edges that the machine never guaranteed. Deterministic replay, if desired, needs a separate account of external inputs and scheduling decisions that influence execution.

The cost ledger must include exception records, tags and generations, committed and speculative mappings, retained payload versions, output-publication buffers when needed, and arbitration on recovery paths. Retaining an old tile can occupy an existing physical pool rather than require a new full-tile copy; the important quantity is unavailable capacity over time, not only copy count. Let b_v be the bytes occupied by version v and $[a_v, r_v)$ its retention interval. The analytical

occupancy integral

$$\mathcal{R} = \sum_v b_v(r_v - a_v) \quad (36)$$

helps compare recovery schemes with different lifetimes, while peak occupancy determines capacity feasibility. Neither quantity alone predicts energy, area, or throughput. Metadata width, array ports, selection logic, checkpoint traffic, and the critical timing path require implementation evidence.

The appropriate correctness result is therefore a stated recovery domain and a proof or verification argument for every effect that can cross it. The appropriate performance result is reduced unnecessary ordering or debugging work after counting the storage and control that preserve that domain. Local retirement, scoped memory order, asynchronous completion, and distributed recovery remain separate contracts whose composition must be designed deliberately.

10 U008 Forward progress under finite resource constraints

Correct dependency tracking does not ensure that a dependent operation will ever execute. A producer can own a valid tag and a correctly counted destination reference while waiting indefinitely for storage; the operation that would release that storage can itself be unable to enter the machine. Backpressure preserves a capacity limit by stopping admission or movement, but can also propagate a wait to the only useful releaser. The architectural question is therefore specific: what may admitted work retain while blocked, who can remove the blocking condition, and why can that agent obtain every resource it needs?

This question spans the layers already encountered in U002 and U006. The algorithm creates dependencies and determines whether a consumer can run without another input. The ISA and API define waits, completion, ownership, and permitted execution scopes. The compiler chooses live ranges, buffering, and stage order. The runtime admits kernels, assigns participants, and manages communication epochs. Hardware allocates finite entries and arbitrates service. Static scheduling, dynamic readiness selection, and explicit dataflow distribute these tasks differently; each still needs a capacity and progress argument.

10.1 State the property before selecting a mechanism

A safety property forbids an invalid event, such as overwriting a live tile or accepting a stale completion. A liveness property requires a useful event eventually to occur. The difference matters because a machine can protect every live value by waiting forever. Four progress outcomes should be kept distinct:

Deadlock freedom

No reachable unfinished state contains a permanently blocked set under the stated environment and execution assumptions. A closed circular resource wait is one cause of deadlock; waiting for a signal that no participant will ever produce is another.

System-wide useful progress

Useful operations continue to complete while work remains. This can hold while one particular operation is indefinitely bypassed.

Starvation freedom

Every admitted operation within the promised scope eventually receives the service required to complete, assuming its external prerequisites satisfy the contract.

Bounded completion latency

Completion occurs within a specified bound. Eventual service alone supplies no finite

numerical upper bound.

A long stall can be legal if its producer or external service eventually responds. Livelock is different again: retries, cancellations, or state changes continue, but no useful result completes. A timeout detects an exceeded waiting interval; without additional evidence it does not identify which of these conditions occurred.

Progress is relative to a scope and model. A resident thread, a workgroup, a complete kernel, a device, and a distributed collective need different assumptions. Research on GPU workgroup progress formalizes how termination depends on the selected relative-progress model [117]. That distinction is useful here; historical experiments on particular devices are not certification of every later scheduler. Similarly, an eventual-service assumption for the network excludes a permanently failed peer. A fault-tolerant claim must say how such a failure changes the goal from normal completion to detected failure, resource recovery, or job restart.

10.2 A capacity ledger and a resource-wait graph

For each finite pool r , partition its capacity into occupied storage O_r , reservations for future occupancy V_r , and unassigned capacity F_r :

$$C_r = O_r + V_r + F_r, \quad O_r, V_r, F_r \geq 0. \quad (37)$$

These categories are disjoint. An arriving payload converts an existing reservation into occupancy; it must not consume a second credit for the same slot. If in-flight data has no destination reservation, some other documented buffer or backpressure rule must be able to retain it. “The load has not bound its final tile” says nothing by itself about where the returned bytes wait. Logical identities, payload slots, transfer descriptors, completion records, and execution contexts are separate pools even when a particular implementation couples their allocation.

A useful wait-for graph has operations and resource instances, not merely data dependencies. An edge from an operation to a resource means that the operation needs that resource to advance. An edge from a resource to an operation identifies its holder or the operation that must run to release it. Signals can be represented through their producing operations. For multi-instance pools, a cycle in a coarse graph is only a warning: a free instance, a preemptible holder, or another release route can break it. A deadlock claim needs a reachable state, the actual capacity counts, and evidence that every relevant route is blocked.

Acyclic algorithmic dataflow does not eliminate these resource edges. Two independent subgraphs can contend for a shared command pool and turn an otherwise straightforward schedule into hold-and-wait. Conversely, cyclic algorithmic dataflow can be live with the correct initial tokens and capacity. The relevant graph must therefore include the initial state, allocation order, release conditions, and any independent completion mechanism. Counting only matrix and vector operations omits the control work on which their continued issue depends.

10.3 A worked saturation example

The following small machine is an illustrative construction, not a report of a defect in any vendor product or the supplied project model. It has two physical payload slots P_0, P_1 , two normal command entries Q_0, Q_1 , and two return-buffer slots R_0, R_1 . At the chosen point, P_0 and P_1 contain completed outputs x and y . Both are safe to publish and have no remaining local consumers. Each output can release its payload slot after a drain transfer has finished reading it. A drain requires a normal command entry in the baseline machine.

Now admit two independent loads, L_0 and L_1 . Each retains its command entry until its returned value has been bound into a physical payload slot. Their returns z_0, z_1 have arrived

Pool	Capacity	Occupancy at the blocked state	Condition needed for release
Payload P	2 tiles	Completed outputs x, y	A drain obtains a command entry and finishes reading
Commands Q	2 entries	Loads L_0, L_1	A return binds into a free payload slot
Returns R	2 tiles	Returned values z_0, z_1	The corresponding value binds into P

Table 2: Illustrative saturated state. The example’s release rules are assumptions, not inferred hardware policies.

and occupy R_0, R_1 . The loads wait for free payload slots; draining x or y waits for a command entry. The allocation policy allowed these two loads to consume the last normal command entries before admitting either drain. No cancellation, spill, preemption, or alternate command path exists in this hypothetical baseline.

There is a concrete circular wait:

$$\text{loads hold } Q \longrightarrow \text{loads need } P \longrightarrow \text{drains must release } P \longrightarrow \text{drains need } Q. \quad (38)$$

The return buffers are full but are not the essential cycle in this construction. Increasing their depth alone cannot make a command entry or payload slot available. Fair selection among the loads also cannot help: both are ineligible for the missing capacity. Nor does a larger instruction window help if every newly visible drain must acquire the same exhausted Q pool. The failure is a resource-admission and release-path problem, despite the absence of a data dependency from x or y to either new load.

Several repairs are possible. Admission could keep one normal command entry available for drains. The scheduler could issue the drains before allowing the loads to occupy all command entries. Command identity could be decoupled from the resource retained during return binding, provided some other completion-tracking capacity is guaranteed. A completed output could be spilled through an independently provisioned path. Each repair changes a particular hold-and-wait edge and has a cost. Merely adding a retry loop changes no edge.

10.4 An escape path with a conditional progress argument

For a more explicit repair, add one escape command entry Q_E and one escape completion record E_E , unavailable to ordinary loads. An escape drain can read an already publishable payload directly, transmit it to a sink with prearranged capacity, and release its payload after source-read completion. Its descriptor, arbitration, completion notification, and capacity return do not require Q or R . The drain may use a shared data path only if it is guaranteed eventual service there. A reserved descriptor is ineffective if the request subsequently joins a blocked normal completion queue.

Temporarily stop admitting new work at saturation, and give the escape path a finite batch of already admitted work to retire. For this example, assume all four payloads x, y, z_0, z_1 are terminal outputs, each safely drainable once resident. An allowed sequence is shown in Table 3. A dash denotes a free slot; entries show the state after each macrostep, rather than a cycle-accurate timing trace.

A feasible sequence alone proves only reachability of completion. To obtain a liveness argument, specify the controller and its service assumptions. It retains a selected drain until that drain completes; the selected drain’s remaining local and external steps receive eventual service. When a return and a free payload slot coexist, binding receives eventual service, and its bookkeeping

Completed macrostep	Payload P	Normal Q	Returns R
Initial blocked state	x, y	L_0, L_1	z_0, z_1
Escape-drain x	$-, y$	L_0, L_1	z_0, z_1
Bind z_0 and finish L_0	z_0, y	$-, L_1$	$-, z_1$
Escape-drain y	$z_0, -$	$-, L_1$	$-, z_1$
Bind z_1 and finish L_1	z_0, z_1	$-, -$	$-, -$
Drain z_0 and z_1	$-, -$	$-, -$	$-, -$

Table 3: One feasible drain sequence for the explicitly bounded example. The existence of this sequence is supplemented by service and scheduling assumptions in the text.

can release Q and R without allocating from either pool. There is no new admission or rollback during this finite drain phase. These assumptions rule out indefinite bypass as well as circular capacity waits in the stated scope.

Let b count the two admitted loads whose binding remains unfinished, and let d count the four output drains that remain unfinished, including outputs not yet bound. Define

$$\Phi = b + d. \quad (39)$$

Every completed bind reduces b ; every completed drain reduces d ; neither step increases the other count because all admitted outputs were counted initially. If P contains a value, an escape drain is available under the example’s terminal-output assumption. If P is empty and $b > 0$, a returned value can bind. The selected macrostep eventually finishes under the service assumptions, so Φ strictly decreases after finite intervals until it reaches zero. Intermediate transfer states need not decrease Φ ; their eventual completion is an explicit premise. This establishes finite-batch local draining and reclamation for the example, without asserting a cycle bound. If a drain releases its source before destination arrival, remote delivery remains an outstanding obligation: its transport records and destination reservation must survive local reclamation. Proving receiver completion requires including that obligation and its eventual-service assumption in the progress measure or in a separate composed argument.

The argument does not cover an arbitrary computation graph. A resident tile might still be speculative, needed by an unfinished consumer, or impossible to drain without another local operand. A remote sink might need the same saturated network path to return a credit. Those cases require a different invariant or a larger protected resource set. Reserving one entry is sufficient only after proving that one entry plus its complete dependency chain can advance. The reserve also consumes capacity that ordinary work could otherwise use, and a drain phase can reduce normal overlap. Its benefit is saturation robustness under this explicit contract.

10.5 NVIDIA GPU residency and collective ordering

CUDA’s ordinary blocks must support independent execution, including sequential scheduling. A resident block therefore cannot assume that an arbitrary nonresident block will execute while it waits. Cooperative grid synchronization uses an occupancy-bounded cooperative launch; compute-capability-9.0 clusters add a co-scheduled group whose scope is the cluster. Independent Thread Scheduling does not supply arbitrary inter-block progress, and Programmatic Dependent Launch explicitly treats concurrency as opportunistic rather than a correctness guarantee [66].

This divides ownership cleanly. The algorithm must use a legal synchronization structure. Compiler-selected register and shared-memory footprints influence residency. The runtime validates or arranges the supported launch, and hardware schedules within that contract. A change in resource footprint can invalidate an informal argument based on a previous occupancy

estimate. Phase-separated kernels can avoid an in-kernel global wait, at the cost of another boundary; cooperative admission can keep required participants resident, at the cost of limiting the admitted grid. Neither mechanism proves a user lock protocol or mismatched barrier correct.

Distributed execution adds ordering among communication operations. NCCL 2.26 introduced optional implicit ordering between communicators on a device, while requiring consistent host launch order across devices; its documented error handling includes asynchronous-error polling and communicator abort/recreation [67]. These are software-contract mechanisms for a bounded class of interactions. They cannot be replaced by an assumption that every queued GPU kernel or collective will run concurrently. The control costs include launch and participant state, resident execution resources, events, and recovery coordination; exact undocumented scheduler fairness is not inferred.

10.6 Ascend lifetimes and core admission

The live Ascend C developer-preview documentation describes a TPipe–TQue model that separates stage handoff from storage reuse. Enqueue/dequeue operations connect producer and consumer stages, while tensor allocation and release govern slots in local pools [38]. The compiler and framework can implement regular stage protocols, but their composition must still fit the finite buffer and synchronization resources. A released tensor slot is useful only if the stage that performs the release can execute.

CANN 9.0 documents an additional admission boundary. Normal scheduling may launch an operator in parts as physical cores become available. The `__schedmode__(1)` qualifier enables batch mode, which waits for the required physical cores before launching the kernel. The documentation recommends this mode for the specified concurrent inter-core-synchronizing scenario to avoid partial operators waiting on one another’s unavailable cores. The illustrated cost is lost overlap between those operators [36]. The live SyncAll developer-preview API also constrains logical block count relative to the physical cores available to the operator [37].

This is a useful example of preventing a residency cycle before it forms. It is scoped to supported products and the documented scheduling mode, not a claim that every Ascend task is gang scheduled. Algorithm authors and libraries choose participants and pair synchronization; the compiler preserves stage dependencies; runtime admission secures the cohort; pipeline hardware advances events and movement. Local queue correctness and cohort admission solve different parts of the problem. Additional buffering may absorb stage imbalance but consumes local memory and lengthens live intervals. Batch admission may simplify an inter-core proof while reducing overlap, and still needs a fair policy among waiting kernels if starvation freedom is promised.

10.7 TPU compiler lifetimes and distributed capacity

Pallas exposes asynchronous unit coordination through compiler-managed pipelines, buffers, and semaphores. Multibuffering consumes local capacity to let movement run ahead of computation [49]. The Pallas asynchronous-operation design also makes lifetime preservation across split start/done operations explicit, including synchronization resources that remain live until completion [50]. A compiler can reason about these intervals only when outstanding work remains represented after its start operation has issued.

Across devices, Pallas documents capacity semaphores or additional slots to restrict producer run-ahead, unmatched-wait failure modes, and barrier-identity discipline across invocations [48]. Capacity control and transfer completion serve different purposes. A received payload can still occupy a slot until its consumer releases it. An old invocation’s signal must not satisfy a later invocation’s rendezvous merely because the same semaphore resource is reused.

The algorithm supplies matching communication and the consumer’s dependency structure. The compiler manages visible local schedules and lifetimes; kernel or library code supplies the distributed protocol; runtime invocation identity and hardware service complete the contract. The benefit of static planning is strongest when shapes and communication structure permit a dependable capacity schedule. Dynamic latency still requires waits or spare slots, and a local schedule does not establish remote service. Costs include VMEM or other scratch occupancy, semaphore and epoch state, scalar control instructions, and startup/drain work. More slots permit skew at a memory cost; stronger per-step synchronization restricts skew by sacrificing overlap. Neither choice is universally preferable.

10.8 Tenstorrent queue and fabric composition

TT-Metalium circular buffers expose bounded producer/consumer ownership. The producer checks capacity before writing and publishing; the consumer waits for available pages and releases them after use. In particular, `cb_reserve_back` is documented as a blocking capacity poll that does not advance a pointer, so its name should not be read as an atomic allocator spanning arbitrary producers [134–136]. Correct page accounting prevents overwrite; it does not prove that a downstream actor can be scheduled or acquire its other inputs.

The Wormhole Ethernet guide describes higher-level flow control using acknowledgments and receiver buffer availability, together with startup and teardown coordination for asynchronously executing programs [130]. The dependency chain includes the controller processing credits and the lifetime of the remote program, not only the payload link. A queue protocol that is live while both programs run can fail to compose with teardown if one side stops before servicing the other’s final acknowledgment.

Algorithm and placement determine graph topology, initial tokens, and buffer depth. Kernels or fabric libraries arrange producer/consumer protocols; controllers and communication engines provide execution and movement; runtime setup establishes participating programs and resources. This explicit decomposition helps expose where capacity is held. It also makes control work visible in RISC instructions, circular-buffer counters, semaphores, routing buffers, and acknowledgments. Increasing L1 buffering can absorb imbalance but cannot repair an unmatched receive or a closed wait involving controllers. The relevant benefit condition is a graph and placement whose local queues and communication lifetimes compose under finite capacity, not an assumption that every dataflow graph is inherently live.

10.9 The proposed superscalar NPU needs a completion proof

The supplied timing-model inspection distinguishes logical tile identity from physical payload binding. A conditional local-load path can defer binding until returned data is ready, while ordinary issue-time binding remains. Guarded release checks consumers, a non-flush boundary, aliases, and physical references. These are useful mechanisms for shortening some holding intervals, with a clearly limited evidence scope. They do not establish that every instruction can bind late, every partial read frees storage, or every admitted operation eventually completes.

Late binding removes an early claim on one pool but extends dependence on another. The unbound operation still needs logical identity, request tracking, return retention, and a future binding opportunity. If it holds an entry needed by a storage-releasing operation, the hypothetical cycle above explains the form of proof required, without asserting that the reviewed model contains that cycle. A dynamic scheduler can exploit a ready releaser only if it is represented in the reachable window, passes ordering gates, and can acquire the resources on its completion path.

Generation tags and reference accounting primarily establish safety. They prevent a stale response from modifying a recycled entry or an old consumer from decrementing a new value’s

count. A liveness argument additionally needs eventual servicing of bind requests, completion records, reference decrements, and releases. A precise-state design from U007 can prolong retention because an old version remains recoverable until retirement; a progress repair must not free that version merely to break a wait. Recovery and liveness therefore constrain the same physical storage, rather than being independent features added after scheduling.

One defensible design method is to enumerate, for each admission class, the resources held before issue, during execution, while awaiting return, and through architectural commitment. Then identify at least one legal route from every admitted unfinished state to useful completion or reclamation, and establish that the permitted scheduler cannot avoid such progress indefinitely. The route can be a compiler-certified phase, a resource-order discipline, reserved completion capacity, or an escape protocol. Its proof must include ordinary and conditional late-binding paths, not only the favorable path drawn in an example. If a retry or rollback is involved, repeated attempts need an argument that excludes unbounded recovery churn.

10.10 Fairness and cost complete the contract

Weak fairness requires eventual service for an action that remains continuously enabled. It does not generally protect an action that repeatedly becomes eligible and loses a short-lived opportunity. Stronger fairness assumptions or a bounded-overtaking policy may be needed when binding and arbitration interact. Aging an instruction can affect selection among eligible candidates, but does not allocate a missing buffer or make a nonresident participant runnable. Admission fairness, issue fairness, network service, and completion/reclamation fairness must be stated at their own boundaries.

A numerical latency claim requires stronger premises still. In a finite batch, if every selected macrostep has a bounded service time and the scheduler bounds the number of overtaking operations, a completion-time bound can be constructed from those limits. Replacing either bound by “eventually” leaves only eventual completion. In an open system with unbounded arrivals, even a deadlock-free queue needs admission or service-rate assumptions before a finite waiting bound is meaningful. These are mathematical conditions for a claim, not performance measurements of the architectures discussed here.

The common ledger includes reserved-but-idle capacity, queue and credit state, generation checks, arbitration and aging, completion bandwidth, extra buffer residence, and the overlap lost to phase or gang admission. A global acquisition order can remove circular hold-and-wait but constrain scheduling freedom. Releasing before waiting can remove an edge but require reconstructible state and safe reacquisition. Escape capacity can secure a drain path but reduces ordinary usable capacity. Timeouts and aborts can bound fault detection or trigger recovery only when recovery itself has a runnable path and a defined effect on outstanding traffic.

A credible forward-progress claim therefore names its participants, capacities, initial state, failure assumptions, release paths, and fairness obligations. Public GPU, Ascend, TPU, and Tenstorrent interfaces provide concrete mechanisms for parts of this contract. The superscalar proposal changes resource lifetimes and selection opportunities, which may reduce stalls under suitable conditions. It must meet the same standard: admitted work retains a complete, safely reclaiming path that the specified execution policy will eventually service. No architecture-wide or distributed guarantee follows solely from backpressure, additional buffering, a ROB, or a larger instruction window.

11 Cross problem synthesis and an evaluation agenda

11.1 The common object is a value with obligations

Across the eight problems, a useful unit of reasoning is a value together with its obligations: the operations that produce it, the valid coordinates and numerical meaning, its consumers, its storage owner, and the events that permit reuse. A value is not exhausted by its payload bytes. A delayed memory request can require identity and return capacity before compute storage is bound. A reduced result can contain only a few logical elements while retaining a larger physical envelope. An asynchronously transmitted value can be locally complete yet still relevant to remote consumption or recovery.

A compiler can establish many obligations statically when control and access patterns are predictable. A kernel can encode them explicitly in thread roles, barriers, and buffers. Hardware can track them dynamically when readiness and resource availability vary. The architectural decision is where each obligation is most economically enforced while preserving a coherent contract between layers. This view avoids treating static scheduling, SIMT, dataflow, and superscalar issue as mutually exclusive answers to every part of an operator.

11.2 Where the proposed mechanisms can help

Fine-grained readiness is useful when consumers need independently valid subsets and when work released early has an available execution path. It provides little benefit if the next stage requires a global statistic, if every candidate waits on the same resource, or if the reduced coordination interval is smaller than the new bookkeeping cost. Partial readiness therefore needs both a semantic proof and a resource feasibility argument.

Late binding is useful when request latency occupies a substantial part of the early-reservation lifetime and when returns can be safely buffered or assigned capacity on arrival. It cannot eliminate the live payload needed by a consumer. Savings in final compute storage must be offset against return buffering, metadata, fragmentation, arbitration, and progress reserves. A system with a full return queue and no way to reclaim final storage can be worse than one with conservative early reservation.

Typed Tiles can carry the logical shape, representation, and axis of an operation, allowing software to avoid repeatedly rebuilding that meaning from individual lanes. This is an expressiveness and compilation benefit when the chosen operations are supported. It does not remove conversion, scale handling, packing, invalid tails, or numerical constraints. An exact-order reduction deliberately limits reassociation; an indexed atomic with returned old values deliberately limits aggregation. Such constraints must remain visible when judging potential throughput.

Independent scalar or event progress is useful when control work would otherwise serialize compute and communication. It must be paired with finite-resource allocation and fair shared-resource access. A second controller that waits on the same exhausted pool may add state without opening a drain path. Event offload that cannot recover or distinguish generations can trade a visible software protocol for a harder-to-debug hardware failure.

11.3 A common experimental design

A future matched evaluation should begin with a workload family rather than one favorable kernel. The dense family should vary independent output count, matrix tile shape, reduction length, dtype, layout, and the proportion of non-matrix work. The irregular family should vary index locality, duplicate rate, fan-out, sparse degree distribution, returned-value use, and supported atomic order. The distributed family should vary tile/message size, rank count,

topology, imbalance, and service delays. These variables expose the conditions in the eight arguments without assuming that one mechanism is always active or useful.

The required outputs include end-to-end latency, steady-state throughput, and separate fill/drain contributions. Storage should be reported as an occupancy trace or live-interval distribution, not only a peak allocation constant. Movement should be counted at each interface, including conversion and staging. Control accounting should include dynamic instructions, events, descriptors, tag and queue state, selection/arbitration work, and time blocked for capacity. Physical comparisons require matched implementation technology and clock/area/power methodology; source-level counts cannot substitute for that evidence.

The numerical baseline must be explicit. Compare exact-order folds with exact-order folds, or state that reassociation is permitted and quantify the resulting error/reproducibility requirement. Distinguish FP32 storage from FP32 arithmetic and accumulation. The memory baseline must likewise be explicit: atomics, acquire/release scopes, result publication, safe reuse, and error recovery must satisfy equivalent obligations. A kernel that omits required synchronization is not a faster implementation of the same contract.

Ablations should disable or constrain one proposed mechanism at a time while keeping the rest of the workload and machine budget meaningful. Examples include whole-Tile versus partial readiness, early versus delayed binding with the same total storage accounting, fixed versus dynamic issue within the same finite window, and shared versus independent completion service. Enlarging every queue to conceal a liveness flaw is not a valid ablation. Each configuration must first satisfy safety and the progress property it claims.

11.4 Correctness and progress before performance conclusions

A useful validation hierarchy starts with executable instruction semantics and boundary cases, then compiler lowering and complete operator equivalence, then finite-capacity protocol testing, and finally performance/physical implementation. It includes shape and mask legality, type-/encoding errors, collisions and old values, deterministic publication, buffer-generation reuse, exceptional numeric status, and memory-order litmus tests. Progress validation must include capacity saturation, delayed peers, simultaneous returns, fairness, cancellation, and the release path itself.

A passing finite test set is evidence, not a general proof. Where a design makes a broad claim such as deadlock freedom, it needs explicit invariants and environmental assumptions, with model checking or another defensible proof method for the relevant state space. Starvation freedom additionally needs an arbitration argument. A bounded latency claim requires bounded service and arrival assumptions, not just the absence of deadlock. These distinctions should survive into performance reports rather than being hidden in a single “works” label.

This article has not executed those model tests, compiler tests, benchmarks, or synthesis flows. It provides the contracts and questions that make such work interpretable. Approximate utilization or improvement percentages in prior advocacy material are excluded because the available provenance does not establish a matched baseline or a reproducible measurement. No novelty claim, architecture ranking, or whole-core PPA conclusion follows from the present evidence.

11.5 Conclusion

A Superscalar NPU is a plausible way to assign selected dependency, readiness, and storage decisions to bounded hardware. Its strongest argument is specific: a value-oriented execution model may reduce exposed coordination and conservative waiting in mixed, irregular, or variable-latency pipelines while preserving scalar programmability. The competing designs already solve

many of these problems through optimized compilation, thread ownership, explicit roles, and asynchronous engines. The question is which placement of responsibility is most effective under a given workload and resource budget.

The eight problems make that decision concrete. Coordination must identify useful granularity; latency hiding must account for all live storage; representation must preserve numeric and layout meaning; irregularity must respect collision and returned-value contracts; reduction must respect physical ownership and permitted order; events must have a progressing agent; faults must have a defined state and visibility boundary; and every admitted operation must have a finite-resource completion path. A successful design closes all eight obligations together. Saving one wait, opcode, or software barrier is valuable only if the complete system remains correct, live, and less costly for the work it must perform.

A Reading and reproducing this article

The article is maintained as modular LaTeX source. Each U001–U008 chapter has its own source file so that technical revisions can be reviewed independently without compressing the material into a slide outline. The accompanying bibliography records public primary sources and their version boundaries. The earlier workshop Markdown analyses remain supporting research; this manuscript does not replace them.

The research snapshot is 7 October 2026. Exact source revisions take precedence over mutable documentation for revision-sensitive statements. Several vendor documents use a release path, while others retain `main` or `latest` URLs with access dates. A final publication should recheck mutable API availability against its named generation. A later page at the same URL is not evidence that a previous implementation had the same feature.

The main contract correction incorporated in U005 is the PTO `TROWSUM` definition at `e182c9b`: typed increasing-column folding from zero, unsupported generic Local CUBE ExecutionMask, and layout-dependent output allocation. The CELL calculations in that chapter are derived from the same pinned geometry. They are byte-capacity examples, not cycle or throughput measurements. In U004, the target architecture is separated from the executable/compiler profile and date-stamped integration metadata. Pending integration is discussed as target intent with validation limits, rather than treated as either absent architecture or completed validation.

The original figures explain dependencies, ownership, and storage geometry. Their distances and widths do not encode measured time, area, power, or bandwidth. Numerical examples follow their stated assumptions; no model or hardware performance result was generated.

Build and review instructions accompany the source. Document-production checks do not validate the processor, compiler, protocols, or physical design.

References

- [1] Dennis Abts, Jonathan Ross, Jonathan Sparling, et al. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture*, pages 145–158, 2020. doi: 10.1109/ISCA45697.2020.00023. URL <https://doi.org/10.1109/ISCA45697.2020.00023>.
- [2] Dennis Abts, Garrin Kimmell, Andrew Ling, et al. A software-defined tensor streaming multiprocessor for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 567–580, 2022. doi: 10.1145/3470496.3527405. URL <https://doi.org/10.1145/3470496.3527405>.
- [3] Arm. SVE Programming Examples. <https://documentation-service.arm.com/static/6659c722876c8d213b7870d3>, 2024. ARM-DAI-0548, release 01.2; especially section A5.1; accessed 7 October 2026.
- [4] Arm. Arm C Language Extensions: SVE. <https://arm-software.github.io/acle/main/acle.html>, 2026. Documentation accessed 7 October 2026.
- [5] Arm. Arm Neon Intrinsics Reference. https://arm-software.github.io/acle/neon_intrinsics/advsimd.html, 2026. Documentation accessed 7 October 2026.
- [6] Arm. Arm C Language Extensions: SVE Addressing Modes. <https://arm-software.github.io/acle/main/acle.html#sve-addressing-modes>, 2026. Documentation accessed 7 October 2026.
- [7] Ascend. MindStudio Debugger user guide. Author-maintained repository, 2026. URL https://github.com/Ascend/msdebug/blob/master/docs/en/user_guide/msdebug_user_guide.md. Mutable master branch checked 7 October 2026; core-dump exception and snapshot limitations.
- [8] Ascend. MindStudio Sanitizer user guide. Author-maintained repository, 2026. URL https://github.com/Ascend/mssanitizer/blob/master/docs/en/user_guide/mssanitizer_user_guide.md. Mutable master branch checked 7 October 2026; racecheck coverage is product and release dependent.
- [9] Dao AI Lab and contributors. FlashAttention csrc softmax local and quad reduction-s. https://github.com/Dao-AILab/flash-attention/blob/94e22c906678e5483fa0e9e24d8e787bc2c0ed4c/csrc/flash_attn/src/softmax.h, 2026. Commit 94e22c906678e5483fa0e9e24d8e787bc2c0ed4c; lines 23–63 and 136–183; retrieved 7 October 2026.
- [10] Dao AI Lab and contributors. FlashAttention Allreduce shuffle helper. https://github.com/Dao-AILab/flash-attention/blob/94e22c906678e5483fa0e9e24d8e787bc2c0ed4c/csrc/flash_attn/src/utils.h, 2026. Commit 94e22c906678e5483fa0e9e24d8e787bc2c0ed4c; lines 111–131; retrieved 7 October 2026.
- [11] Dao-AILab and FlashAttention contributors. Hopper flash forward mainloop with TMA and GMMA warp specialization. flash-attention source repository, 2026. URL https://github.com/Dao-AILab/flash-attention/blob/dc8fd708575a530bf773645efdc36b6bc6c53552/hopper/mainloop_fwd_sm90_tma_gmma_ws.hpp. Commit dc8fd708575a530bf773645efdc36b6bc6c53552, mainloop_fwd_sm90_tma_gmma_ws.hpp, producer and intra-warpgroup overlap paths. Pinned source read 7 October 2026.
- [12] FlashAttention authors. FlashAttention 4 Algorithm and Kernel Pipelining Co Design for Asymmetric Hardware Scaling. <https://arxiv.org/html/2603.05451v1>, 2026. arXiv:2603.05451v1; sections 3.1.2–3.1.3; 2026.

- [13] Google Cloud. TPU architecture. <https://docs.cloud.google.com/tpu/docs/system-architecture-tpu-vm?hl=en>, n.d. Mutable documentation; accessed 6 October 2026.
- [14] Huawei. Basic architecture. Ascend C programming guide, CANN Community Edition 9.0.0, 2026. URL https://www.hiascend.com/doc_center/source/en/CANNCommunityEdition/900/programug/Ascendcopdevg/atlas_ascendc_10_0008.html. The main example is Atlas A2. Accessed 7 October 2026.
- [15] Huawei. FreeTensor. Ascend C API reference, CANN 8.5.0, 2026. URL https://www.hiascend.com/doc_center/source/en/canncommercial/850/API/ascendcopapi/atlasascendc_api_07_0139.html. Accessed 7 October 2026.
- [16] Huawei. InitBuffer. Ascend C API reference, CANN 8.5.0, 2026. URL https://www.hiascend.com/doc_center/source/en/canncommercial/850/API/ascendcopapi/atlasascendc_api_07_0110.html. Accessed 7 October 2026.
- [17] Huawei. Introduction to TQue. Ascend C API reference, CANN 8.5.0, 2026. URL https://www.hiascend.com/doc_center/source/en/canncommercial/850/API/ascendcopapi/atlasascendc_api_07_0137.html. Accessed 7 October 2026.
- [18] Huawei. Ascend C: NPU Architecture Version 3510. https://asc.gitcode.com/guide/programming_guide/advanced_programming/hardware_implementation/architecture_spec/npu_arch_3510.html, 2026. Public development-preview documentation; generation-scoped SIMD/SIMT, storage, and Fixpipe mechanisms; accessed 7 October 2026.
- [19] Huawei. Ascend C Cast API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/memory_vector_compute/type_conversion/Cast.md, 2026. CANN 9.1; generation-specific type and rounding tables; accessed 7 October 2026.
- [20] Huawei. Ascend C MmadMx API. https://asc.gitcode.com/api/SIMD-API/basic_api/cube_compute_ISASI/mmad_compute/MmadMx.html, 2026. 950PR/950DT-specific microscaling matrix interface; accessed 7 October 2026.
- [21] Huawei. Ascend C SetAtomicAdd API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/atomic_operations/SetAtomicAdd.md, 2026. CANN 9.1; source review 7 October 2026.
- [22] Huawei. Ascend C Gather API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/memory_vector_compute/scatter_gather/Gather.md, 2026. CANN 9.1; accessed 7 October 2026.
- [23] Huawei. Ascend C Register-Vector Histograms API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/reg_vector_compute/histogram_compute/Histograms.md, 2026. CANN 9.1; source review 7 October 2026.
- [24] Huawei. Ascend C AtomicAdd API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/atomic_operations/AtomicAdd.md, 2026. CANN 9.1; 950 scalar GM atomic path; accessed 7 October 2026.

- [25] Huawei. Ascend C Scatter (ISASI) API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/basic_api/memory_vector_compute/scatter_gather/Scatter_ISASI.md, 2026. CANN 9.1; product and duplicate-offset restrictions; accessed 7 October 2026.
- [26] Huawei. Using the Reduction Instruction Properly in Different Scenarios. https://www.hiascend.com/doc_center/source/en/canncommercial/800/opdevg/ascendcbestP/atlas_ascendc_best_practices_10_0031.html, n.d.. CANN 8.0; retrieved 7 October 2026.
- [27] Huawei. SoftMax API. https://www.hiascend.com/document/detail/en/CANNCommunityEdition/910/API/ascendcopapi/docs/en/api/SIMD-API/advanced_api/activation_functions/SoftMax_api/SoftMax.md, n.d.. CANN 9.1; source packet records update 17 September 2026; accessed 6 October 2026.
- [28] Huawei. WholeReduceSum. https://www.hiascend.com/doc_center/source/en/CANNCommunityEdition/900/API/ascendcopapi/atlasascendc_api_07_0081.html, n.d.. CANN Community 9.0; Atlas A2/A3 type scope distinguished from other products.
- [29] Huawei. Ascend C hardware architecture. https://www.hiascend.com/doc_center/source/en/CANNCommunityEdition/900/programug/Ascendcopdevg/atlas_ascendc_10_0008.html, n.d.. CANN Community 9.0; accessed 6 October 2026.
- [30] Huawei. CrossCoreSetFlag. https://www.hiascend.com/doc_center/source/en/canncommercial/850/API/ascendcopapi/atlasascendc_api_07_0273.html, n.d.. CANN 8.5; retrieved 7 October 2026.
- [31] Huawei. SetFlag and WaitFlag. https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/81RC1beta1/API/ascendcopapi/atlasascendc_api_07_0177.html, n.d.. CANN 8.1.RC1; Chinese reference; accessed 6 October 2026.
- [32] Huawei. HCCL kernel usage. https://www.hiascend.com/doc_center/source/zh/canncommercial/900/API/ascendcopapi/atlasascendc_api_07_0869.html, n.d.. CANN 9.0; Chinese reference; retrieved 7 October 2026.
- [33] Huawei Ascend. NPU architecture version 3510. Official Ascend C documentation, 2026. URL https://asc.gitcode.com/guide/programming_guide/advanced_programming/hardware_implementation/architecture_spec/npu_arch_3510.html. Chinese-language architecture specification. Accessed 7 October 2026.
- [34] Huawei Ascend. FA/FAG operator design introduction. cann-ops-adv source repository, n.d. URL <https://gitee.com/ascend/cann-ops-adv/blob/186db5bf93202bea2bde82795165098fa62ce84a/docs/common/FA-FAG%E7%AE%97%E5%AD%90%E8%AE%BE%E8%AE%A1%E4%BB%8B%E7%BB%8D.md>. Historical operator design, commit 186db5bf93202bea2bde82795165098fa62ce84a. Chinese source. Accessed 7 October 2026.
- [35] Huawei Technologies. Ascend C CANN 8.5: Introduction to synchronization control. Ascend developer documentation, 2026. URL https://www.hiascend.com/doc_center/source/en/CANNCommunityEdition/850/API/ascendcopapi/atlasascendc_api_07_0179.html. Versioned documentation checked 7 October 2026; product-specific pipeline restrictions apply.
- [36] Huawei Technologies. CANN 9.0: SIMD built-in keywords. Ascend developer documentation, 2026. URL https://www.hiascend.com/document/detail/en/CANNCommunityEdition/900/programug/Ascendcopdevg/atlas_ascendc_10_10053.html. Checked 7 October 2026; normal and batch scheduling modes, subject to product support.

- [37] Huawei Technologies. Ascend C SyncAll. Ascend C API documentation, 2026. URL https://asc.gitcode.com/api/SIMD-API/basic_api/sync_control/inter_core_sync/SyncAll.html. Chinese-language developer preview checked 7 October 2026; page identifies master build 5c07153f668d with uncommitted changes; logical-block and physical-core constraints.
- [38] Huawei Technologies. Ascend C: Programming principles of the TPipe-TQue framework. Ascend C documentation, 2026. URL <https://asc.gitcode.com/guide/%E7%BC%96%E7%A8%8B%E6%8C%87%E5%8D%97/%E7%BC%96%E7%A8%8B%E6%A8%A1%E5%9E%8B/AI-Core-SIMD%E7%BC%96%E7%A8%8B/%E5%9F%BA%E4%BA%8ETPipe-TQue%E6%A1%86%E6%9E%B6%E7%BC%96%E7%A8%8B/TPipe-TQue%E6%A1%86%E6%9E%B6%E7%BC%96%E7%A8%8B%E5%8E%9F%E7%90%86.html>. Chinese-language master-branch developer preview checked 7 October 2026; AI Core SIMD programming model.
- [39] Intel. Intel AVX-512: Permuting Data Within and Between AVX Registers. <https://builders.intel.com/docs/networkbuilders/intel-avx-512-permuting-data-within-and-between-avx-registers-technology-guide-1668169807.pdf>, 2022. Technology guide; accessed 7 October 2026.
- [40] Intel. Intel 64 and IA-32 Architectures Software Developer Manuals. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, 2026. Official manual index accessed 7 October 2026; instruction review uses Volume 2.
- [41] Intel. Improve Vectorization Performance Using Intel AVX-512. <https://www.intel.com/content/www/us/en/developer/articles/technical/improve-vectorization-performance-using-intel-advanced-vector-extensions-512.html>, 2026. Documentation accessed 7 October 2026.
- [42] JAX Authors. Pallas TPU Tests: Explicit MXU Paths. https://github.com/jax-ml/jax/blob/main/tests/pallas/tpu_pallas_test.py, 2026. Moving source reference recorded in 6 October 2026 review; experimental generation-scoped APIs, not a deployed-kernel guarantee.
- [43] JAX Authors. Writing TPU Kernels with Pallas: Precision Control. <https://docs.jax.dev/en/latest/pallas/tpu/details.html#precision-control>, 2026. Documentation accessed 7 October 2026.
- [44] JAX Authors. Pallas TPU: Matrix Multiplication. <https://docs.jax.dev/en/latest/pallas/tpu/matmul.html>, 2026. Documentation accessed 7 October 2026.
- [45] JAX Authors. Scalar Prefetch and Block-Sparse Computation. <https://docs.jax.dev/en/latest/pallas/tpu/sparse.html#dynamic-block-indexing-with-scalar-prefetch>, 2026. Documentation accessed 7 October 2026.
- [46] JAX Authors. SparseCore Kernel Writing. <https://docs.jax.dev/en/latest/pallas/tpu/sparsecore.html>, 2026. Documentation accessed 7 October 2026.
- [47] JAX Authors. Pallas debugging guide. Author-maintained repository, 2026. URL <https://github.com/jax-ml/jax/blob/main/jax/experimental/pallas/g3doc/debugging.md>. Mutable main branch checked 7 October 2026; interpreted race detection and compiler dumps.
- [48] JAX Authors. Distributed computing in Pallas for TPUs. JAX documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/distributed.html>. Live documentation checked 7 October 2026; DMA completion, capacity, barriers, and failure modes.

- [49] JAX Authors. TPU pipelining. JAX Pallas documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/pipelining.html>. Live documentation checked 7 October 2026; Pallas TPU programming interface.
- [50] JAX Authors. Pallas async operations. JAX Pallas design documentation, 2026. URL https://docs.jax.dev/en/latest/pallas/design/async_note.html. Experimental design note checked 7 October 2026; asynchronous dependency and lifetime representation.
- [51] JAX contributors. Writing TPU kernels with Pallas. <https://docs.jax.dev/en/latest/pallas/tpu/details.html>, n.d.. Mutable documentation; retrieved 7 October 2026.
- [52] JAX contributors. Pallas Core specific Programming. https://docs.jax.dev/en/latest/pallas/tpu/core_map.html, n.d.. Mutable documentation; retrieved 7 October 2026.
- [53] JAX contributors. Distributed Computing in Pallas for TPUs. <https://docs.jax.dev/en/latest/pallas/tpu/distributed.html>, n.d.. Mutable documentation; retrieved 7 October 2026.
- [54] JAX scaling book contributors. How to Think About TPUs. <https://github.com/jax-m1/scaling-book/blob/main/tpus.md>, n.d. Appendix A v5p discussion; mutable main branch; retrieved 7 October 2026.
- [55] LinxISA Contributors. Linx TileOP API: Typed Operation Emitters. https://github.com/LinxISA/Linx-TileOP-API/blob/fac242a95bc70cd1783547555e5bd7335d1969bf/include/jcore/template_asm.hpp, 2026. Pinned revision fac242a95bc70cd1783547555e5bd7335d1969bf; conversion, row/column reduction, and retirement guards.
- [56] LinxISA Contributors. Linx TileOP API: Tile Geometry and Physical Layout. https://github.com/LinxISA/Linx-TileOP-API/blob/fac242a95bc70cd1783547555e5bd7335d1969bf/include/common/pto_tile.hpp, 2026. Pinned revision fac242a95bc70cd1783547555e5bd7335d1969bf; source review 6 October 2026.
- [57] LinxISA Contributors. Linx TileOP API Elementwise Integration. <https://github.com/LinxISA/Linx-TileOP-API/pull/249>, 2026. Head a70915a5e18e109d680d82068622a42e3a24138e recorded 7 October 2026, 05:28 UTC; availability separate from ISA scope.
- [58] LinxISA Contributors. LLVM Integration: Logical Indices, TLEA, and Elementwise Lowering. <https://github.com/LinxISA/llvm-project/pull/117>, 2026. Head 74c93843 recorded 7 October 2026; earlier 05a1e626 recognizer review is historical; new-head execution validation incomplete.
- [59] LinxISA Contributors. SuperScalarModel: Indexed Address and Atomic Candidate Path. https://github.com/LinxISA/SuperScalarModel/blob/aa91094d3f41fd8aef03c2c96072df32823faf38/TimingSim/group/tlsu/L2/l2_agu.cpp, 2026. Pinned candidate aa91094d3f41fd8aef03c2c96072df32823faf38; bounded development profile, not comprehensive validation.
- [60] LLVM Project. Auto-Vectorization in LLVM. <https://llvm.org/docs/Vectorizers.html>, 2026. Documentation accessed 7 October 2026.
- [61] LLVM Project. MLIR Quantization Dialect. <https://mlir.llvm.org/docs/Dialects/QuantDialect/>, 2026. Documentation accessed 7 October 2026.
- [62] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. <https://arxiv.org/abs/1805.02867>, 2018. arXiv:1805.02867v2; 28 July 2018.

- [63] NVIDIA. Improving Network Performance Using NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async>, 2022. 22 November 2022; numerical results not reused.
- [64] NVIDIA. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/archive/13.0.3/cuda-c-programming-guide/index.html>, 2025. Archived CUDA 13.0.3; memory, atomic, and execution-model sections; accessed 7 October 2026.
- [65] NVIDIA. Parallel Thread Execution ISA 8.7: Sparse Matrix Instructions. https://docs.nvidia.com/cuda/archive/12.8.0/pdf/ptx_isa_8.7.pdf, 2025. Archived PTX 8.7, section 9.7.14.6; source review 6 October 2026.
- [66] NVIDIA. CUDA C++ Programming Guide 13.0. NVIDIA archived developer documentation, 2025. URL <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-programming-guide/index.html>. Checked 7 October 2026; thread hierarchy, cooperative grid synchronization, and programmatic dependent launch.
- [67] NVIDIA. NCCL 2.26.5: Creating a communicator. NVIDIA archived developer documentation, 2025. URL https://docs.nvidia.com/deeplearning/nccl/archives/nccl_2265/user-guide/docs/usage/communicators.html. Checked 7 October 2026; concurrent communicator ordering and asynchronous error handling.
- [68] NVIDIA. Hopper Tuning Guide, 2026. URL <https://docs.nvidia.com/cuda/hopper-tuning-guide/>. Hopper architecture scope. Accessed 7 October 2026.
- [69] NVIDIA. PTX ISA: Asynchronous warpgroup-level matrix multiply-accumulate instructions, 2026. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/>. PTX ISA 9.4 documentation, section on wgmma. Accessed 7 October 2026.
- [70] NVIDIA. tcgen05 MMA Programming Guide. CUTLASS 4.5.2 documentation, 2026. URL https://docs.nvidia.com/cutlass/4.5.2/media/docs/pythonDSL/mma_docs/tcgen05_programming.html. Blackwell SM100 scope. Accessed 7 October 2026.
- [71] NVIDIA. Multi-head, multi-query, and group-query attention. TensorRT-LLM documentation, 2026. URL <https://nvidia.github.io/TensorRT-LLM/features/attention.html>. Accessed 7 October 2026.
- [72] NVIDIA. Asynchronous data copies. CUDA Programming Guide, section 4.12, 2026. URL <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/asnc-copies.html>. Accessed 7 October 2026.
- [73] NVIDIA. Efficient GEMM in CUDA: Hopper warp specialization. CUTLASS documentation, 2026. URL https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html#hopper-warp-specialization. Accessed 7 October 2026.
- [74] NVIDIA. PTX ISA: Miscellaneous instructions—setmaxnreg, 2026. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/#miscellaneous-instructions-setmaxnreg>. Accessed 7 October 2026.
- [75] NVIDIA. CUTLASS: Blackwell SM100 GEMMs. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionality.html, 2026. Documentation accessed 7 October 2026.
- [76] NVIDIA. Efficient GEMM in CUDA. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html, 2026. Documentation accessed 7 October 2026.

- [77] NVIDIA. Parallel Thread Execution ISA: Conversion and Matrix Instructions. <https://docs.nvidia.com/cuda/parallel-thread-execution/>, 2026. Living virtual-ISA documentation; accessed 7 October 2026.
- [78] NVIDIA. Efficient GEMM in CUDA: Hopper Warp Specialization. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html#hopper-warp-specialization, 2026. Documentation accessed 7 October 2026.
- [79] NVIDIA. CUB block reduction warp reduction specialization. https://github.com/NVIDIA/cccl/blob/dc7fcbdc47b3c75ca8871d448fe3a52cbc5a2e8c/cub/cub/block/specializations/block_reduce_warp_reductions.cuh, 2026. Commit dc7fcbdc47b3c75ca8871d448fe3a52cbc5a2e8c; deterministic ApplyWarpAggregates and alternative path distinguished; retrieved 7 October 2026.
- [80] NVIDIA. CUDA C++ Programming Guide shared memory and distributed shared memory. <https://docs.nvidia.com/cuda/archive/13.0.3/cuda-c-programming-guide/index.html#distributed-shared-memory>, 2026. CUDA 13.0.3; sections 5.2 and 6.2.4–6.2.5; retrieved 7 October 2026.
- [81] NVIDIA. Compute Sanitizer user manual. NVIDIA developer documentation, 2026. URL <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>. Live manual checked 7 October 2026; tool coverage and error-action settings are release dependent.
- [82] NVIDIA. CUDA Programming Guide: Asynchronous execution. NVIDIA developer documentation, 2026. URL <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html>. Live manual checked 7 October 2026.
- [83] NVIDIA. CUDA Driver API 13.4: Context management. NVIDIA developer documentation, 2026. URL [https://docs.nvidia.com/cuda/cuda-driver-api/cuda_driver_api/group_CUDA_CTX.html](https://docs.nvidia.com/cuda/cuda-driver-api/cuda_driver_api_group_CUDA_CTX.html). Live manual checked 7 October 2026; asynchronous error reporting notes.
- [84] NVIDIA. CUDA-GDB 13.4 documentation. NVIDIA developer documentation, 2026. URL <https://docs.nvidia.com/cuda/cuda-gdb/index.html>. Live manual checked 7 October 2026; precise attribution and autostep sections.
- [85] NVIDIA. PTX ISA 9.4: Memory consistency model. NVIDIA developer documentation, 2026. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/#memory-consistency-model>. Live manual checked 7 October 2026; architecture and instruction restrictions apply.
- [86] NVIDIA. CUDA C++ Programming Guide warp reduction and cooperative groups. <https://docs.nvidia.com/cuda/archive/13.0.3/cuda-c-programming-guide/index.html#warp-reduce-functions>, n.d.. CUDA 13.0.3; sections 10.21–10.22 and 11.6.3.2.
- [87] NVIDIA. Parallel Thread Execution ISA redux.sync. <https://docs.nvidia.com/cuda/archive/12.8.0/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-redux-sync>, n.d.. PTX ISA 8.7 in CUDA 12.8 archive; section 9.7.13.12.
- [88] NVIDIA. CUTLASS Hopper cooperative warp specialized kernel. https://github.com/NVIDIA/cutlass/blob/main/include/cutlass/gemm/kernel/sm90_gemm_tma_warp_specialized_cooperative.hpp, n.d.. Mutable main branch; implementation example accessed 6 October 2026.

- [89] NVIDIA. Hopper Tuning Guide. <https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html>, n.d.. Sections 1.4.1.1–1.4.1.3; accessed 6 October 2026.
- [90] NVIDIA. NCCL Device Initiated Communication. https://docs.nvidia.com/deeplearning/nccl/archives/nccl_2287/user-guide/docs/usage/deviceapi.html, n.d.. Archived 2.28.7 path; page banner says 2.28.6; GIN text introduces 2.28.7; retrieved 7 October 2026.
- [91] NVIDIA. NVSHMEM Performance. <https://docs.nvidia.com/nvshmem/release-notes-install-guide/best-practice-guide/performance.html>, n.d.. Mutable documentation; retrieved 7 October 2026.
- [92] NVIDIA. Using NVSHMEM. <https://docs.nvidia.com/nvshmem/api/latest/using.html>, n.d.. Mutable documentation; launch and progress rules; accessed 6 October 2026.
- [93] OneFlow contributors. CUDA softmax warp and block all reduce helpers. <https://github.com/Oneflow-Inc/oneflow/blob/25c8978c1c8b1371ef6aa4187dae4495bd233c35/oneflow/core/cuda/softmax.cuh>, 2026. Commit 25c8978c1c8b1371ef6aa4187dae4495bd233c35; lines 47–64; retrieved 7 October 2026.
- [94] OpenXLA Contributors. A Deep Dive into SparseCore for Large Embedding Models. <https://openxla.org/xla/sparsecore>, 2026. Documentation accessed 7 October 2026.
- [95] OpenXLA contributors. XLA GPU emitters. <https://openxla.org/xla/emitters>, n.d.. Mutable documentation; accessed 6 October 2026.
- [96] OpenXLA contributors. XProf Utilization Viewer. https://openxla.org/xprof/utilization_viewer, n.d.. Page updated 5 February 2026; accessed 6 October 2026.
- [97] OpenXLA Project. Operation semantics: AfterAll. OpenXLA documentation, 2026. URL https://openxla.org/xla/operation_semantics#afterall. Live documentation checked 7 October 2026; compiler-IR semantics.
- [98] PTO-ISA contributors. SuperNPUBench one-level attention kernel. SuperNPUBench source repository, 2026. URL https://github.com/PTO-ISA/SuperNPUBench/blob/349c98964144dfd9ca6a2665eca5a16ca8f3e045/benchmark/one-level-arch/kernels/basic_op/fa/fa_2d_unroll_gmma.hpp. Commit 349c98964144dfd9ca6a2665eca5a16ca8f3e045, fa_2d_unroll_gmma.hpp. Pinned source read 7 October 2026.
- [99] PTO-ISA contributors. SuperNPUBench multi-thread result-check synchronization. SuperNPUBench source repository, 2026. URL https://github.com/PTO-ISA/SuperNPUBench/blob/349c98964144dfd9ca6a2665eca5a16ca8f3e045/benchmark/one-level-arch/test/common/multi_thread_res_check.h. Commit 349c98964144dfd9ca6a2665eca5a16ca8f3e045, multi_thread_res_check.h. Verification wrapper only. Pinned source read 7 October 2026.
- [100] PTO-ISA Contributors. SuperNPUBench: Level-One HiF4 Matmul Quantization. https://github.com/PTO-ISA/SuperNPUBench/blob/349c98964144dfd9ca6a2665eca5a16ca8f3e045/benchmark/one-level-arch/kernels/basic_op/matmul/matmul_hif4_l1_quantize.hpp, 2026. Pinned revision 349c98964144dfd9ca6a2665eca5a16ca8f3e045; source review 6 October 2026.
- [101] PTO-ISA Contributors. SuperNPUBench: Low-Precision Attention Kernel. https://github.com/PTO-ISA/SuperNPUBench/blob/349c98964144dfd9ca6a2665eca5a16ca8f3e045/benchmark/one-level-arch/kernels/basic_op/fa/fa_lowp.hpp, 2026. Pinned revision 349c98964144dfd9ca6a2665eca5a16ca8f3e045; source review 6 October 2026.

- [102] PTO-ISA Contributors. SuperNPUBench: Tile and Elementwise Irregular Workload Integration. <https://github.com/PTO-ISA/SuperNPUBench/pull/202>, 2026. Recorded candidate 8c3268baf24097a0cb9cd53cf720f482dd78f42b; existing source and reported evidence, not tests rerun for this article.
- [103] PTO-ISA Contributors. Indexed Memory Byte-Displacement Addressing Contract. <https://github.com/PTO-ISA/pto-spec/blob/e182c9b70d54a3a264bac54af9b915a0e43bb519/asl/tile/model/memory/addressing.asl>, 2026. Recorded main revision e182c9b70d54a3a264bac54af9b915a0e43bb519; source review 6–7 October 2026.
- [104] PTO-ISA Contributors. Global-Memory Atomic and Reduction Contract. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/asl/tile/model/memory/gm-atom-red.asl>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [105] PTO-ISA Contributors. Global-Memory Atomic and Reduction Execution: Preflight and Results. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/asl/tile/model/memory/gm-atom-red-execution.asl>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [106] PTO-ISA Contributors. B.CATR Block Attributes: Contract and Executable Limitations. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/docs/block/attributes/B.CATR.md>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [107] PTO-ISA Contributors. Architectural Memory Events and Coherence. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/asl/arch/memory-model/memory-events.asl>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [108] PTO-ISA Contributors. Tile Execution-Mask State. <https://github.com/PTO-ISA/pto-spec/blob/e182c9b70d54a3a264bac54af9b915a0e43bb519/asl/tile/model/execution/execution-mask-state.asl>, 2026. Recorded main revision e182c9b70d54a3a264bac54af9b915a0e43bb519; source review 6–7 October 2026.
- [109] PTO-ISA Contributors. Architectural Memory Ordering. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/asl/arch/memory-model/ordering.asl>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [110] PTO-ISA Contributors. TLEA Logical-Index Scaling. <https://github.com/PTO-ISA/pto-spec/blob/cab1978fb6056aecd19b475b87ef9c418766d45e/asl/tile/model/execution/lea.asl>, 2026. Target revision cab1978fb6056aecd19b475b87ef9c418766d45e; normative and executable evidence distinguished; source review 7 October 2026.
- [111] PTO ISA contributors. CUBE Cell geometry. <https://github.com/PTO-ISA/pto-spec/blob/e182c9b70d54a3a264bac54af9b915a0e43bb519/docs/tile/model/shape/cube-cell.md>, n.d.. Commit e182c9b70d54a3a264bac54af9b915a0e43bb519; retrieved 7 October 2026.

- [112] PTO ISA contributors. Reduction and expansion legality. <https://github.com/PTO-ISA/pto-spec/blob/e182c9b70d54a3a264bac54af9b915a0e43bb519/docs/tile/model/legality/reduction-and-expansion.md>, n.d.. Commit e182c9b70d54a3a264bac54af9b915a0e43bb519; retrieved 7 October 2026.
- [113] PTO ISA contributors. TROWSUM typed row sum contract. <https://github.com/PTO-ISA/pto-spec/blob/e182c9b70d54a3a264bac54af9b915a0e43bb519/docs/tile/reduce-and-expand/row-reduction/TROWSUM.md>, n.d.. Commit e182c9b70d54a3a264bac54af9b915a0e43bb519; retrieved 7 October 2026.
- [114] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024. URL <https://arxiv.org/html/2407.08608v2>. Version 2.
- [115] Jay Shah et al. FlashAttention 3 Fast and Accurate Attention with Asynchrony and Low precision. <https://tridao.me/publications/flash3/flash3.pdf>, 2024. 2024 paper; sections 2.2 and 3.1–3.2; Hopper specific.
- [116] James E. Smith and Andrew R. Pleszkun. Implementing precise interrupts in pipelined processors. *IEEE Transactions on Computers*, 37(5):562–573, 1988. URL <https://american.cs.ucdavis.edu/academic/readings/papers/smithpleszkun.pdf>.
- [117] Tyler Sorensen, Lucas F. Salvador, Harmit Raval, Hugues Evrard, John Wickerson, Margaret Martonosi, and Alastair F. Donaldson. Specifying and testing GPU workgroup progress models. *arXiv:2109.06132*, 2021. URL <https://arxiv.org/abs/2109.06132>. Submitted 13 September 2021; progress-model formalization, not certification of later architectures.
- [118] SSNPU project. Tile granularity and programming-example source inspection. Supplied inspection record, 6 October 2026. SuperNPUBench commit 349c98964144dfd9ca6a2665eca5a16ca8f3e045; separate Tile API commit fac242a95bc70cd1783547555e5bd7335d1969bf. The two checkouts are not established as one executable’s build provenance, 2026.
- [119] SSNPU project. Timing-model implementation inspection for coordination and storage lifetime. Supplied source-inspection record, 6 October 2026. TimingSim branch feat_pyc_model, commit d5419b3e721df47daae02c38ef2a9b82187e5149. Qualitative implementation evidence only; no new execution or physical implementation result, 2026.
- [120] Tenstorrent. Metalium Programming Guide. tt-metal official repository, 2026. URL https://github.com/tenstorrent/tt-metal/blob/main/METALIUM_GUIDE.md. API-level lifecycle description; moving documentation read 7 October 2026, not a pinned kernel execution result.
- [121] Tenstorrent. Compute engines and data flow within Tensix. TT-Metalium documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/compute_engines_and_dataflow_within_tensix.html. Accessed 7 October 2026.
- [122] Tenstorrent. CircularBuffers. TT-Metalium host API documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/host_apis/buffers/CircularBuffers.html. Accessed 7 October 2026.

- [123] Tenstorrent. Wormhole B0 NoC Counters. Official tt-isa-documentation repository, 2026. URL <https://github.com/tenstorrent/tt-isa-documentation/blob/main/WormholeB0/NoC/Counters.md>. Read-request issue, first-hop acceptance, and response-completion semantics. Accessed 7 October 2026.
- [124] Tenstorrent. cb_reserve_back. TT-Metalium kernel API documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/cb_reserve_back.html. Accessed 7 October 2026.
- [125] Tenstorrent. Compute Engines and Data Flow within Tensix. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/compute_engines_and_dataflow_within_tensix.html, 2026. Documentation accessed 7 October 2026.
- [126] Tenstorrent. Wormhole B0 ISA: Matrix Unit. <https://github.com/tenstorrent/tt-isa-documentation/blob/main/WormholeB0/TensixTile/TensixCoproprocessor/MatrixUnit.md>, 2026. Moving source reference recorded in 6 October 2026 review; Wormhole-specific.
- [127] Tenstorrent. Wormhole B0 ISA: Unpacker Format Conversion. <https://github.com/tenstorrent/tt-isa-documentation/blob/main/WormholeB0/TensixTile/TensixCoproprocessor/Unpackers/FormatConversion.md>, 2026. Moving source reference recorded in 6 October 2026 review; Wormhole-specific.
- [128] Tenstorrent. Device 2.0 Data Movement API Migration Guide. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/data_movement/device_api_migration_guide.html, 2026. Documentation accessed 7 October 2026.
- [129] Tenstorrent. Metalium Kernel API: noc_semaphore_inc. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/data_movement/noc_semaphore_inc.html, 2026. Documentation accessed 7 October 2026.
- [130] Tenstorrent. Basic Ethernet guide. Author-maintained TT-Metal repository, 2026. URL https://github.com/tenstorrent/tt-metal/blob/main/tech_reports/EthernetMultichip/BasicEthernetGuide.md. Mutable main branch checked 7 October 2026; cited flow-control discussion scoped to Wormhole.
- [131] Tenstorrent. TT-Metalium NOC Debug Dump. TT-Metalium developer documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tools/noc_debug_dump.html. Live documentation checked 7 October 2026; experimental checker with timing and overhead limitations.
- [132] Tenstorrent. TT-Metalium: Ordering (NOC, CMD_BUF, VC). TT-Metalium developer documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/data_movement/ordering.html. Live documentation checked 7 October 2026; stated NoC operation scope.
- [133] Tenstorrent. TT-Metalium Watcher. TT-Metalium developer documentation, 2026. URL <https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tools/watcher.html>. Live documentation checked 7 October 2026; hardware-exception PC/cause reporting marked Quasar only.
- [134] Tenstorrent. TT-Metalium circular buffer apis. TT-Metalium developer documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/circular_buffers.html. Live documentation checked 7 October 2026; bounded producer/consumer model.

- [135] Tenstorrent. TT-Metalium cb_pop_front. TT-Metalium kernel API documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/cb_pop_front.html. Live documentation checked 7 October 2026; consumer release contract.
- [136] Tenstorrent. TT-Metalium cb_reserve_back. TT-Metalium kernel API documentation, 2026. URL https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/cb_reserve_back.html. Live documentation checked 7 October 2026; capacity poll and pointer behavior.
- [137] Tenstorrent. Compute Engines and Data Flow within Tensix. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/compute_engines_and_dataflow_within_tensix.html, n.d.. Mutable latest documentation; accessed 6 October 2026.
- [138] Tenstorrent. TT Metalium reduce tile reference. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/compute/reduce_tile.html, n.d.. Mutable latest documentation; retrieved 7 October 2026.
- [139] Tenstorrent. Basic Ethernet Guide. https://github.com/tenstorrent/tt-metal/blob/main/tech_reports/EthernetMultichip/BasicEthernetGuide.md, n.d.. Wormhole and Blackhole scope; mutable main branch; retrieved 7 October 2026.
- [140] Tenstorrent. Device 2.0 Data Movement API Migration Guide. https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/data_movement/device_api_migration_guide.html, n.d.. Mutable latest documentation; accessed 6 October 2026.
- [141] Tenstorrent. Ordering NOC CMD BUF VC. https://docs.tenstorrent.com/tt-metal/v0.70.1/tt-metalium/tt_metal/apis/kernel_apis/data_movement/ordering.html, n.d.. TT Metalium 0.70.1; retrieved 7 October 2026.
- [142] Tenstorrent. Hello world data movement kernel. https://github.com/tenstorrent/tt-metal/blob/main/tt_metal/programming_examples/hello_world_datamovement_kernel/hello_world_data_movement.md, n.d.. Mutable main branch; accessed 6 October 2026.
- [143] Tenstorrent contributors. SFPU exp on the math thread serializes with the matmuls in the fp32_dest_acc_en SDPA kernel; the streaming kernel issues the exp from the pack thread. tt-metal issue 55759, 2026. URL <https://github.com/tenstorrent/tt-metal/issues/55759>. Implementation investigation, not a universal hardware result. Accessed 7 October 2026.
- [144] The JAX Authors. Writing TPU kernels with Pallas. JAX documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/details.html>. Accessed 7 October 2026.
- [145] The JAX Authors. TPU Pipelining. JAX Pallas documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/pipelining.html>. Accessed 7 October 2026.
- [146] The JAX Authors. Scalar prefetch and block-sparse computation. JAX Pallas documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/sparse.html>. Accessed 7 October 2026.
- [147] The JAX Authors. `jax.experimental.pallas.tpu.make_async_copy`. JAX API documentation, 2026. URL https://docs.jax.dev/en/latest/autosummary/jax.experimental.pallas.tpu.make_async_copy.html. Accessed 7 October 2026.

- [148] The JAX Authors. Distributed computing in Pallas for TPUs. JAX documentation, 2026. URL <https://docs.jax.dev/en/latest/pallas/tpu/distributed.html#run-ahead-and-race-conditions>. Run-ahead and race conditions. Accessed 7 October 2026.
- [149] The OpenXLA Authors. Memory space assignment: Values, use segments, and allocation sequences. OpenXLA source repository, 2026. URL https://github.com/openxla/xla/blob/82e6c3119a9463bfc27add150c8ae8b12c2ca405/xla/service/memory_space_assignment/memory_space_assignment.h. Commit 82e6c3119a9463bfc27add150c8ae8b12c2ca405, memory_space_assignment.h. Pinned source read 7 October 2026.
- [150] Triton Contributors. Gluon Tutorial: Tensor Layouts. <https://triton-lang.org/main/getting-started/tutorials/gluon/layouts.html>, 2026. Documentation accessed 7 October 2026.
- [151] Triton contributors. Fused softmax tutorial. <https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html>, n.d. Mutable tutorial; accessed 6 October 2026.
- [152] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. arXiv:2603.05451, version 1, 2026. URL <https://arxiv.org/html/2603.05451v1>.